

数据库系统

课 程 期 末 梳 理

邱日宏

June 11, 2023



01

备考建议

数据库期末考试怎么考？



- 选择题+大题



- 大题知识点和题型每年相似，可以参考历年情况进行重点复习梳理

- 选择题知识点较为零碎，可借助A4纸辅助记忆

- 关注老师最后一次课程的复习和回顾

往年经验仅供参考，具体以今年为准



数据库系统课程 重点内容



- 关系模型和SQL语句
 - E-R模型（E-R图设计与绘制）
 - 关系范式（正则覆盖、函数闭包、BCNF范式）
 - 索引（B+树，插入、删除、计算）
 - 查询处理（选择、连接、代价估计）
 - 并发控制（前驱图、冲突可串行）
 - 错误恢复算法（基于日志的、Aries错误恢复）
- 往年经验仅供参考，具体以今年为准**



02

关系代数

关系型数据库

- 关系型数据库是一系列表的集合
- 一张表是一个基本单位
- 表中的一行表示一条关系

Entity set and relationship set $\leftrightarrow$ real world

Relation --- table, tuple --- row $\leftrightarrow$ machine world

关系型数据库

表 (table)、关系 (relation)、
元组 (tuple)、属性 (attribute)

- **表中的一行**代表了一组值之间的**关系 (relation)**。
- **关系 (relationship)**：用来指代**表**。
- **元组 (tuple)**：用来指代**行**。
- **属性 (attribute)** 就用来指代**列**。
- **关系实例 (relation instance)**：一个关系的特定实例，就是一**组特定的行**。
- **域 (domain)**：每个属性所允许的**取值的集合**，称为该属性的域。
- **原子的 (atomic)**：如果域中元素被看做是**不可再分的单元**，则域是原子的。
- **空值 (null)**：是一种特殊的值，表示值**未知或者不存在**。

关系型数据库

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
12121	Wu	Finance	90000
22222	Einstein	Physics	95000
33456	Gold	Physics	87000
83821	Brandt	Comp. Sci.	92000

- **表中的一行**代表了一组值之间的**关系 (relation)**。
- **关系 (relationship)**：用来指代**表**。
- **元组 (tuple)**：用来指代**行**。
- **属性 (attribute)** 就用来指代**列**。
- **关系实例 (relation instance)**：一个关系的特定实例，就是一**组特定的行**。
- **域 (domain)**：每个属性所允许的**取值的集合**，称为该属性的域。
- **原子的 (atomic)**：如果域中元素被看做是**不可再分的单元**，则域是原子的。
- **空值 (null)**：是一种特殊的值，表示值**未知或者不存在**。

关系的性质

- Tuples的顺序是无关的
- 关系中~~没有重复的~~tuple
- 属性的值满足原子性

码 key

- **超码** super key:
 - 一个或多个属性的集合，这些属性的组合可以**唯一标识一个元组**。超码的任意超集也是超码。
- **候选码** candidate key:
 - **最小的超码**。如果该超码的任意真子集都不能成为超码，那么该集合就是候选码。
- **主码** primary key:
 - 代表被数据库设计者**选中的**，主要用来在一个关系中区分不同元组的候选码。
 - 主码应该选择那些值从不或者很少发生变化的属性。

码 key

- 外码 foreign key:
 - 一个关系模式 (r_1) 可能在它的属性中包括另一个关系模式 (r_2) 的主码。
 - 这个属性在关系模式 r_1 上被称作参照 r_2 的外码。
 - 关系 r_1 称作外码依赖的参照关系 (referencing relation)
 - 关系 r_2 称作外码的被参照关系 (referenced relation)

E.g.1: 学生(学号, 姓名, 性别, 专业号, 年龄) --- 参照关系
 专业(专业号, 专业名称) --- 被参照关系 (目标关系)
其中属性 专业号 称为关系 学生的 外码.

码 key

- **外码 foreign key:**
 - 一个关系模式 (r_1) 可能在它的属性中包括另一个关系模式 (r_2) 的主码。
 - 这个属性在关系模式 r_1 上被称作参照 r_2 的外码。
- **参照完整性约束 referential integrity constraint:**
 - 在参照关系中任意元组在特定属性上的取值必然等于被参照关系中某个元组在特定属性上的取值。

关系运算

- 基本运算：
 - Select 选择
 - Project 投影
 - Union 并
 - set difference 差 (集合差)
 - Cartesian product 笛卡儿积
 - Rename 改名 (重命名)

关系运算

- Select 选择:

Relation $r =$

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$\sigma_{A=\beta \wedge D>5}(r) =$

A	B	C	D
β	β	23	10

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

关系运算

- Project 投影:

Relation $r =$

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$\Pi_{A,C}(r) =$

A	C
α	1
α	1
β	1
β	2



A	C
α	1
β	1
β	2

$\Pi_{A,C}(r) = ?$

$\Pi_{A1, A2, \dots, Ak}(r)$

关系运算

- Union 并:

Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r \cup s$:

A	B
α	1
α	2
β	1
β	3

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

关系运算

- Set Difference 差:

Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r - s$:

A	B
α	1
β	1

$$r - s = \{t \mid t \in r \text{ and } t \notin s\}$$

关系运算

- Cartesian-Product 笛卡尔积:

Relations r, s :

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

$r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

$$r \times s = \{\{t \ q\} \mid t \in r \text{ and } q \in s\}$$

关系运算

- rename 改名:

$$\rho_x(E)$$

$$\rho_{x(A_1, A_2, \dots, A_n)}(E)$$

(对relation E 及其attributes都重命名)

关系运算

- 基本运算
- Additional Relational-Algebra Operation
 - 可以由基本运算推出
- Extended Relational-Algebra Operation
 - 可以由assign语句推出

关系运算

- Additional Relational-Algebra Operation
 - Set intersection 交
 - Natural join 自然连接
 - Division 除
 - Assignment 赋值

关系运算

- Additional Relational-Algebra Operation
 - Set intersection 交

Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$$r \cap s = \{t \mid t \in r \text{ and } t \in s\}$$

$$r \cap s = r - (r - s)$$

$r \cap s =$

A	B
α	2

关系运算

- Additional Relational-Algebra Operation
 - Natural Join 自然连接

Example: $R = (A, B, C, D)$, $S = (B, D, E)$

- Result schema of the natural-join of r and $s = (A, B, C, D, E)$
- $r \bowtie s = \Pi_{r.A, r.B, r.C, r.D, s.E}(\sigma_{r.B = s.B \wedge r.D = s.D}(r \times s))$

关系运算

- Additional Relational-Algebra Operation

- Natural Join 自然连接

Relations r, s :

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
δ	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

$r \bowtie s =$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
δ	1	γ	a	α
δ	1	γ	a	γ
δ	2	β	b	δ

注意: (1) r, s 必须含有共同属性(名和域都对应相同); (2) 连接二个关系中同名属性值相等的元组; (3) 结果属性是二者属性集的并集, 但消去重名属性.

拓展:

- 左连接
- 右连接

关系运算

- Additional Relational-Algebra Operation
 - Division 除法 —— 关键词 “for all”

Let r and s be relations on schemas R and S , respectively, where $R = (A_1, \dots, A_m, B_1, \dots, B_n)$ and $S = (B_1, \dots, B_n)$. Then, the result of $r \div s$ is a relation on the schema $R - S = (A_1, \dots, A_m)$ and $r \div s = \{t \mid t \in \Pi_{R-S}(r) \wedge \forall u \in s(tu \in r)\}$.

关系运算

- Additional Relational-Algebra Operation

- Division 除法 —— 关键词 “for all”

例：查询选修了所有课程的学生的学号。

enrolled	Sno	Cno
	95001	1
	95001	2
	95001	3
	95002	2
	95002	3

÷

course	Cno
	1
	2
	3

=

Sno
95001

$$\Pi_{Sno, Cno}(enrolled) \div \Pi_{Cno}(course)$$

Enrolled(sno, cno, grade)
Course(cno, cname, credits)

关系运算 • Additional Relational-Algebra Operation

• Division 除法 —— 关键词 “for all”

Relations r, s :

A	B	C	D	E
α	a	α	a	1
α	b	γ	a	1
α	b	γ	b	1
β	a	γ	a	1
γ	a	γ	c	2
β	a	γ	b	3
γ	a	γ	a	1
γ	a	γ	b	1
γ	c	β	b	1

r

D	E
a	1
b	1

s

$r \div s =$

A	B	C
α	b	γ
γ	a	γ

r :

	A	B	C	D	E	
1						
2	α	a	α	a	1	✓
3	α	b	γ	a	1	
4	α	b	γ	b	1	
5	β	a	γ	a	1	
6	β	a	γ	b	3	
7	γ	a	γ	c	2	✓
8	γ	a	γ	a	1	
9	γ	a	γ	b	1	
10	γ	c	β	b	1	

关系运算

- Additional Relational-Algebra Operation
 - Assignment 赋值

Write $r \div s$ as

$$temp1 \leftarrow \Pi_{R-S}(r)$$

$$temp2 \leftarrow \Pi_{R-S}((temp1 \times s) - \Pi_{R-S,S}(r))$$

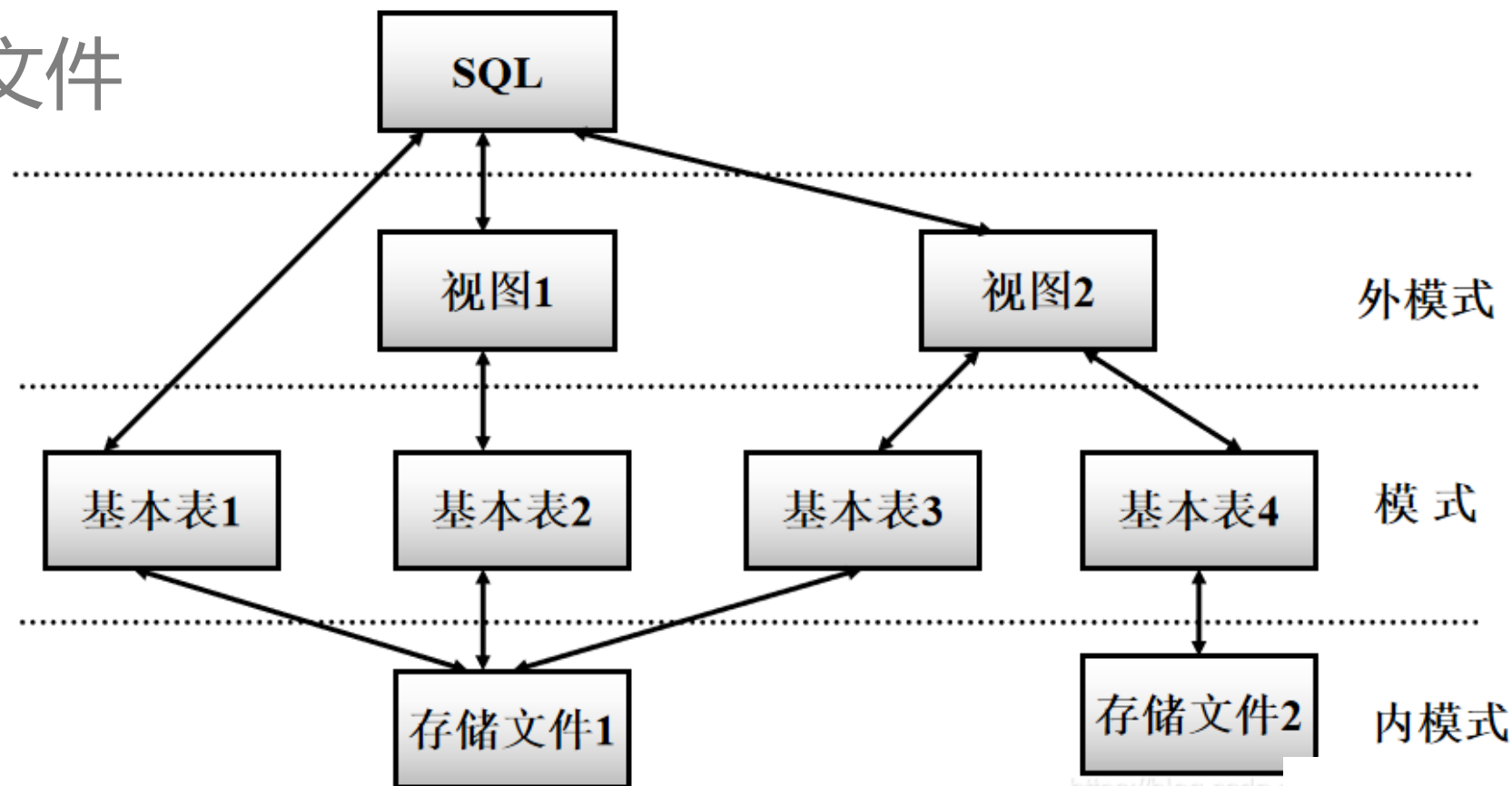
$$result = temp1 - temp2$$



03

S Q L 语 句

- 外模式：视图与部分基本表
- 模式：基本表
- 内模式：存储文件



SQL操作类别

- 数据定义语言 (DDL)
 - Create, alter, drop
- 数据操作语言 (DML)
 - Select, insert, delete, update
- 数据控制语言 (DCL)
 - Grant, revoke

SQL操作类别

- 数据定义语言 (DDL)
 - Create, alter, drop
- 数据操作语言 (DML)
 - Select, insert, delete, update
- 数据控制语言 (DCL)
 - Grant, revoke

操 作 对 象	操 作 方 式		
	创 建	删 除	修 改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视 图	CREATE VIEW	DROP VIEW	
索 引	CREATE INDEX	DROP INDEX	

Data Definition Language

```
CREATE TABLE <表名>(
    <列名> <数据类型>[ <列级完整性约束条件> ]
    [, <列名> <数据类型>[ <列级完整性约束条件>] ]
    .....
    [, <表级完整性约束条件> ]
);
```

Data Definition Language

```
ALTER TABLE <表名>  
    [ ADD <新列名> <数据类型> [ 完整性约束 ] ]  
    [ DROP <完整性约束名> ]  
    [ ALTER COLUMN <列名> <数据类型> ];
```

Data Definition Language

DROP TABLE <表名> [RESTRICT| CASCADE];

- RESTRICT：删除表是有限制的
 - 欲删除的基本表不能被其他表的约束所引用
 - 如果存在依赖该表的对象，则此表不能被删除
- CASCADE：删除该表没有限制。
 - 在删除基本表的同时，相关的依赖对象一起删除

Data Definition Language

```
CREATE UNIQUE INDEX Stusno ON Student(Sno);  
/*这里Stusno就是索引名*/
```

Data Definition Language

```
ALTER INDEX SCno RENAME SCSno;
```

Data Definition Language

```
DROP INDEX Stusname;
```

Data Definition Language

Example: Declare branch_name as the primary key for *branch* and ensure that the values of *assets* are non-negative.

Method 1:

```
CREATE TABLE branch
(branch_name char(20) not null,
branch_city char(30),
assets integer,
primary key (branch_name),
check (assets >= 0));
```

Method 2:

```
CREATE TABLE branch2
(branch_name char(20)
primary key,
branch_city char(30),
assets integer,
check (assets >= 0));
```

Data Manipulation Language

```
SELECT A1, A2, ..., An  
      FROM r1, r2, ..., rm  
      WHERE P ;
```

注意点:

- SQL是大小写不敏感的, 大小写都是正确的写法
- Distinct关键字可以去除重复字段
- Select * 表示选择所有属性
- Select属性中可以有表达式
 - 如select amount * 10 from loan

Data Manipulation Language

```
SELECT A1, A2, ..., An  
FROM r1, r2, ..., rm  
WHERE P ;
```

选择语句：

- Where语句后指明了选择的条件
- From 会将各表做笛卡尔积
 - 多表中包含同一名称的字段需要指明表名

```
SELECT customer_name, borrower.loan_number, amount  
FROM borrower, loan  
WHERE borrower.loan_number = loan.loan_number and  
      branch_name = 'Perryridge'
```

The prefix is necessary.

Data Manipulation Language

Old name AS new name

选择语句：

- As可以为表或者字段做重命名
 - 同一张表需要在两处出现时非常有用

例题：

Find the names of all branches that have greater assets than **some branch** located in city Brooklyn.

```
branch(branch-name, branch-city, assets)
```

Data Manipulation Language

Old name AS new name

例题:

Find the names of all branches that have greater assets than *some branch* located in city Brooklyn.

```
SELECT distinct T.branch_name  
FROM branch as T, branch as S  
WHERE T.assets > S.assets and S.branch_city = 'Brooklyn'
```

branch(branch-name, branch-city, assets)

Data Manipulation Language

Old name AS new name

字符串操作:

- %: 匹配任意字符串
- _: 匹配任意单个字符
-

Data Manipulation Language

```
SELECT A1, A2, ..., An  
FROM r1, r2, ..., rm  
WHERE P ;
```

选择语句：

- ORDER BY 语句可以对字段进行排序
 - Order by <XXX> 默认升序
 - Order by <XXX> desc 降序排序

The prefix is necessary.

```
SELECT customer_name, borrower.loan_number, amount  
FROM borrower, loan  
WHERE borrower.loan_number = loan.loan_number and  
branch_name = 'Perryridge'
```

Data Manipulation Language

集合操作:

UNION

- (SELECT column_list
 - FROM table_1)
- UNION
- (SELECT column_list
 - FROM table_2

UNION和UNION ALL的ORDER BY子句要写在最后一条SELECT语句后面!

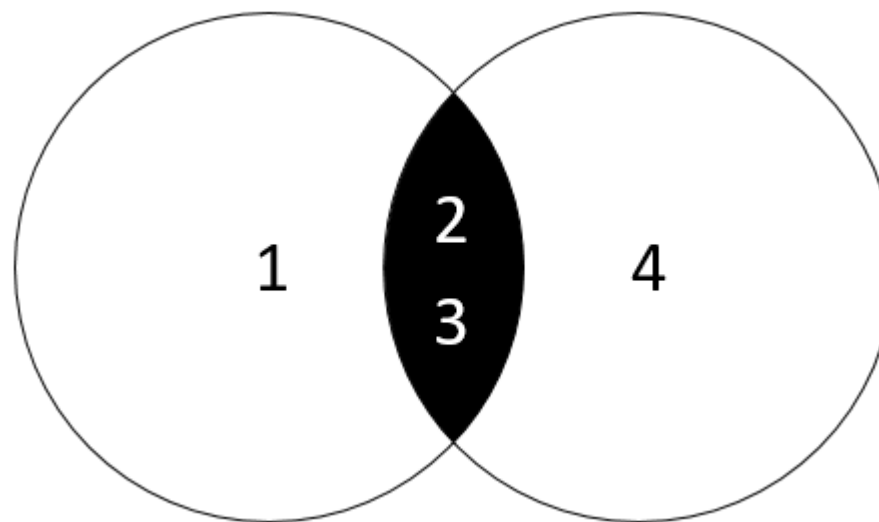
虽然ORDER BY似乎只是最后一条SELECT语句的组成部分,但实际上MySQL将用它来排序所有SELECT语句返回的所有结果。

Data Manipulation Language

集合操作:

INTERSECT >MySQL 8.0.31

- (SELECT column_list
• FROM table_1)
- INTERSECT
- (SELECT column_list
- FROM table_2

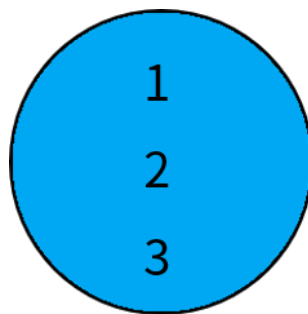


Data Manipulation Language

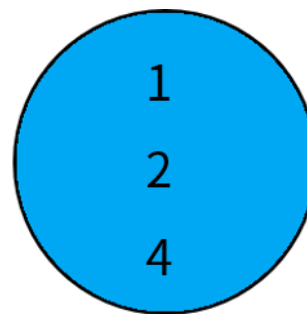
集合操作:

EXCEPT >MySQL 8.0.31

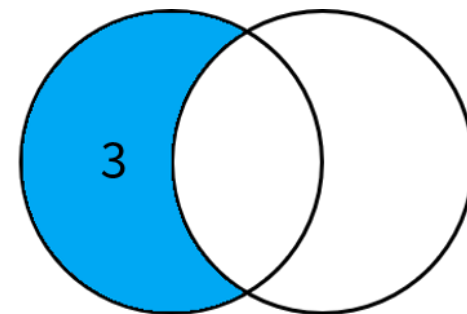
- (SELECT column_list
• FROM table_1)
- EXCEPT
- (SELECT column_list
- FROM table_2



query1



query2



query1 EXCEPT query2

插入、删除和更新



- 插入: `insert into table_name values( );`
 - 可以用select查询子句得到的结果作为values, 此时可以同时插入多条结果
- 删除: `delete from table_name where xxxxxx`
- 更新: `update table_name set xxx where xxxxx`



聚合函数

- Avg(col)
- Min(col)
- Max(col)
- Sum(col)
- Count(col)



聚合函数

本身并不难；关键在使用

```
SELECT avg(balance) avg_bal  
FROM account  
WHERE branch_name = 'Perryridge'
```

如果想要达到下面的效果呢？

<u>Branch_name</u>	<u>avg_bal</u>
Perryridge	25000



聚合函数

聚合函数配合GROUP BY

```
SELECT branch_name, avg(balance) avg_bal  
FROM account  
GROUP BY brach_name
```

<u>Branch_name</u>	<u>avg_bal</u>
Downtown	600
Mianus	725
Round Hill	350
Redwood	700

如果想要使avg满足一定的条件呢?



聚合函数

GROUP BY配合HAVING

```
SELECT A.branch_name, avg(balance)
FROM account A, branch B
WHERE A.branch_name = B.branch_name and
      branch_city = 'Brooklyn'
GROUP BY A.branch_name
HAVING avg(balance) > 1200
```



聚合函数知识点

• 格式

SELECT <[DISTINCT] c1, c2, ...>

FROM <r1, ...>

[WHERE <condition>]

[GROUP BY <c1, c2, ...> [HAVING <cond2>]]

[ORDER BY <c1 [DESC] [, c2 [DESC|ASC], ...]>]

执行顺序: from->where->group by->having->select->distinct->order by
聚合函数不能直接用在where语句中!



SQL中的Null

- SQL里的三值逻辑

OR: (*unknown or true*) = *true*

(*unknown or false*) = *unknown*

(*unknown or unknown*) = *unknown*

AND: (*true and unknown*) = *unknown*

(*false and unknown*) = *false*

(*unknown and unknown*) = *unknown*

NOT: (*not unknown*) = *unknown*

- Where语句中谓词unknown最终被作为false处理



SQL中的Null

- SQL里null的使用
 - Var is null
 - Var is not null



聚合函数与Null

- AVG()、MAX()、MIN()、SUM()函数忽略列值为NULL的行。
- COUNT()函数有两种情况：
 - 1.使用COUNT(column)对特定列中具有值的行进行计数，忽略NULL值。
 - 2.使用COUNT(*)对表中行的数目进行计数，不管表列中包含的是空值（NULL）还是非空值。



聚合函数与Null

- 看个例子:

```
CREATE TABLE student2(  
    id INT,  
    sname VARCHAR(20),  
    address VARCHAR(20)  
);  
  
INSERT INTO student2 VALUES (NULL,NULL,NULL);  
  
1.      SELECT COUNT(id) FROM student2; //结果为0。  
2.      SELECT COUNT(*) FROM student2; //结果为1。
```



嵌套查询

• In

- SELECT distinct customer_name
FROM borrower
WHERE customer_name in (SELECT customer_name
FROM depositor)

• Not in

- SELECT distinct customer_name
FROM borrower
WHERE customer_name not in (SELECT customer_name
FROM depositor)



嵌套查询

嵌套查询

- Some、Any、All、…… (其实它们都可以有其他的实现方式)

(= some) $\equiv$ in

However, ($\neq$ some) $\neq$ not in

$$(5 < \text{some} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{true}$$

$$(5 < \text{some} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

$$(5 = \text{some} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$$

$$(5 \neq \text{some} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$$

($\neq$ all) $\equiv$ not in

However, (= all) $\neq$ in

$$(5 < \text{all} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{false}$$

$$(5 < \text{all} \begin{array}{|c|} \hline 6 \\ \hline 10 \\ \hline \end{array}) = \text{true}$$

$$(5 = \text{all} \begin{array}{|c|} \hline 4 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

$$(5 \neq \text{all} \begin{array}{|c|} \hline 4 \\ \hline 6 \\ \hline \end{array}) = \text{true}$$

嵌套查询

- 【例题】找到每一家银行中余额balance最多的账户

account(account-number, branch-name, balance)



嵌套查询

- 【例题】找到每一家银行中余额balance最多的账户

account(account-number, branch-name, balance)

- 先看几个错误的做法

```
SELECT account_number, balance
FROM account
WHERE balance >= max(balance)
GROUP BY branch_name
```

```
SELECT account_number, max(balance)
FROM account
GROUP BY branch_name
```

执行顺序: *from->where->group by->having->select->distinct->order by*



嵌套查询

- 【例题】找到每一家银行中余额balance最多的账户

account(account-number, branch-name, balance)

- 一个正确的示例:

```
SELECT account_number AN, balance
FROM account A
WHERE balance >= (SELECT max(balance)
                   FROM account B
                   WHERE A.branch_name = B.branch_name)
ORDER by balance
```



case语句



1. 简单函数

CASE [col_name] WHEN [value1] THEN [result1]...ELSE [default] END

2. 搜索函数

CASE WHEN [expr] THEN [result1]...ELSE [default] END

case语句



1. 简单函数 -> 枚举这个字段所有可能的值, 行列互换

CASE [col_name] WHEN
[value1] THEN
[result1]...ELSE [default] END

```
# 行列互换
select 学号,
max(case 课程号 when '0001' THEN 成绩 else 0 END) as `课程0001`,
max(case 课程号 when '0002' THEN 成绩 else 0 END) as `课程0002`,
max(case 课程号 when '0003' THEN 成绩 else 0 END) as `课程0003`
from score
GROUP BY 学号
```

164 GROUP BY 学号
165 原表

信息	结果1	概况	状态
学号	课程号	成绩	
0001	0001	80	
0001	0002	90	
0001	0003	99	
0002	0002	60	
0002	0003	80	

转换后的表

信息	结果1	概况	状态
学号	课程0001	课程0002	课程0003
0001	80	90	99
0002	0	60	80
0003	80	80	80

case语句



```
UPDATE account
```

```
SET balance = case
```

```
    when balance <= 10000
```

```
    then balance * 1.05
```

```
    else  balance * 1.06
```

```
end
```

2. 搜索函数

```
CASE WHEN [expr] THEN  
[result1]...ELSE [default] END
```


视图



- 视图：一种只显示数据表中**部分属性值**的机制

- 不会在数据库中创建新的表

- 只是隐藏了部分数据

- 优点：

- 安全

- 简单易用，逻辑独立

```
CREATE VIEW <v_name> AS
```

```
SELECT c1, c2, ... From ...
```

```
CREATE VIEW <v_name> (c1, c2, ...) AS
```

```
SELECT e1, e2, ... FROM ...
```


视图



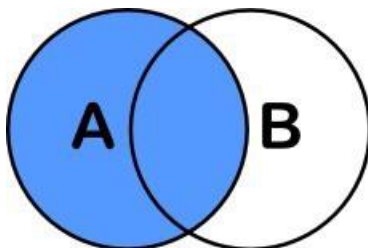
- View是**虚表**，对其进行的所有操作都将转化为对基表的操作

视图的更新

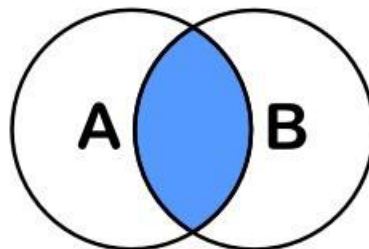


- 使用insert、update等语句更新视图
- 条件：
 - 创建时只使用了一张表的数据
 - 创建时没有进行distinct和聚合操作
 - 没有出现空值和default

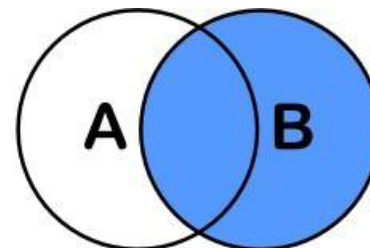
Join



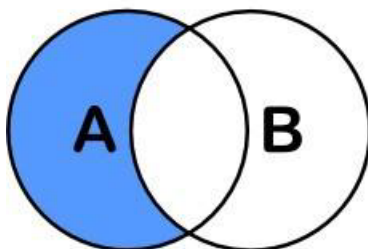
```
SELECT <auswahl>
FROM tabelleA A
LEFT JOIN tabelleB B
ON A.key = B.key
```



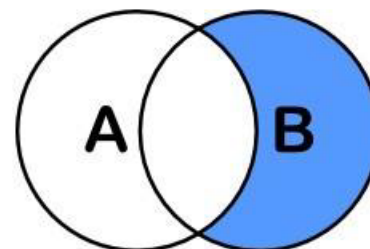
```
SELECT <auswahl>
FROM tabelleA A
INNER JOIN tabelleB B
ON A.key = B.key
```



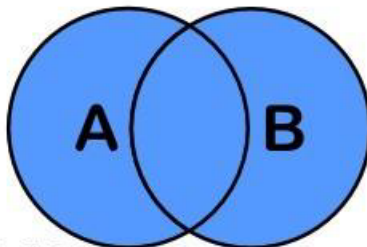
```
SELECT <auswahl>
FROM tabelleA A
RIGHT JOIN tabelleB B
ON A.key = B.key
```



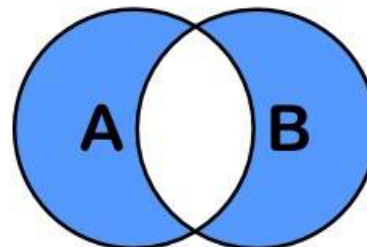
```
SELECT <auswahl>
FROM tabelleA A
LEFT JOIN tabelleB B
ON A.key = B.key
WHERE B.key IS NULL
```



```
SELECT <auswahl>
FROM tabelleA A
RIGHT JOIN tabelleB B
ON A.key = B.key
WHERE A.key IS NULL
```



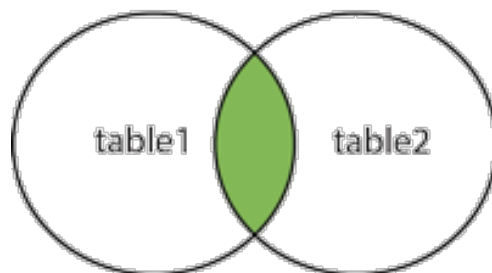
```
SELECT <auswahl>
FROM tabelleA A
FULL OUTER JOIN tabelleB B
ON A.key = B.key
```



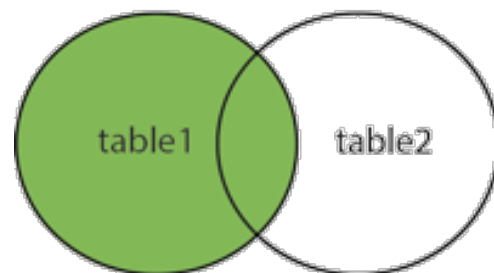
```
SELECT <auswahl>
FROM tabelleA A
FULL OUTER JOIN tabelleB B
ON A.key = B.key
WHERE A.key IS NULL
OR B.key IS NULL
```

Join——MySQL

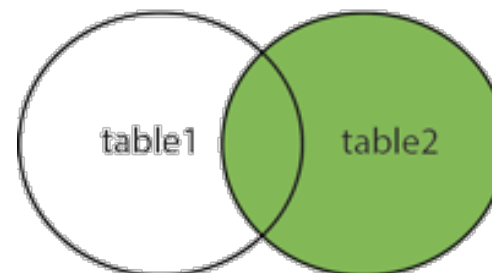
INNER JOIN



LEFT JOIN



RIGHT JOIN





Derived Relations

```
SELECT branch_name, avg_bal
FROM (SELECT branch_name, avg(balance)
      FROM account
      GROUP BY branch_name)
      as result (branch_name, avg_bal)
WHERE avg_bal > 500
```

With 语句

```
WITH max_balance(value) as
      SELECT max(balance)
      FROM account
SELECT account_number
FROM account, max_balance
WHERE account.balance = max_balance.value
```

• 注：这里相当于创建了一个视图再进行操作

SQL的数据类型

- Domain只是约束，而不是强类型

Create domain **Dollars** as numeric(12, 2) **not null**;

Create domain **Pounds** as numeric(12,2);

Create table employee

(*eno* char(10) primary key,

ename varchar(15),

job varchar(10),

salary **Dollars**,

comm **Pounds**);

SQL的大文件存储

- Clob：用于存储大量的文本数据。大字段的操作常常以流的方式处理。
- Blob：用于存储二进制数据，常常为图片或音频。

类型	最大大小
TinyText	255字节
Text	65535字节 (约65K)
MediumText	16 777 215字节 (约16M)
LongText	4 294 967 295 (约4G)

类型	最大大小
TinyBlob	255字节
Blob	65535字节 (约65K)
MediumBlob	16 777 215字节 (约16M)
LongBlob	4 294 967 295 (约4G)

SQL练习1

Consider the following relational schemas with the primary keys underlined.

Movie(title, type, director)

Comment(title, user_name, grade)

- 1) Write a *relational algebra expression* to find all the movie titles that are directed by “Yimou Zhang” and exist the comment grade of greater than or equal to 4.
- 2) Write a *SQL statement* to change the null value of grade to 0.
- 3) Write a *SQL statement* to find which movies have the highest average grade.
- 4) Write a *SQL statement* to find all the movie titles where every user gives higher grade than movie “the avenger”.

SQL练习1参考答案

1) $\Pi_{\text{Title}}(\sigma_{\text{director}=\text{"Yimou Zhang"}}(\text{movie}) \bowtie \sigma_{\text{grade} \geq 4}(\text{comment}))$

评分细则:

错一处扣 4 分

2) **Update comment set grade=0 where grade is null**

评分细则:

错一处扣 4 分, **grade=null, grade is null** 等类似答案均给分

3) **Select type from movie, comment**

Where movie.title=comment.title

Group by title

Having avg(grade) >=all (Select avg(grade)

From movie, comment

Where movie.title=comment.title

Group by title)

评分细则:

写出 **having**.....且对均给 2 分, 用其他 SQL 语句写出相同效果均给分

4) **Select title from movie**

Except

Select title from movie

Where exists (select *

From comment A, comment B

Where A.title=movie.title and A.user_name = B.user_name

And B.title=' the avenger'

And A.grade <=B.grade)

评分细则:

写出 4 个正确条件给 2 分, 全对给 4 分

SQL练习2

Following relational schemas represent information of a campus card database in a university.

card(cno:char(5), name:char(8), depart:char(10), balance:integer)

pos(pno:char(4), campus:char(8), location:char(10))

**detail(cno:char(5), pno:char(4), cdate:date, ctime:time,
amount:integer,
remark:char(10))**

card

cno	name	depart	balance
c0001	张帅	CS	100
c0002	李丽	EN	200
c0003	王浩	CS	300
c0004	刘萌	CS	400
c0005	赵亮	MA	500

pos

pno	campus	location
p001	玉泉	教育超市
p002	玉泉	四食堂
p003	玉泉	四食堂
p004	紫金港	教育超市
p005	紫金港	教育超市
p006	紫金港	一食堂

detail

cno	pno	cdate	ctime	amount	remark
c0001	p002	2018-07-01	08:10:10	6	餐饮
c0001	p002	2018-07-01	12:05:12	12	餐饮
c0001	p002	2018-07-01	17:30:20	20	餐饮
c0001	p001	2018-07-01	18:10:10	60	购物
c0002	p002	2018-07-02	08:10:10	8	餐饮
c0002	p001	2018-07-02	08:10:10	20	购物
c0003	p003	2018-07-02	08:10:10	25	餐饮

Following relational schemas represent information of a campus card database in a university.

SQL练习2

```
card( cno:char(5), name:char(8), depart:char(10), balance:integer )
pos( pno:char(4), campus:char(8), location:char(10) )
detail( cno:char(5), pno:char(4), cdate:date, ctime:time,
amount:integer,
        remark:char(10) )
```

Given following SQL query on above relations:

```
select cno, name
from card natural join detail
where depart= 'CS' and (cdate , pno) in
(   select cdate, pno
    from detail
    where cno='c0002' )
```

Please answer following questions:

- (1) Transform above query to a SQL statement without nested subquery.
- (2) Transform above query to an equivalent relational algebra expression.
- (3) Write a SQL statement to find out cards consumed in only one campus in 2018.
- (4) Write a SQL statement to find out the pos in “紫金港” campus that has the maximum total amount of card consumption in 2018.
- (5) Write a sequence of SQL statements to complete following transaction:
card “c0002” consumes 20 at pos “p001” at 2018-07-02 08:08:08

SQL练习2参考答案

(1)
select c1.cno, c1.name
from (card as c1) natural join (detail as d1),
detail as d2
where c1.depart= 'CS' and d2.cno ='c0002' and
d1.cdate = d2.cdate and d1.pno=d2.pno;

another answer:

```
select c1.cno, c1.name
from card as c1, detail as d1, detail as d2
where c1.cno=d1.cno and
c1.depart= 'CS' and d2.cno ='c0002'
d1.cdate = d2.cdate and d1.pno=d2.pno;
```

(2) $\Pi_{c1.cno, c1.name} (\sigma_{d1.cdate=d2.cdate \wedge d1.pno=d2.pno} ((\sigma_{c1.depart='CS'} (\rho_{c1}(card))) \bowtie \sigma_{(\rho_{d1}(detail)) \times (\sigma_{d2.cno='c0002'} (\rho_{d2}(detail))) })))$

(3)
select cno
from detail natural join pos
where year(detail.cdate)=2018
group by cno
having count(distinct campus)=1;

another answer:

```
select *
from card c1
where exists(
    select *
    from (detail natural join pos) as r1
    where r1.cno=c1.cno )
and not exists(
    select *
    from (detail natural join pos) as r1,
    (detail natural join pos) as r2
    where r1.cno=c1.cno and r2.cno=c1.cno and
    year(r1.cdate)=2018 and year(r2.cdate)=2018
    and r1.campus<> r2.campus
)
```

SQL练习2参考答案

- (4)
- ```
select pno
from detail natural join pos
where pos.campus='紫金港' and year(detail.cdate)=2018
group by pno
having sum(amount)>=all (
 select sum(amount)
 from detail natural join pos
 where pos.campus='紫金港' and year(detail.cdate)=2018
 group by pno
)
```
- (5)
- ```
update card set balance = balance -20 where cno='c0002';
insert into detail(cno, pno, cdate, ctime, amount)
    values('c0002', 'p001','2018-07-02', '08:08:08', 20);
commit;
```



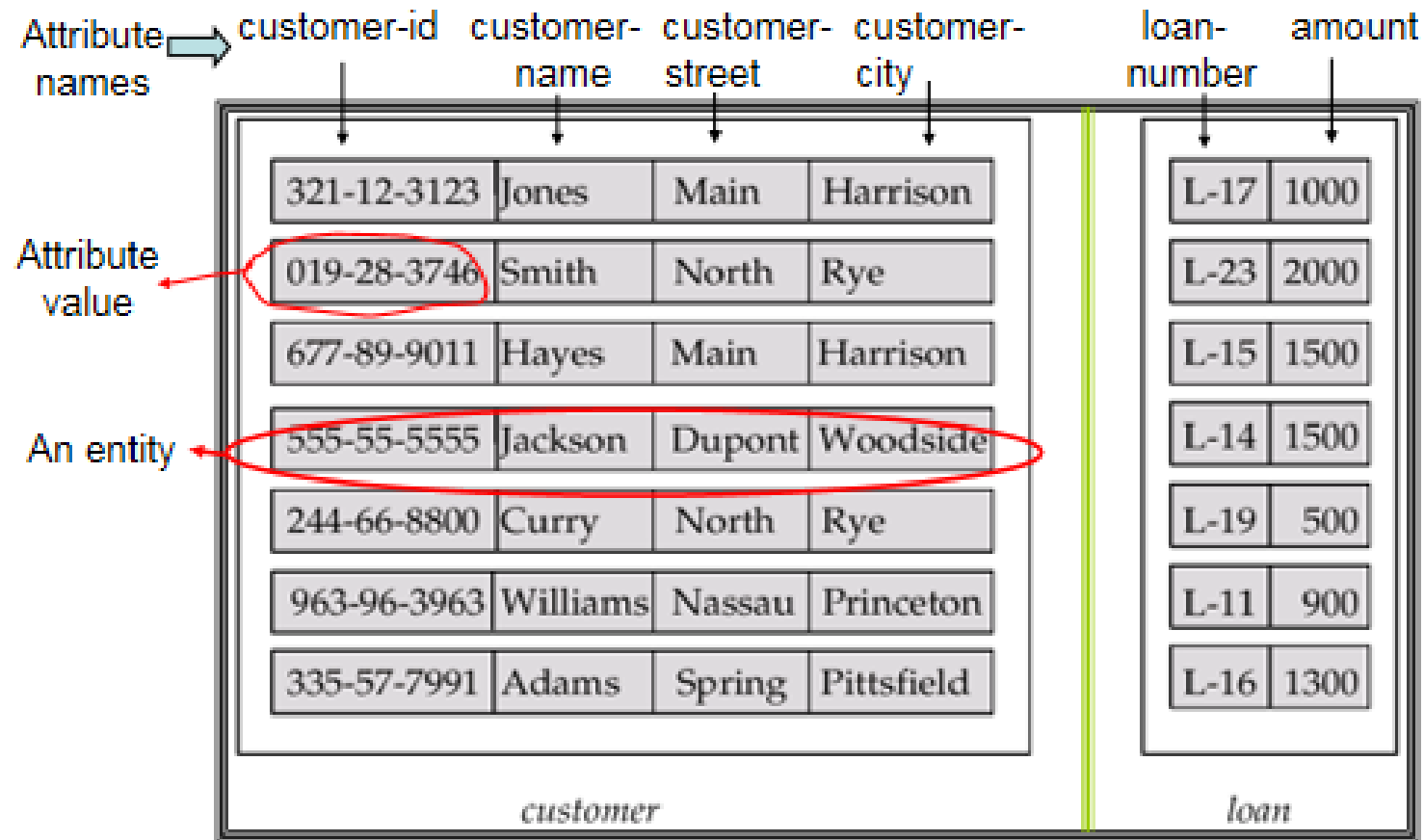

03

E - R 模型

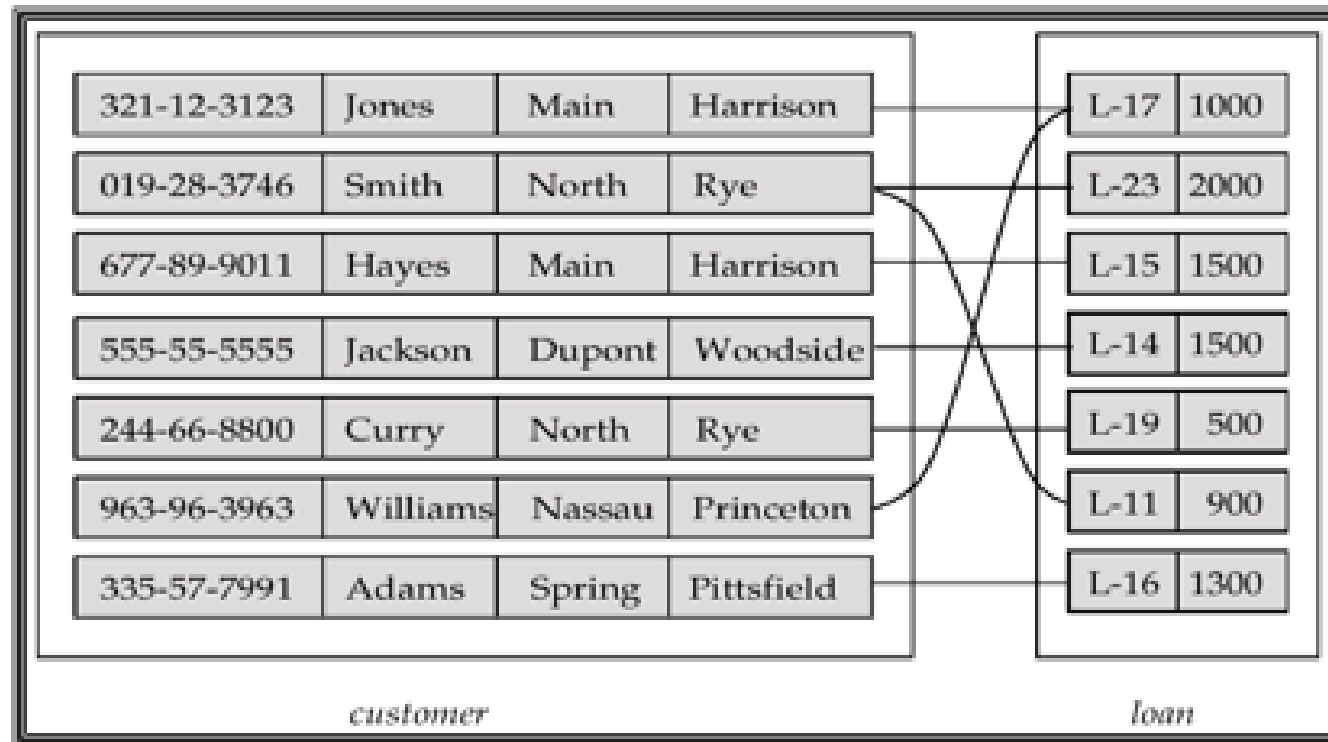
E-R模型

- E-R模型由实体entities和关系relation组成
 - 实体：用于区分其他种类的物体
 - 可以是具体的或抽象的
 - 拥有自身的属性
 - 关系：实体之间的联系
 - 一对一、一对多、多对多

E-R模型——实体的一个例子



E-R模型——关系的一个例子

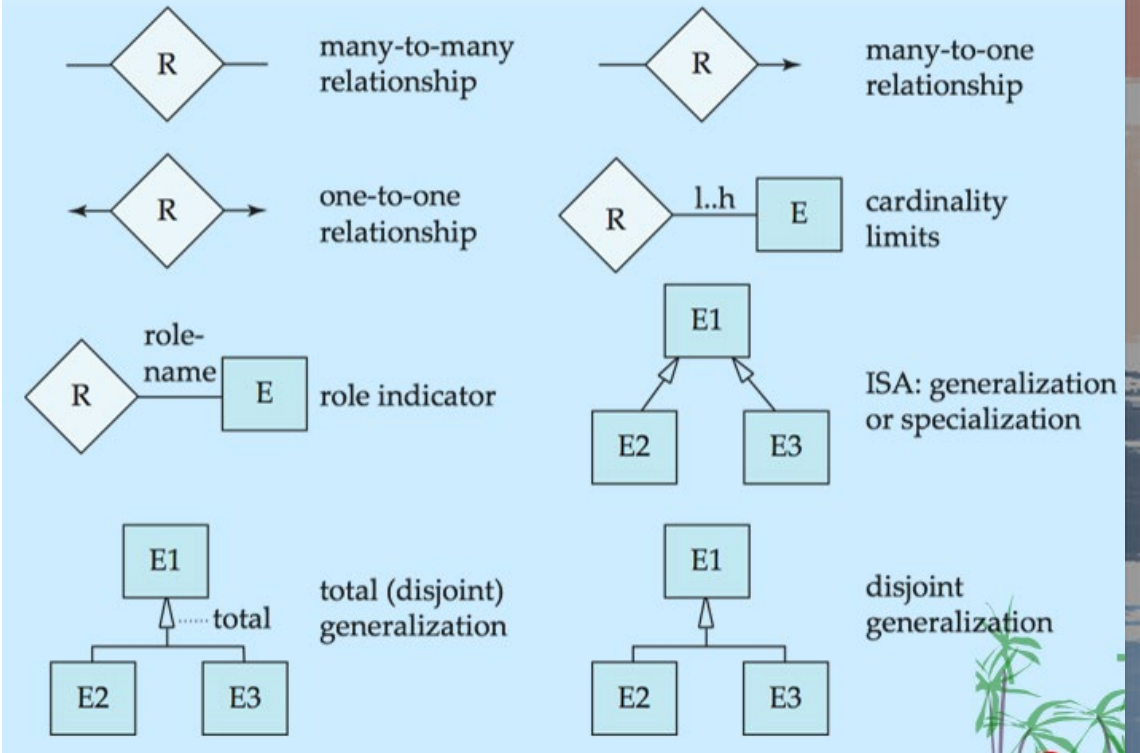
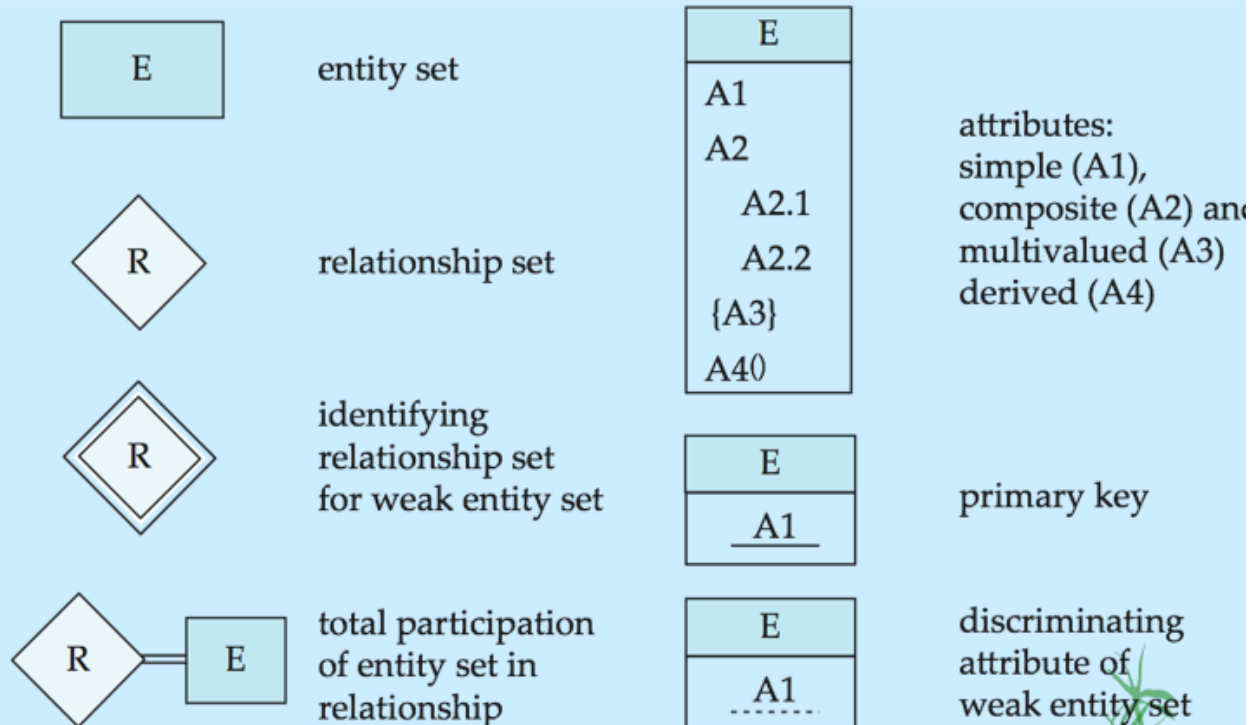


Borrower(customer-name, loan-number)

E-R模型

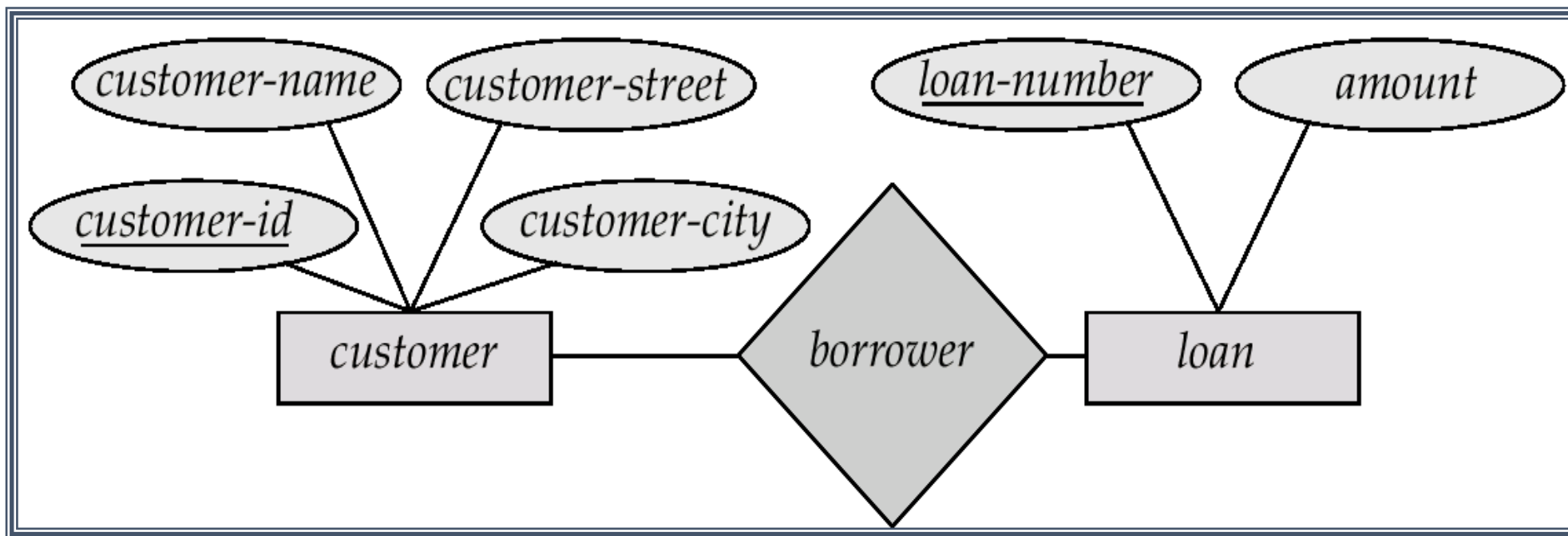
- E-R模型由实体entities和关系relation组成
 - 实体：长方形
 - 属性：椭圆形
 - 关系：连线

E-R图



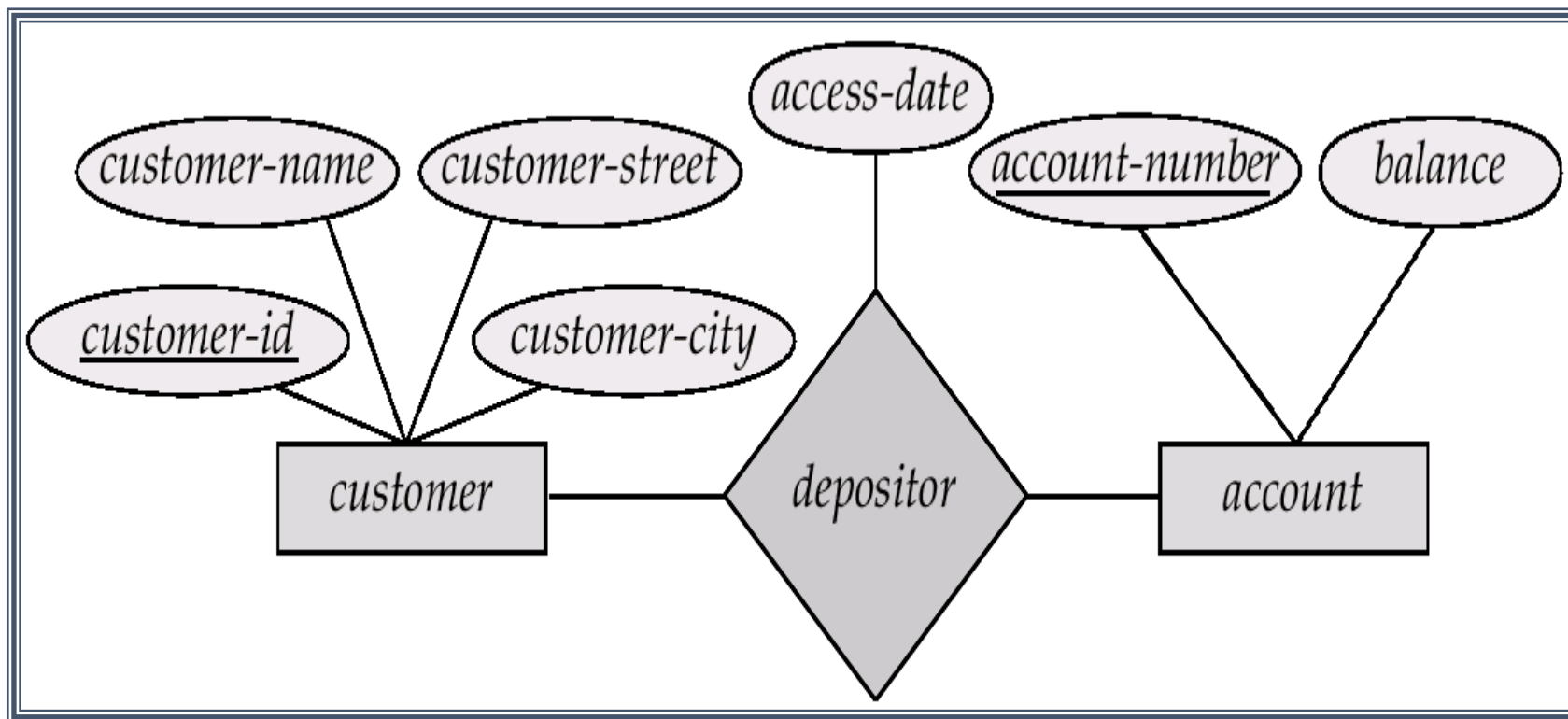
E-R模型

有一点点抽象，我们用例子来说吧

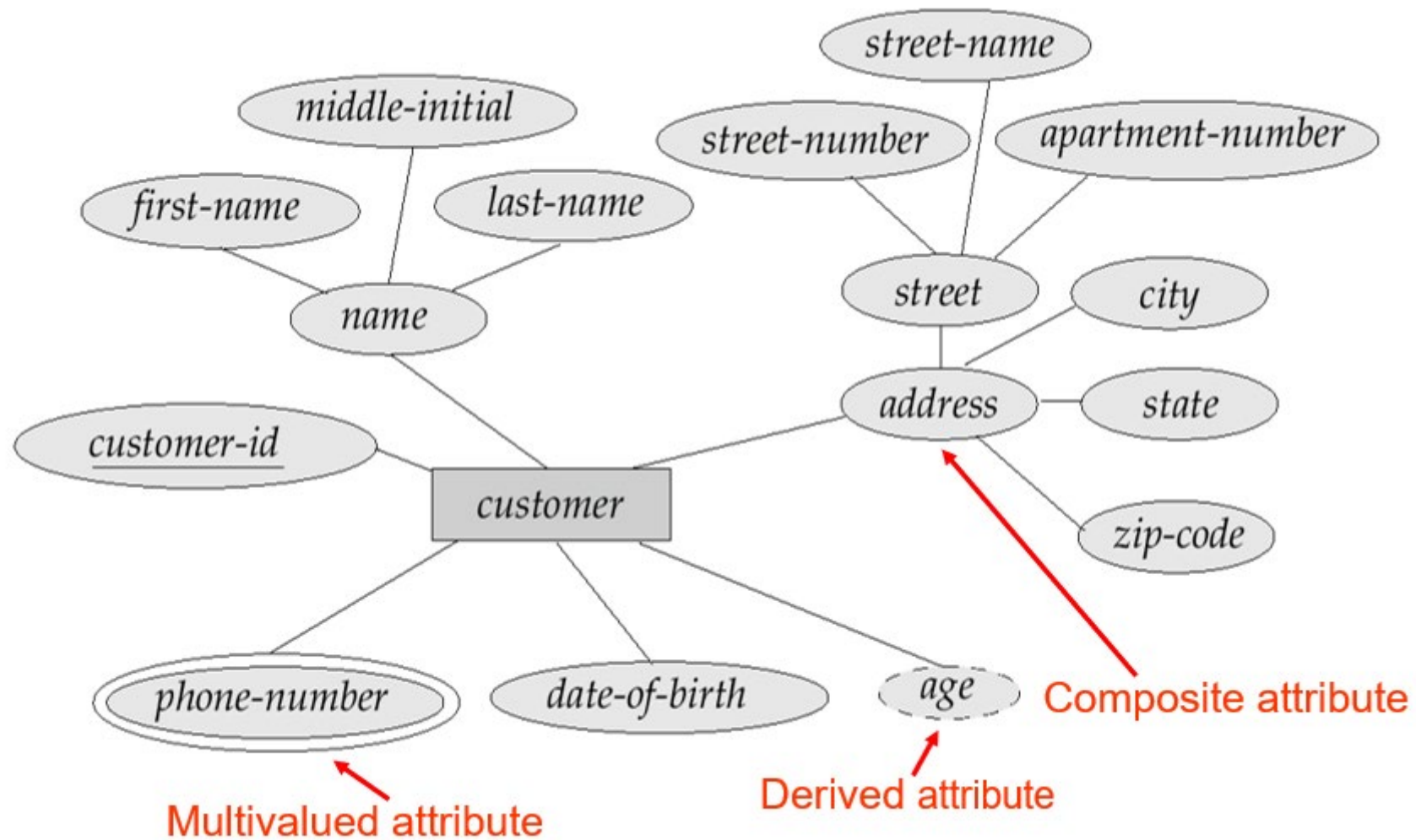


E-R模型

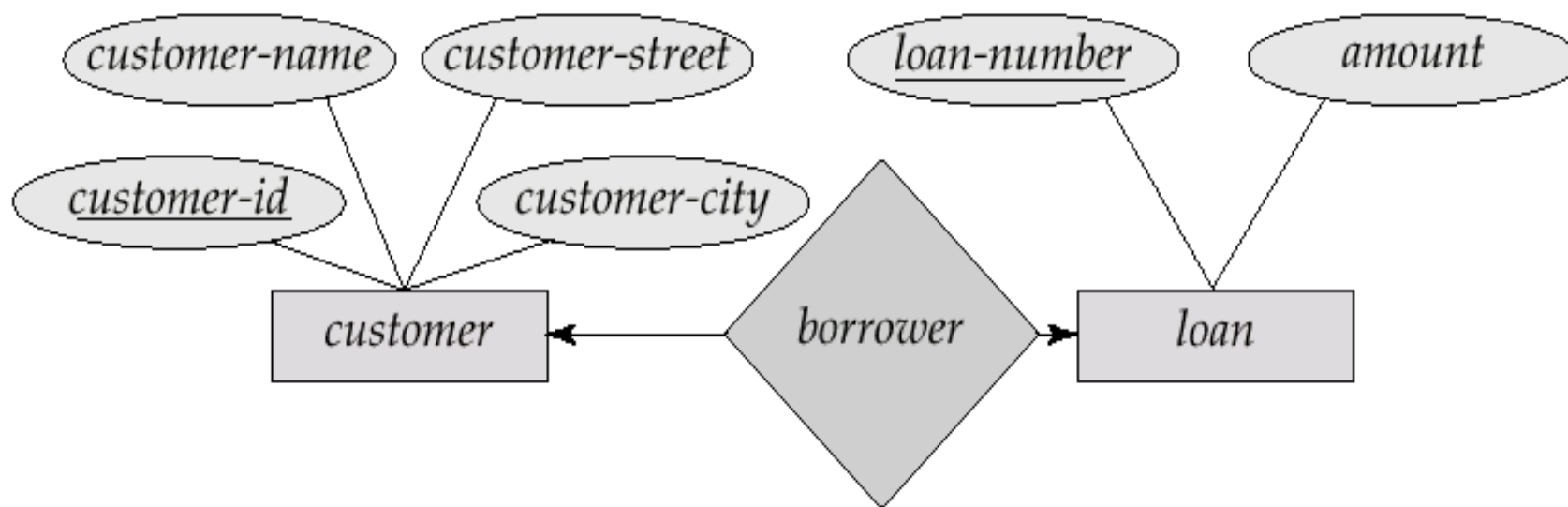
有一点点抽象，我们用例子来说吧



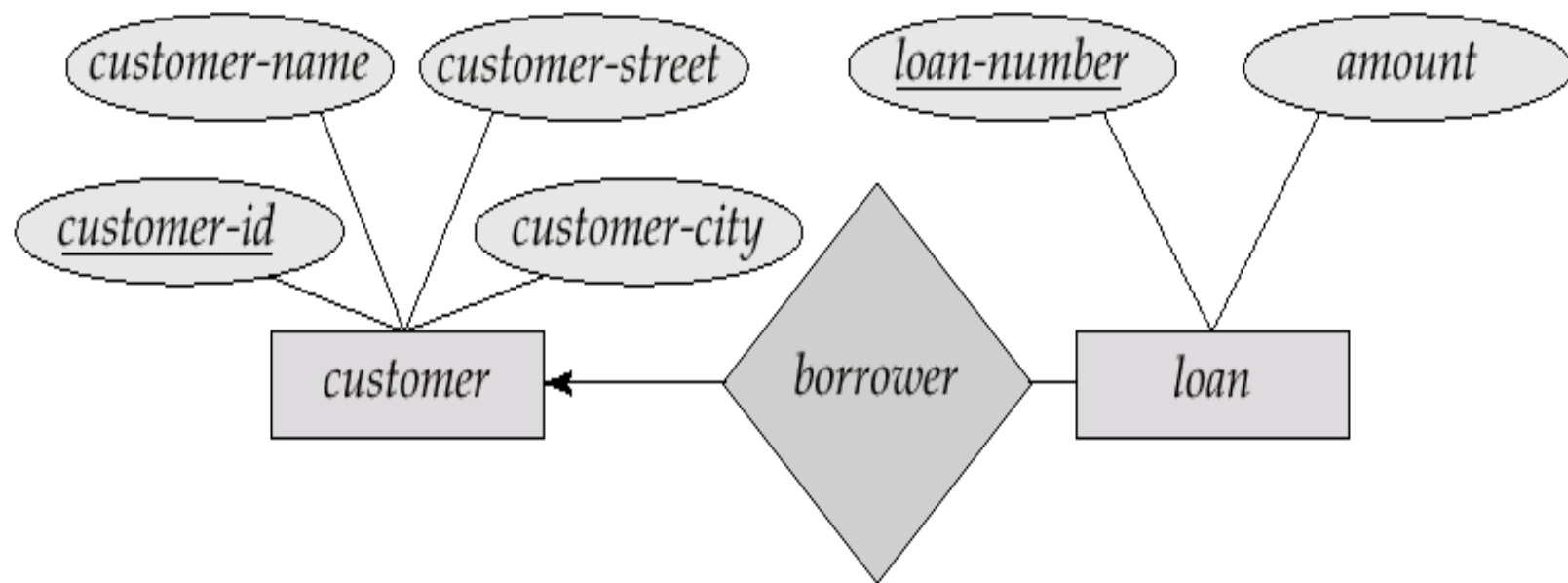
E-R模型



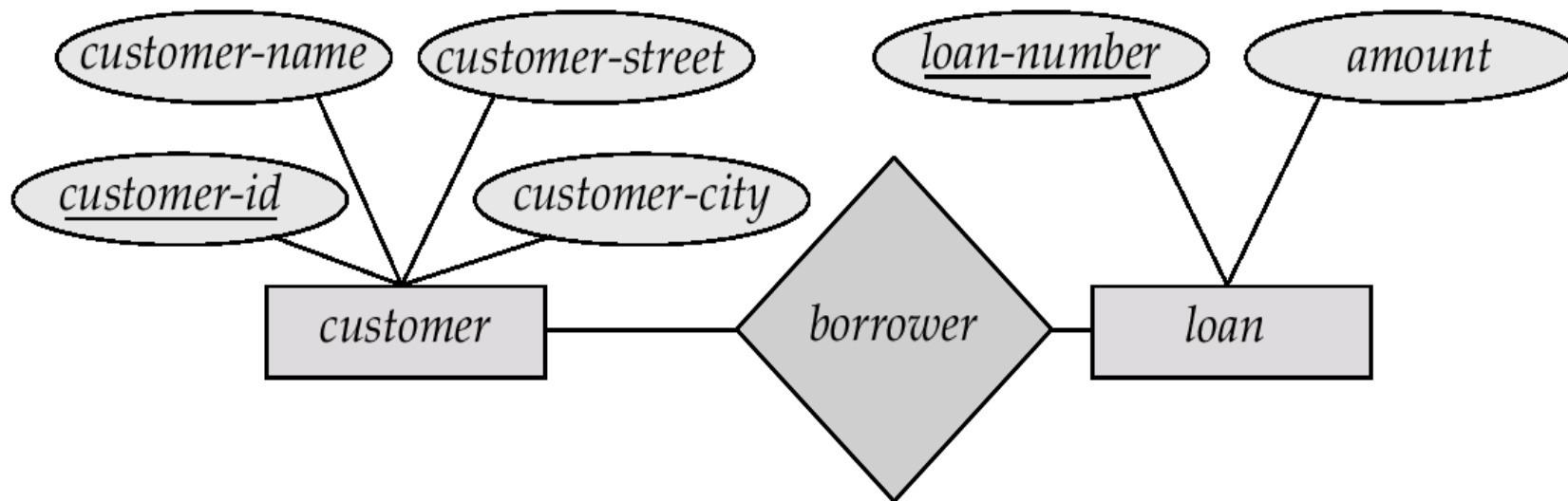
E-R模型——一对一关系



E-R模型——一对多关系

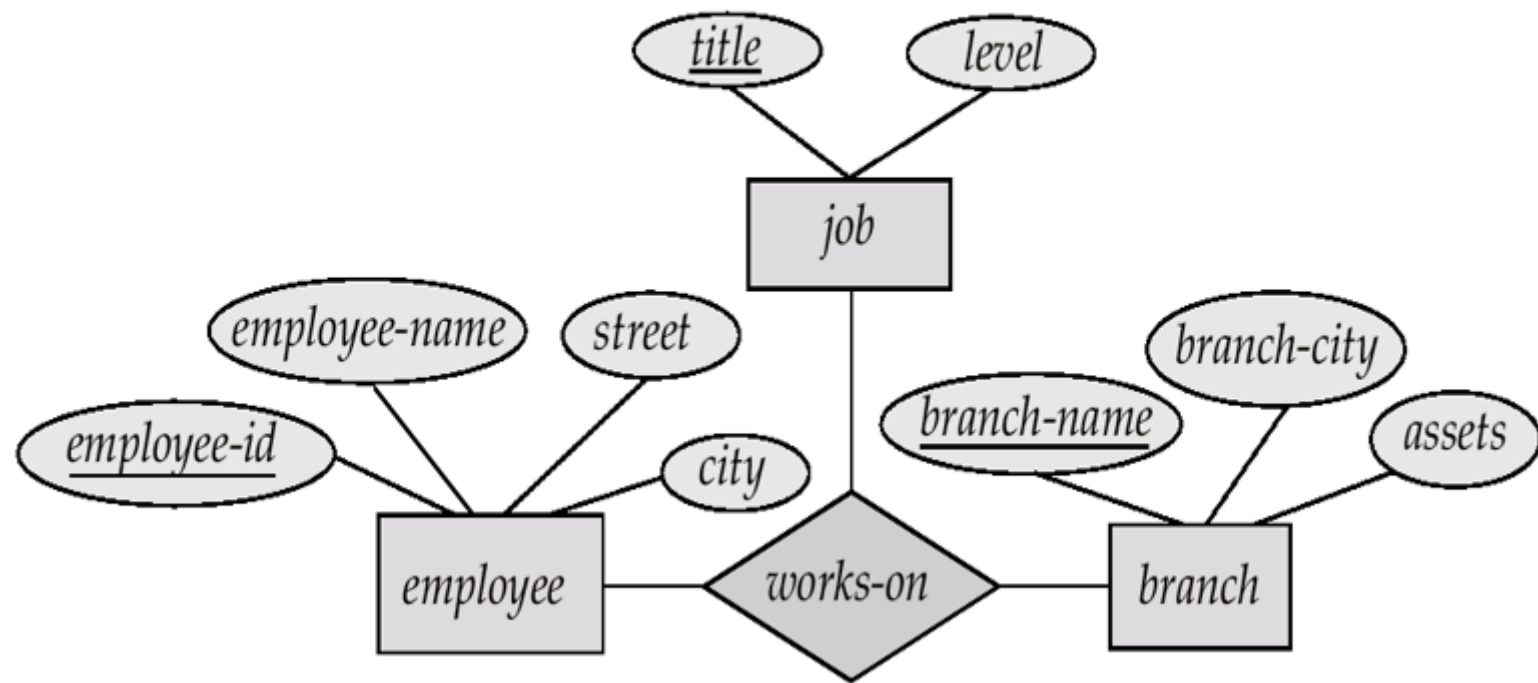


E-R模型——多对多关系

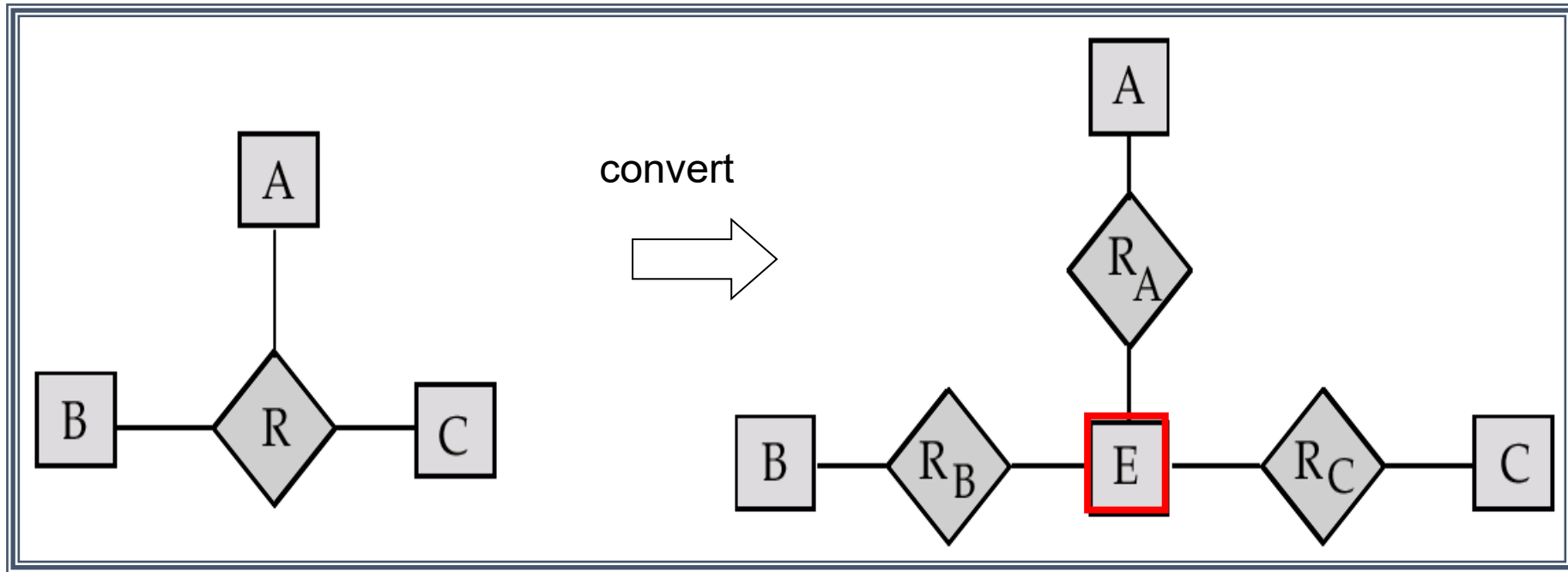


E-R模型——例子

- 一个银行职员在多个支行兼职，并承担不同类型的工作。



E-R模型——转换



E-R模型——转换

弱实体集：

没有主码

一个实体对于另一个实体(一般为强实体，也可以是依赖于其他强实体的弱实体)具有很强的依赖联系，而且该实体主键的一部分或全部从其强实体(或者对应的弱实体依赖的强实体)中获得

E-R模型练习1

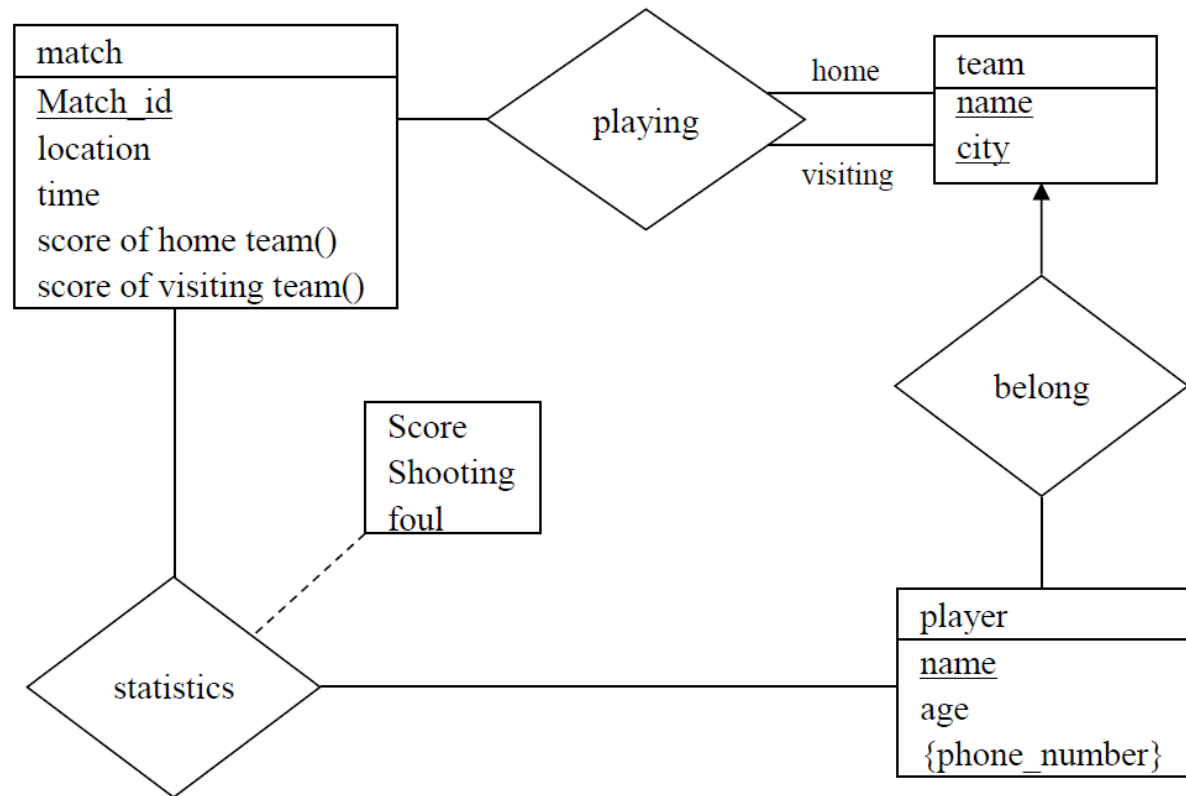
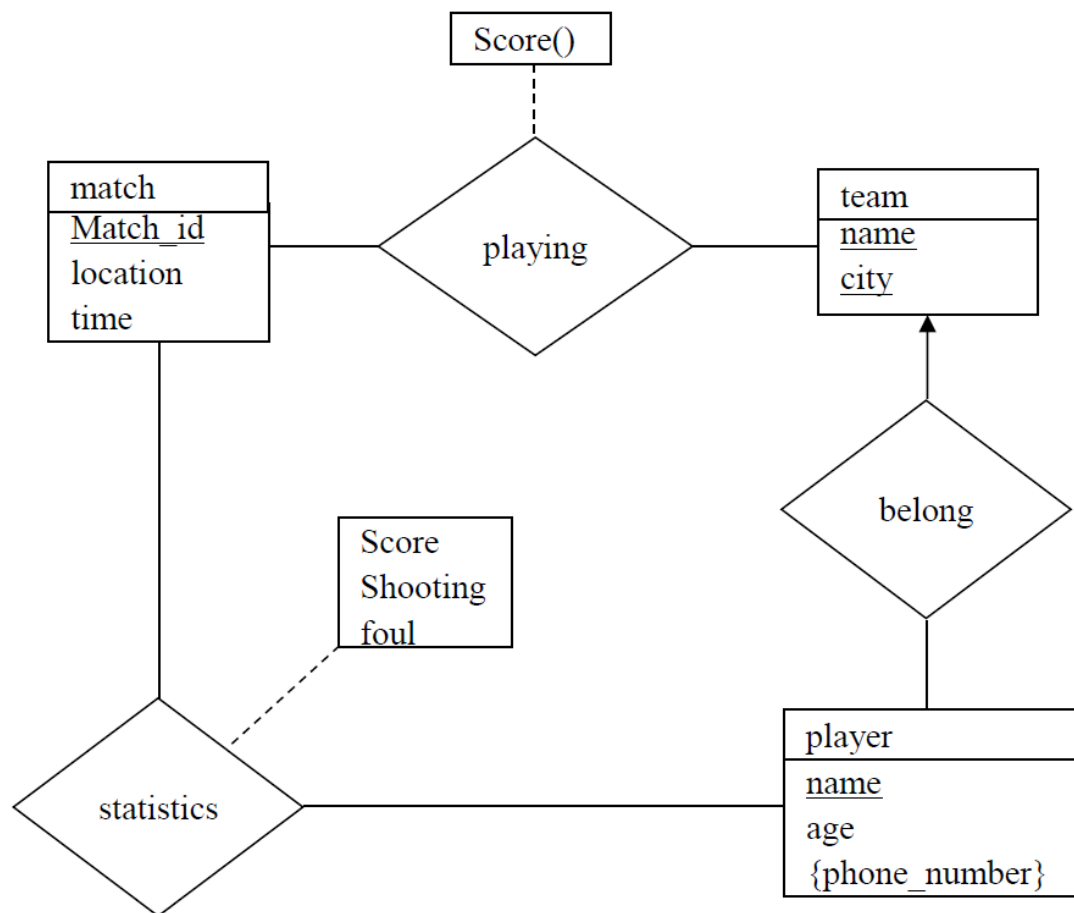
We need to store information about football teams in league matches. The information includes matches (identified by match_id, with attributes location, time) as well as the score of each team for the match, teams (identified by name, with attribute city), players (identified by name, with attributes age and several phone numbers), and individual player statistics (such as score, shooting, foul, etc.) for each match. Note that one player only belongs to one team.

Please answer the following questions:

- 1) Draw an *E-R diagram* for this database model with primary key underlined. (5 points)
- 2) Transform the E-R diagram into a number of *relational database schemas*, with the primary key underlined. (4 points)

E-R模型练习1参考答案

(1) 答案不止唯一，这里列出两种参考答案



E-R模型练习1参考答案

(2)

2) (4 points)

Match(match_id, location, time)

Team(name, city)

Playing(match_id, name, score)

Player(name, age)

Phone(player name, phone number)

Statistics(match_id, player name, score, shooting, foul)

或者其中的 match 和 playing 改为:

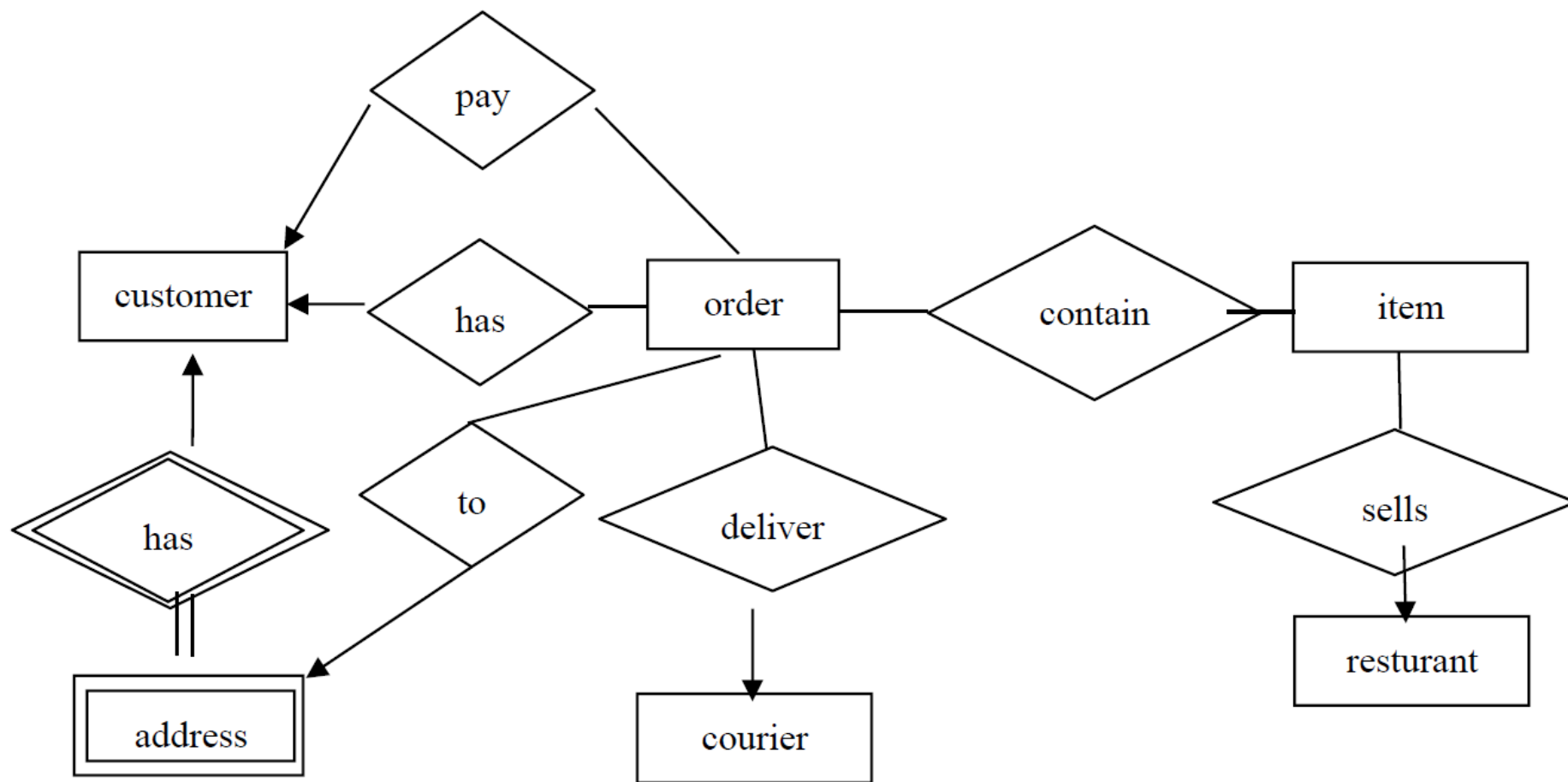
**Match(match_id, location, time, home_team_name, visiting_team_name,
score_of_home_team, score_of_visiting_team)**

Each match has one home team and one visiting team.

E-R模型练习2

Please design an ER diagram for a takeaway platform (外卖平台) to capture information about **restaurants**, **customers**, **couriers** (快递员), **orders**, **payments**, and **deliveries**, etc. A **restaurant** sell **items** on the platform. A **customer** has several **addresses**. A **courier** delivers packages of order to customers.

E-R模型练习2参考答案





04

数据库设计

回顾

- 关系型数据库设计目的：
 - 存储信息时没有不必要的冗余
 - 检索信息的效率高

第一范式：First Normal Form

- 定义：关系模式R的所有属性都是满足原子性（atomic）的
 - 原子性：属性不能再向下拆分
- 在任何关系数据库中，第一范式都是最基本的要求

第一范式：First Normal Form

- 第一范式的不足：
- 数据冗余
- 更新数据复杂
- 插入和删除数据异常

函数依赖

A	B
1	4
1	5
3	7

- 函数依赖的定义

- 对于一个关系模式R，如果 $\alpha \subset R$ 并且 $\beta \subset R$ 则函数依赖 $\alpha \rightarrow \beta$ 定义在R上，当且仅当
 - 如果对于R的任意关系r(R) 当其中的任意两个元组t1和t2，如果他们的 α 属性值相同可以推出他们的 β 属性值也相同
- 如果某个属性集A可以决定另一个属性集B的值，就称 $A \rightarrow B$ 是一个函数依赖
- 函数依赖和键的关系：函数依赖实际上是键的概念的一种泛化
 - K是关系模式R的超键当且仅当 $K \rightarrow R$
 - K是R上的候选主键当且仅当 $K \rightarrow R$ 并且不存在 $\alpha \subset K, \alpha \rightarrow R$
- 一个平凡的结论：子集一定对自己函数依赖

函数依赖闭包

- F^+ : 根据原始函数依赖集合 F 推出所有函数依赖关系产生的集合
- Armstrong 定理

If $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (**reflexivity**, 自反律) --- **trivial**

If $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$ (**augmentation**, 增补律)

$\gamma\alpha \rightarrow \beta$

If $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (**transitivity**, 传递律)

函数依赖闭包

- F^+ : 根据原始函数依赖集合 F 推出所有函数依赖关系产生的集合
- Armstrong 定理补充定律

If $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds (union, 合并律)

If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ holds (decomposition, 分解律)

If $\alpha \rightarrow \beta$ and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds (pseudotransitivity, 伪传递律)

函数依赖闭包

- 函数依赖闭包计算算法 （实际上画图更直观）

$F^+ = F$

Repeat

For each functional dependency f in F^+

Apply **reflexivity** and **augmentation rules** on f

Add the resulting functional dependencies to F^+

For each pair of functional dependencies f_1 and f_2 in F^+

If f_1 and f_2 can be combined using **transitivity**

Then add the resulting functional dependency to F^+

Until F^+ does not change any further

函数依赖闭包

那么，为什么要计算属性闭包呢？

- 1、测试属性是否为主键
 - 如果a的闭包包含了所有属性，则a是主键
- 2、测试函数独立
 - 验证函数依赖关系是否成立
- 3、计算函数依赖闭包 F^+
 - 得到整个关系模式的闭包

Decomposition

- 有损分解：不能用分解后的几个关系重建原本的关系
- 无损分解：
 - R分解为 (R_1, R_2) 并且 $R = R_1 \cup R_2$
 - 对任何关系模式R上的关系r有 $r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r)$
 - 无损分解判定方法：
 - **关键：分解后函数依赖关系能否保持**

当且仅当 $R_1 \cap R_2 \rightarrow R_1$ 或者 $R_1 \cap R_2 \rightarrow R_2$ 至少有一个 F^+ 中

Decomposition

- 有损分解和无损分解的示例：

$$R = (A, B, C), F = \{A \rightarrow B, B \rightarrow C\}$$

One way: $R_1 = (A, B), R_2 = (B, C)$

- Lossless-join decomposition:

$$R_1 \cap R_2 = \{B\} \text{ and } B \rightarrow C, \therefore (B)^+ = \{BC\} \supseteq R_2$$

- Dependency preserving: $F_1 = \{A \rightarrow B\}$ for R_1 , $F_2 = \{B \rightarrow C\}$ for R_2 , $\therefore (F_1 \cup F_2)^+ = F^+$

2nd way: $R_1 = (A, B), R_2 = (A, C)$

- Lossless-join decomposition:

$$R_1 \cap R_2 = \{A\} \text{ and } (A)^+ = \{AB\} \supseteq R_1$$

- $F_1 = \{A \rightarrow B\}$ for R_1 , $F_2 = \{A \rightarrow C\}$ for R_2 , $(F_1 \cup F_2)^+ = \{A \rightarrow B, A \rightarrow C\}^+ \neq F^+$, cannot check $B \rightarrow C$ in R_1, R_2 without computing $R_1 \bowtie R_2$

Decomposition

• 测试是否为无损分解算法:

Dependency preservation 独立性保护,把R和F的闭包按照关系的对应进行划分

- 用 F_i 表示只包含在 R_i 中出现的元素的函数依赖构成的集合
- 我们希望的结果是 $(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$
 - BCNF的分解一定是有独立性保护的
- 独立性保护的验证算法
- 如果最终的结果result包含了所有属性, 那么函数依赖 $\alpha \rightarrow \beta$ 就是被保护的

正则覆盖

- 产生缘由：
 - 由于数据库管理系统每时每刻都需要确保没有违反函数依赖FD的关系F，当F很大时，检查成本就会很高，因此，我们需要简化函数依赖
- 标准定义：
 - F的**正则覆盖** F_c 是与原函数依赖FDs**等价**的**最小**集合
- **正则覆盖是函数依赖关系的最小集合**

正则覆盖

- F 的**正则覆盖** F_c 是与原函数依赖FDs**等价的最小集合**
 - 没有冗余的函数依赖或冗余的部分
 - 每一个左侧都是唯一的
- 非正则覆盖的示例:

$$F = \{\cancel{A \rightarrow C}, A \rightarrow B, B \rightarrow C\}$$

▶ E.g., $F = \{A \rightarrow B, B \rightarrow C, \cancel{AC \rightarrow D}\}$ can be inferred to
 $\Rightarrow \{A \rightarrow B, B \rightarrow C, \underline{AC \rightarrow D}, A \rightarrow D\}, \Rightarrow \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

→ $F_c = \{A \rightarrow B, B \rightarrow C\}$

E.g., $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow \cancel{CD}\}$ can be inferred to
 $\Rightarrow \{A \rightarrow B, B \rightarrow C, \underline{A \rightarrow C}, A \rightarrow D\},$

but $A \rightarrow C$ is implied by $A \rightarrow B, B \rightarrow C,$

F is simplified to: $F' = \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$, (即 F' 蕴涵 F), i.e.,
Attribute C is **extraneous**.

正则覆盖

- 非正则覆盖的产生主要是因为有了无关属性
- 无关属性定义：

定义：对于函数依赖集合F中的一个函数依赖 $\alpha \rightarrow \beta$

- α 中的属性A是多余的，如果F逻辑上可以推出 $(F - \{\alpha \rightarrow \beta\}) \vee \{(\alpha - A) \rightarrow \beta\}$
- β 中的属性A是多余的，如果 $(F - \{\alpha \rightarrow \beta\}) \vee \{\alpha \rightarrow (\beta - A)\}$ 逻辑上可以推出F

正则覆盖

- 怎么判断一个属性是否为无关属性呢？
 - 判断 $\alpha \rightarrow \beta$ 中的一个属性是不是多余的
 - 测试 α 中的属性A是否为多余的
 - 计算 $(\alpha - A)^+$
 - 检查结果中是否包含 β ，如果有就说明A是多余的
 - 测试 β 中的属性A是否为多余的
 - 只用 $(F - \{\alpha \rightarrow \beta\}) \vee \{\alpha \rightarrow (\beta - A)\}$ 中有的依赖关系计算 α^+
 - 如果结果包含A，就说明A是多余的

正则覆盖

- 计算正则覆盖的算法
- (其实还是比较复杂, 更多的靠经验和感觉找)

repeat

 Use the union rule to replace any dependencies in F like $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1\beta_2$

 Find a functional dependency $\alpha \rightarrow \beta$ with an extraneous attribute either in α or in β

 If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$

until F does not change

正则覆盖

- 计算正则覆盖的例题

- $R = (A, B, C)$

- $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$

- Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$

- Set is now $F' = \{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$

- A is extraneous in $AB \rightarrow C$

- Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies $B \rightarrow C$

- Set is now $F' = \{A \rightarrow BC, B \rightarrow C\}$

- C is extraneous in $A \rightarrow BC$

- Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and $B \rightarrow C$

- The canonical cover is: $F_C = \{A \rightarrow B, B \rightarrow C\}$

BCNF

- 什么是BCNF?

闭包 F^+ 中的所有函数依赖 $\alpha \rightarrow \beta$ 至少满足下面的一条

- $\alpha \rightarrow \beta$ 是平凡的(也就是 β 是 α 的子集)
- α 是关系模式 R 的一个超键, 即 $\alpha \rightarrow R$

BCNF

$R = (A, B, C)$

$F = \{A \rightarrow B$

$B \rightarrow C\}$

Key = $\{A\}$

- 一个例子 (如右图)

- 这里, R 不是BCNF, 因为 B 不是主键
- 但是, 通过分解 R , 可以生成满足BCNF条件的关系, 如下:

Decomposition $R_1 = (A, B), R_2 = (B, C)$

- R_1 and R_2 in BCNF ($\because A$ is the key for R_1 , B is the key for R_2).
- Lossless-join decomposition ($\because R_1 \cap R_2 = B$, and B is the key for R_2).
- Dependency preserving ($\because F_1 = \{A \rightarrow B\}$ holds on R_1 , $F_2 = \{B \rightarrow C\}$ holds on R_2).

BCNF

- 检测一个非平凡的函数依赖 $\alpha \rightarrow \beta$ 是否违背了BCNF的原则
 - 计算 α 的属性闭包
 - 如果这个属性闭包包含了所有的元素, 那么 α 就是一个**超键**
 - 如果 α 不是超键而这个函数依赖又不平凡, 就打破了BCNF的原则
- 简化的检测方法:
 - 只需要看关系模式R和已经给定的函数依赖集合**F中的各个函数依赖**是否满足BCNF的原则
 - 不需要检查F闭包中所有的函数独立
 - 可以证明如果F中没有违背BCNF原则的函数依赖, 那么F的闭包中也没有
 - 这个方法不能用于检测R的分解

BCNF分解算法

```
result := {R};  
done := false;  
compute  $F^+$ ;  
while (not done) do  
  if (there is a schema  $R_i$  in result that is not in BCNF)  
    then begin  
      let  $\alpha \rightarrow \beta$  be a nontrivial functional  
      dependency that holds on  $R_i$  such  
      that  $\alpha \rightarrow R_i$  is not in  $F^+$ , and  $\alpha \cap \beta = \emptyset$ ;  
      result := (result -  $R_i$ )  $\cup (\alpha, \beta)$   $\cup (R_i - \beta)$ ;  
      end  
    else done := true;
```

It means there is nontrivial FD
 $\alpha \rightarrow \beta$ in R_i , and α is not a key.

将 R_i 分解为二个子模式:
 $R_{i1} = (\alpha, \beta)$ 和 $R_{i2} = (R_i - \beta)$,
 α 是 R_{i1} R_{i2} 的共同属性.

R_{i1}

R_{i2}

关系型数据库设计例题1

Problem 3: Relational Formalization(12 points, 3 points per part)

For relation schema $R(A, B, C, D, E)$ with functional dependencies set $F=\{A \rightarrow CD, C \rightarrow B, B \rightarrow D, B \rightarrow E\}$

- (1) Compute the Canonical Cover of F .
- (2) Compute the Closure of attribute set $\{B\}$.
- (3) Decompose the relation R into a collection of BCNF relations, and give out the functional dependency set for each relation.
- (4) Explain whether above decomposition be dependency preserving or not.

关系型数据库设计练习1参考答案

Answers of Problem 3:

(12 points, 3 points per part)

(1) $F_c = \{A \rightarrow C, C \rightarrow B, B \rightarrow DE\}$

(2) $(B)^+ = (B, D, E)$

(3) $R_1(B, D, E), F_1 = \{B \rightarrow DE\}$

$R_2(C, B), F_2 = \{C \rightarrow B\}$

$R_3(A, C), F_3 = \{A \rightarrow C\}$

评分细则:

每个实体或联系错扣 1 分，直至扣完。

存在多种分解方法，需区分判题。

(4) The decomposition is dependency preserving,

because $(F_1 \cup F_2 \cup F_3)^+ = F^+$

评分细则:

针对 (3) 中的不同分解方法，其函数保持的判断有不同，需区分判题。

关系型数据库设计练习2

Problem 3: Relational Formalization (12 points, 4 points each)

For relation schema $R(A, B, C, D, E)$ with functional dependencies set $F=\{A \rightarrow B, BC \rightarrow D, C \rightarrow A\}$

- 1) Find all *candidate keys* of R .
- 2) Decompose the relation R into a collection of *BCNF relations*.
- 3) Explain whether above decomposition be *dependency preserving* or not.

关系型数据库设计练习2参考答案

Problem 3: Relational Formalization (12 points, 4 points each)

1) {C E}

评分细则:

写对一个键给两分，多写一个扣 1 分。诸如写{ACE,CE,BCE}的不得分

2) Decompose R into R1(A, B) and R2(A, C, D, E), decompose R2 into R21(A, C) R22(C, D, E), and further decompose R22 into R221(C, D) and R222(C, E)

评分细则:

由于根据不同模式分出来的步骤可能不一，但是由于最终关系为{C→A,C→B,C→D}，所以最终结果一定是诸如{A,C}{B,C}{C,D}{D,E}等二元组，根据实际情况没分彻底的如(B,C,D)每个 0.5 分，分彻底的 1 个 1 分，分错但是结果正确的酌情给 2-3 分。

3) The decomposition is dependency preserving.

评分细则:

(2)中答案正确并且此处正确的给 4 分，

(2)中答案错误并且根据实际拆分情况，若判断一致此处给 3 分

(2)中答案错误并且此处正确的给 2 分，（不给出解释扣 1 分）

其余情况不给分

3NF

- BCNF范式会不会太强了呢？有没有弱一些的范式？
- 有——第三范式！
 - 第三范式的定义：对于函数依赖的闭包 F^+ 中的所有函数依赖 $\alpha \rightarrow \beta$ 下面三条至少满足一条
 - $\alpha \rightarrow \beta$ 是平凡的
 - α 是关系模式R的超键
 - 每一个 $\beta - \alpha$ 中的 属性A都包含在一个R的候选主键中
 - BCNF一定是3NF，实际上3NF是为了保证独立性保护的BCNF
 - 3NF有冗余，某些情况需要设置一些空值

3NF

• 第三范式的判定

- 不需要判断闭包中的所有函数依赖，只需要对已有的F中的所有函数依赖进行判断
- 用闭包可以检查 $\alpha \rightarrow \beta$ 中的 α 是不是超键
- 如果不是，就需要检查 β 中的每一个属性包含在R的候选键中

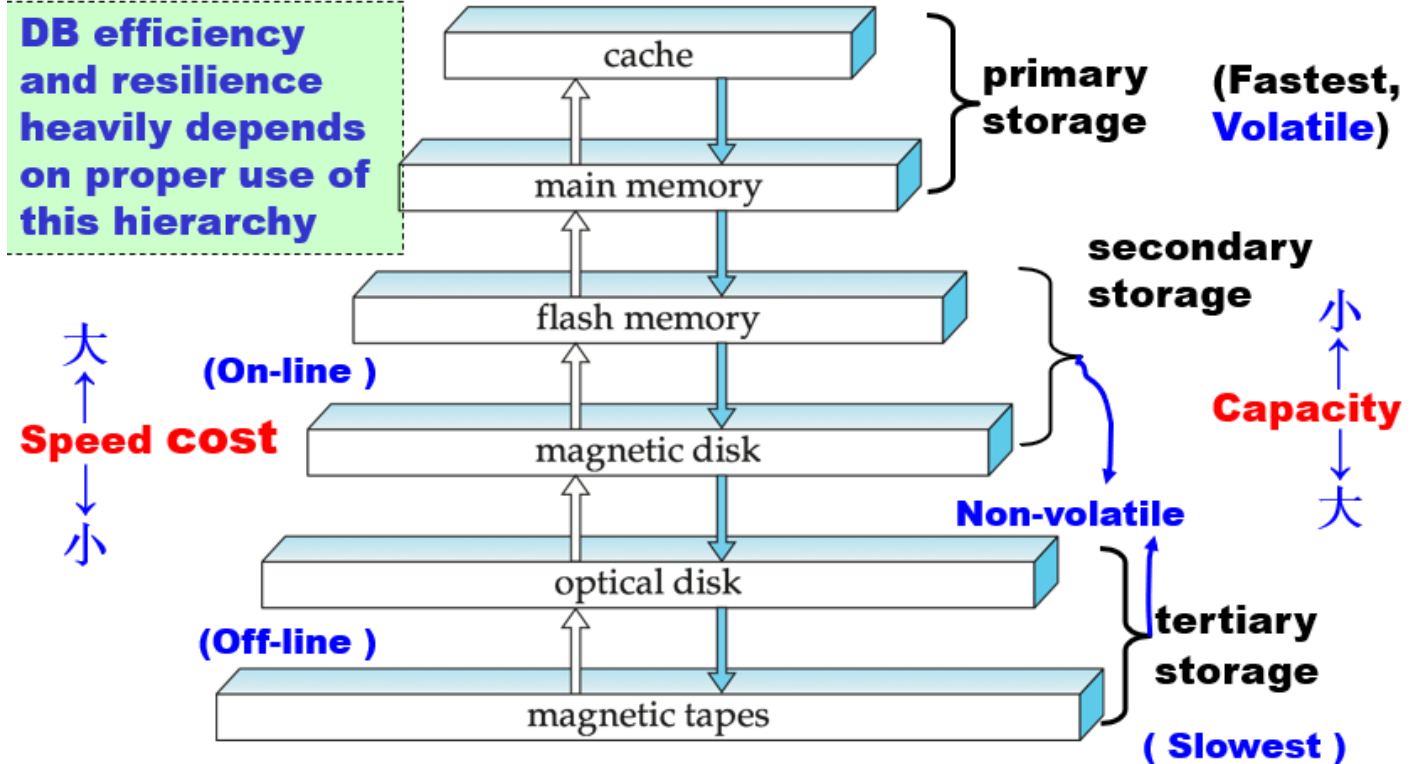


05

数据库存储

层次化存储

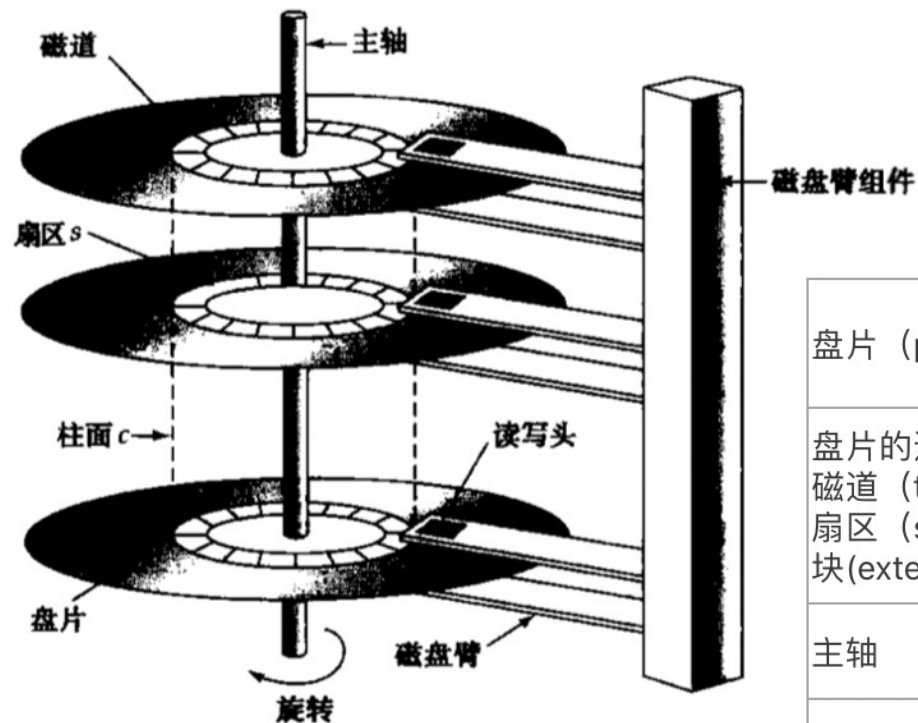
DB efficiency and resilience heavily depends on proper use of this hierarchy



层次化存储

高速缓冲存储器 (cache)	最快最昂贵的存储介质。 一般体积较小，由计算机系统硬件来管理它的使用。
主存储器 main memory	主存可以用来存放可处理的数据。 通用计算指令在主存上执行。 如果电源或系统故障，主存中内容会丢失。
快闪存储器 flash memory	有两种类型的快闪存储器：NAND和NOR快闪。 NAND具有更高的存储容量，并广泛的作为照相机、手机和音乐播放器的数据存储。 相比主存：快闪存储器更便宜，并具有非易失性。电源或系统故障，快闪存储器内容不会丢失，可保存下来。 应用：USB，固态硬盘 固态硬盘：在存储中等数据量的数据时，可以用快闪存储器替代磁盘存储器使用。这种驱动器替代品就是固态驱动器（solid-driver）。它广泛的用于服务器中，因为它比磁盘的速度快，比主存的容量大（便宜）
磁盘 magnetic-disk storage	用于长期联机数据存储的主要介质。 通常数据库存储在磁盘上。 为了能够访问数据，系统要将数据从磁盘移到主存上；完成操作后，修改后的数据必须再写回磁盘。 磁盘存储器不会因为系统故障或者系统崩溃而丢失数据。但是磁盘存储设备本身也有可能发生故障，导致数据损坏，但是磁盘故障的概率远低于系统故障发生的概率。
光学存储器 optical storage	光盘（Compact Disk, CD）- 存储视频数据 数字视频光盘（Digital Video Disk, DVD）- 存储任何数字数据，而不仅仅是视频数据。 数据通过光学的方法存储到光盘上，并通过激光器读取。 种类： 只读光盘（CD-ROM） 只读DVD（DVD-ROM） 写一次读多次光盘（Write-One, Read-Many, WORM） 写多次光盘（CD-RW） 写多次DVD（DVD-RW, DVD-RAM, DVD+RW） 自动光盘机（jukebox）系统包含少量驱动器和大量机械手自动装入某一驱动器的光盘。
磁带存储器 tape storage	主要用于备份数据和归档数据。 磁带存储器必须从头顺序访问，因此，它也称为顺序访问的（sequential-access）存储器。 而磁盘存储器则称为直接访问（diirect-access）的存储器，因为可以从磁盘的任何位置读入数据。

磁盘



盘片 (plate)	一张磁盘包括多个盘片。 磁盘的每个盘片是一个扁平的圆盘。 它的两个表面都附着磁性物质，信息就记录在表面上。
盘片的逻辑划分： 磁道 (track) 扇区 (sector) 块 (extent)	盘片的表面从逻辑上可以划分为磁道，磁道又划分为扇区。 扇区是从磁盘读出和写入的最小单位。一般为512字节大小。 外侧的磁道比内侧的磁道拥有更多的扇区。 块：是一个逻辑单元，包含固定数目的扇区。
主轴	所有盘片安装在主轴上，当磁盘被使用时，驱动马达可以驱动转轴使磁盘以很高的恒定速度旋转。
读写头 (read-write head)	磁盘的每个盘面的两面都有一个读写头。 读写头可以通过在盘片上来回移动来访问不同的磁道。
磁盘臂 (disk arm)	一张磁盘包括多个盘片，所有盘片的读写头被安装在磁盘臂上，并且一起移动。
磁头-磁盘装置 head-disk assembly	安装在主轴上的磁盘盘片和安装在磁盘臂上的所有读写头统称为磁头-磁盘装置。
柱面 (cylinder)	因为所有盘片上的读写头一起移动，所以当某个盘片上的读写头在第i条磁道上，所有其他盘片的读写头也在第i条磁道上。 所有盘片的第i条磁道合在一起，称为第i个柱面。

磁盘性能的度量

- 磁盘质量的主要度量指标：容量，访问时间，数据传输率，可靠性。

容量(capacity)	磁盘的可存储数据的大小
访问时间 access time	从发出请求到数据开始传输之间的时间。 访问时间 = 寻道时间+ 旋转等待时间 为了访问磁盘上的数据，需要：磁盘臂首先移动以定位到正确的磁道；然后等待磁盘旋转，直到指定的扇区出现在它的正下方。 寻道时间(seek time)：磁盘臂重定位到正确的磁道所需的时间。它随磁盘臂移动距离的增大而增大。平均寻道时间大约是最长寻道时间的1/2。通常平均寻道时间约为4-10毫秒。 旋转等待时间 (rotational latency time)：一旦读写头到达了所需的磁道，等待访问的扇区出现在读写头下所花费的时间称为旋转等待时间。平均旋转等待时间大约是磁盘旋转一周时间的1/2。 通常访问时间大概是8-20毫秒。
数据传输率 data-transfer rate	当等待访问的第一个扇区来到读写头的正下方，数据传输就开始了，数据传输率是从磁盘获取数据或向磁盘存储数据的速率。
可靠性 - 平均故障时间MTTF Mean Time To Failure	平均故障时间就是，平均来说我们希望设备无故障运行的时间。 例如：现代磁盘的平均故障时间在50000-1200000小时（57-136年）。但是注意的是，这个平均故障时间是基于全新磁盘来说的。通常磁盘工作的时间越久，其故障发生的概率也会显著提高。

磁盘块访问的优化

- 磁盘的连续请求可分为两类：
 - 顺序访问(sequential access)模式：连续请求会请求处于相邻的磁道或者相邻的磁道上连续的块。顺序访问中只有读取第一个块时需要寻道，后续请求不需要寻道。
 - 随机访问模式(random access)模式：相继的请求会请求那些随机位于磁盘上的块，每一次请求都需要一次磁盘寻道。一张磁盘在每秒能够满足的随机块访问的数量取决于寻道时间。

RAID

- RAID, Redundant Array of Independent Disk, 独立磁盘冗余阵列。

- 通过冗余提高可靠性

- 通过并行提高性能



a) RAID 0: 无冗余拆分



b) RAID 1: 镜像磁盘



c) RAID 2: 内存风格的纠错码



d) RAID 3: 位交叉奇偶校验



e) RAID 4: 块交叉奇偶校验



f) RAID 5: 块交叉的分布奇偶校验



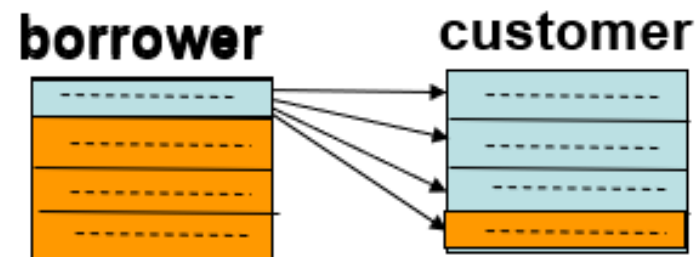
g) RAID 6: P+Q冗余

Buffer-Replacement Policy

- LRU
 - 最近最少使用策略
- MRU
 - 最近最常使用策略

➤ E.g.,

borrower ✕ **customer**



设 $M = 5$ (buffer has 5 blocks), 1 for borrower, 3 for customer, 1 for out
对customer, LRU的块可能是下面快要用的块(循环), 而最近刚用过的块则暂时不用, 当空间不够时倒是可以将其覆盖的, 故**LRU策略不佳**。

Buffer-Replacement Policy

- LRU

- 最近最少使用策略

- MRU

- 最近最常使用策略

- b. LRU is preferable to MRU where $R_1 \bowtie R_2$ is computed by sorting the relations by join values and then comparing the values by proceeding through the relations. Due to duplicate join values, it may be necessary to “back up” in one of the relations. This “backing up” could cross a block boundary into the most recently used block, which would have been replaced by a system using MRU buffer management, if a new block was needed.

Under MRU, some unused blocks may remain in memory forever. In practice, MRU can be used only in special situations like that of the nested-loop strategy discussed in Exercise Section 13.8a.

文件组织

- 数据库存储在一系列的文件中，每个文件是一系列的记录records，每条记录包含一系列的fields
- 记录类型
 - 定长记录
 - 变长记录

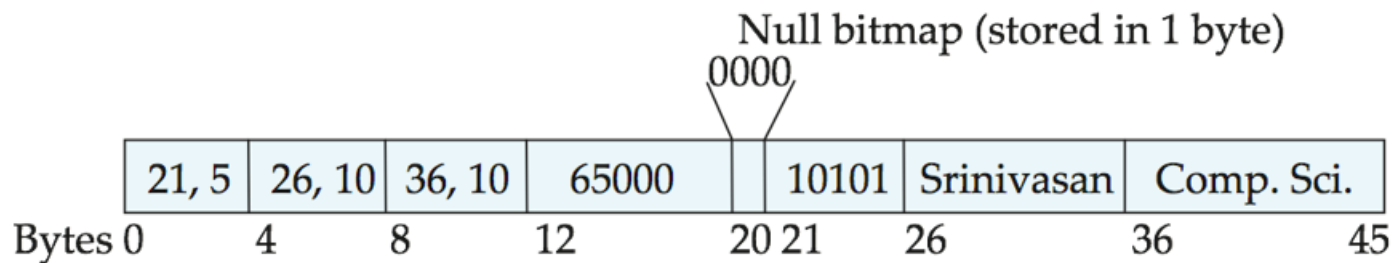
定长记录

- 每个文件被划分为固定长度的block，block是数据存取/存储空间分配的基本单位
- - 记录的长度不能超过block
- - 每条记录一定都是完整的
- - 参考实现：
 - Free List 用链表的形式来存储records
- 优势：存储和读取访问寻址简单
- 缺点：可能浪费存储空间

<u>header</u>				Link for free list
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

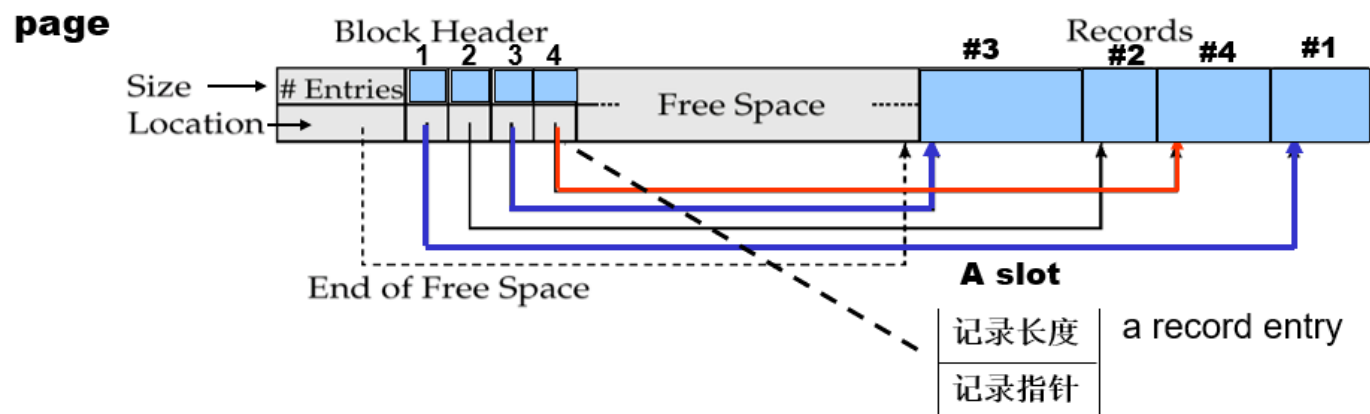
变长记录

- 典型的变长记录
 - 属性按照顺序存储
 - 变长的变量用offset+data的形式存储，空值用null-value bitmap存储



slotted page结构

- Block Header内容
 - 记录的总数
 - lock中的空闲区域的end
 - 每条记录所在的位置和大小



当向堆表中插入一条记录时，一种简单的做法是，沿着 `TablePage` 构成的链表依次查找，直到找到第一个能够容纳该记录的 `TablePage` (*First Fit* 策略)。当需要从堆表中删除指定 `RowId` 对应的记录时，框架中提供了一种逻辑删除的方案，即通过打上 `Delete Mask` 来标记记录被删除，在之后某个时间段再从物理意义上真正删除该记录（本节中需要完成的任务之一）。对于更新操作，需要分两种情况进行考虑，一种是 `TablePage` 能够容纳下更新后的数据，另一种则是 `TablePage` 不能够容纳下更新后的数据，前者直接在数据页中进行更新即可，后者的实现方式留给同学们自行思考。此外，在堆表中还需要实现迭代器 `TableIterator` (`src/include/storage/table_iterator.h`)，以便上层模块遍历堆表中的所有记录。

列式存储的一个例子

- 这是我们要存储的数据：

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

- 面向行的存储：

A1	B1	C1	A2	B2	C2	A3	B3	C3
----	----	----	----	----	----	----	----	----

- 面向列的存储：

A1	A2	A3	B1	B2	B3	C1	C2	C3
----	----	----	----	----	----	----	----	----

面向列的存储

- 优点：
 - 当只访问部分属性时，减少IO开销
 - 提高CPU缓存性能
 - 提高压缩率
 - 可以在现代CPU架构中实现Vector Processing
- 缺点：
 - 行重构开销更大
 - 删除和更新的开销更大
 - 解压所需时间更长



06

数据库索引

我们为什么需要索引?

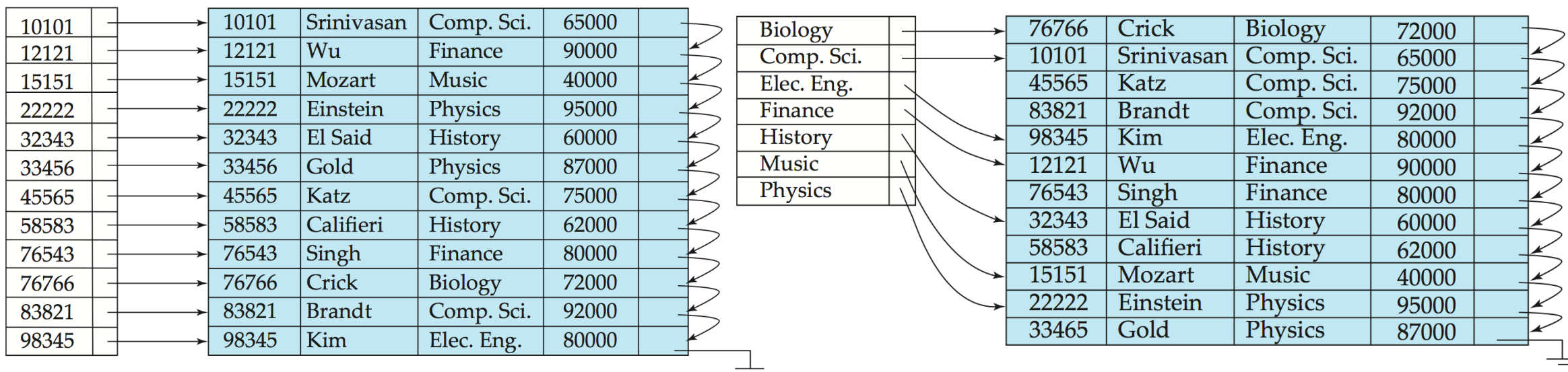
- 为了提高搜索的速度!
- (线性扫描 → 二分搜索等)
- **时间效率**和**空间效率**是衡量索引技术最重要的指标, 因此, 索引文件通常需要比原始文件小得多, 且对提高性能又很大帮助。

索引的种类

- 顺序索引
 - 索引记录按顺序存储
 - 如：稠密索引、稀疏索引
- 散列索引
 - 索引记录随机分布

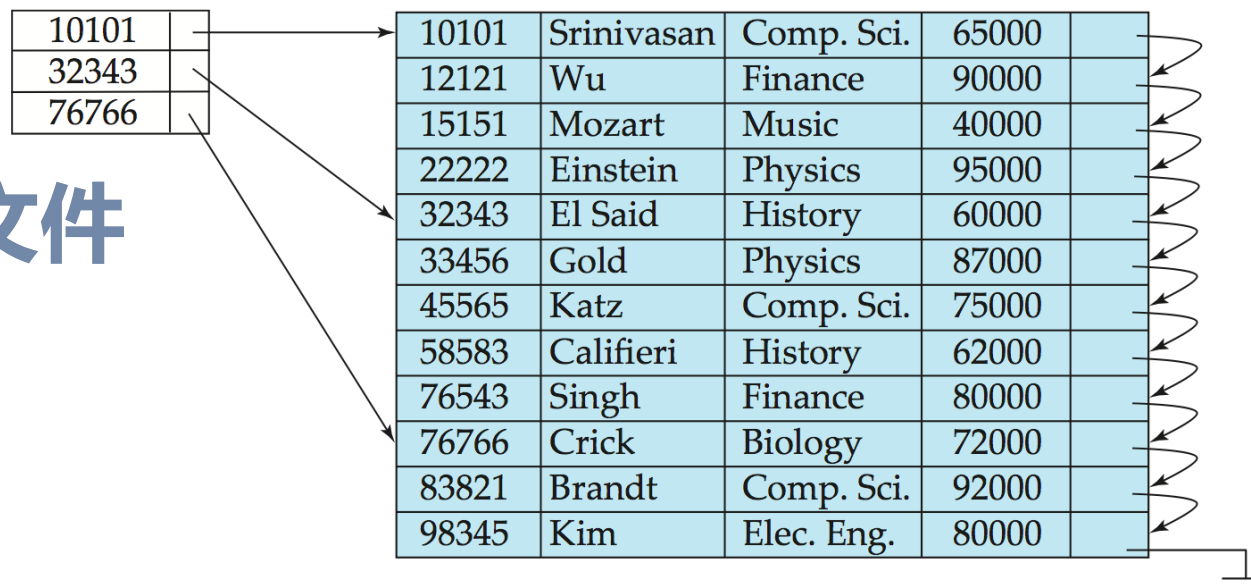
稠密索引 Dense Index

- 特征： **每一条记录都有对应的索引**



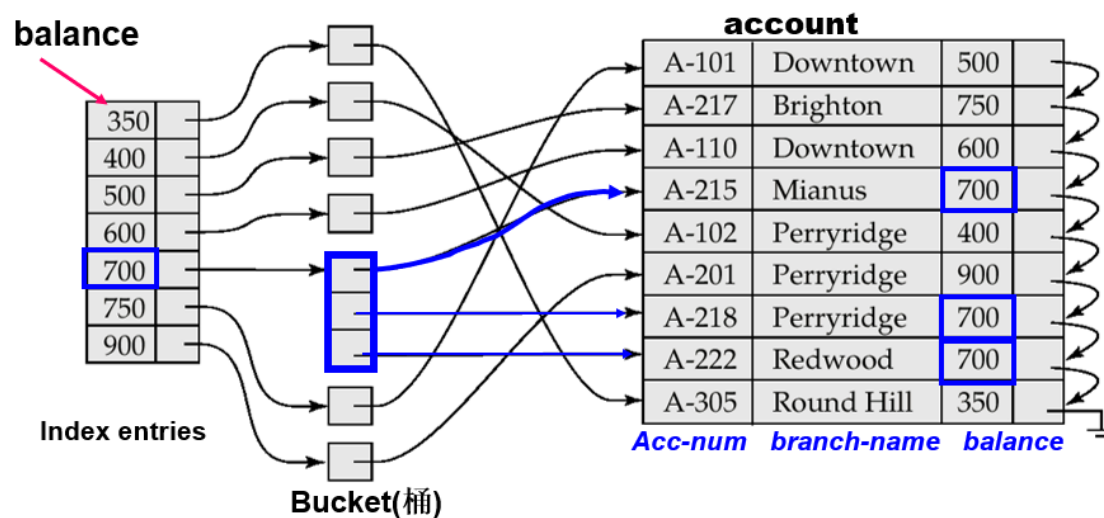
稀疏索引 Sparse Index

- 特征： **只有部分search-key有索引**
 - 需要的空间和插入删除新索引的开销较小，但是比密集的索引要慢
- 只能够用于顺序文件



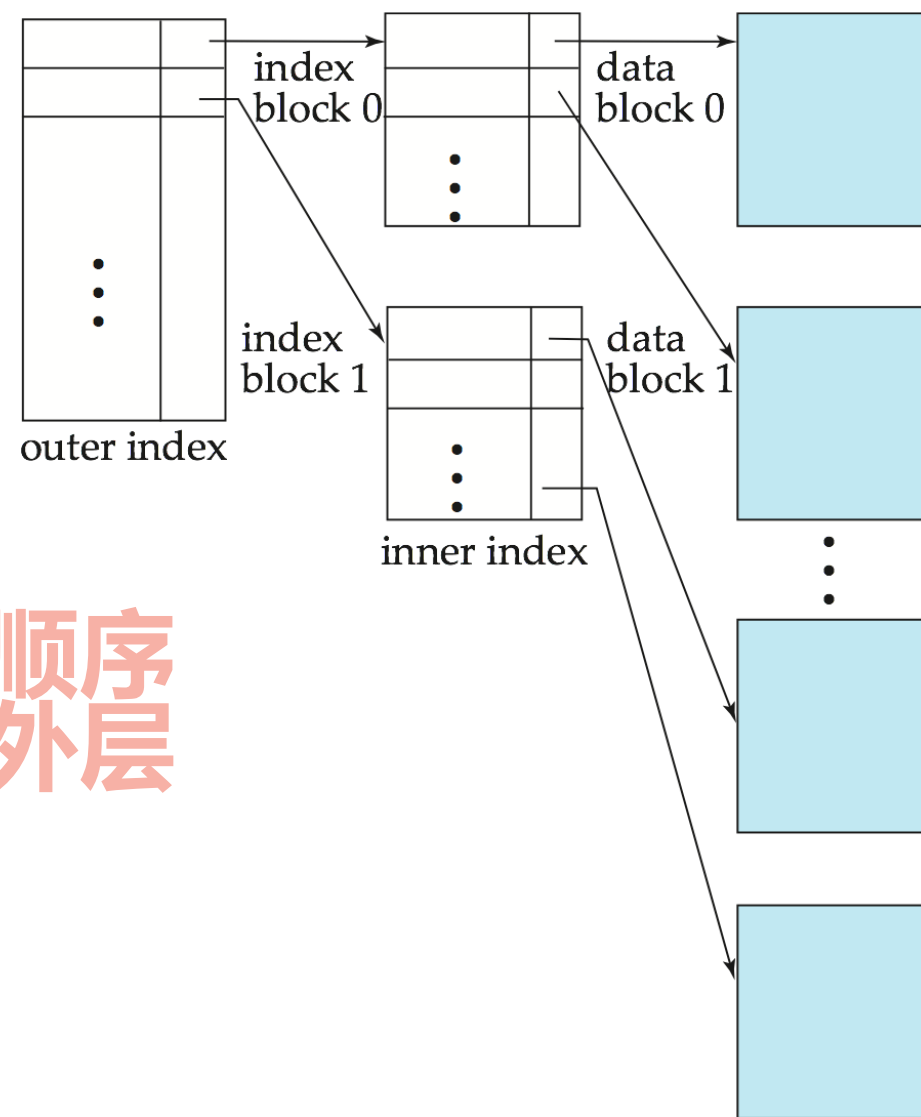
辅助索引 Secondary Indices

- 特征：数据文件中的记录顺序与索引文件中索引项的顺序不一致
- 应用：需要查询的field并非主键

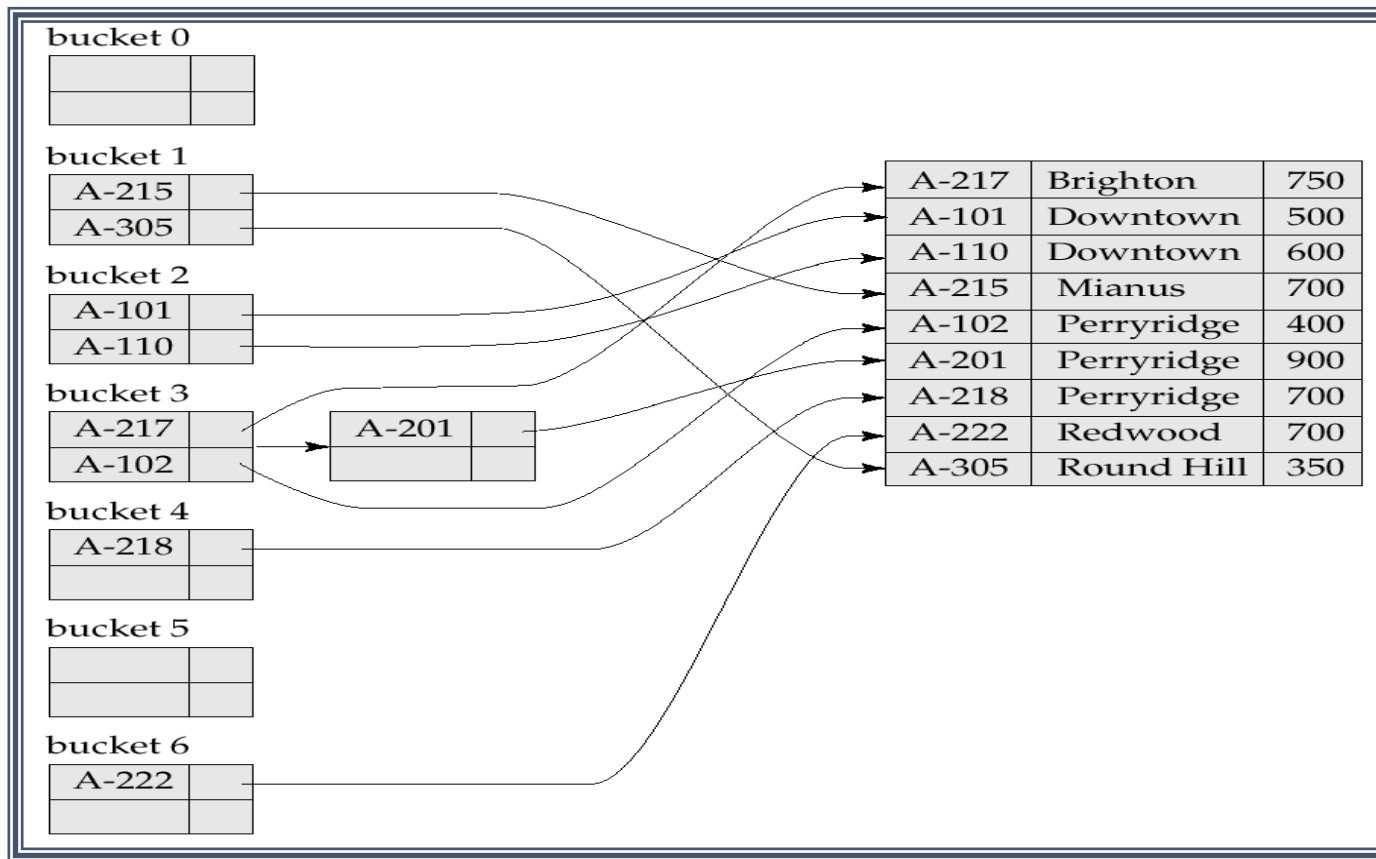


多级索引 Multilevel Indices

- 特征：把内层索引文件看作顺序数据文件一样，在其上建立外层的稀疏索引

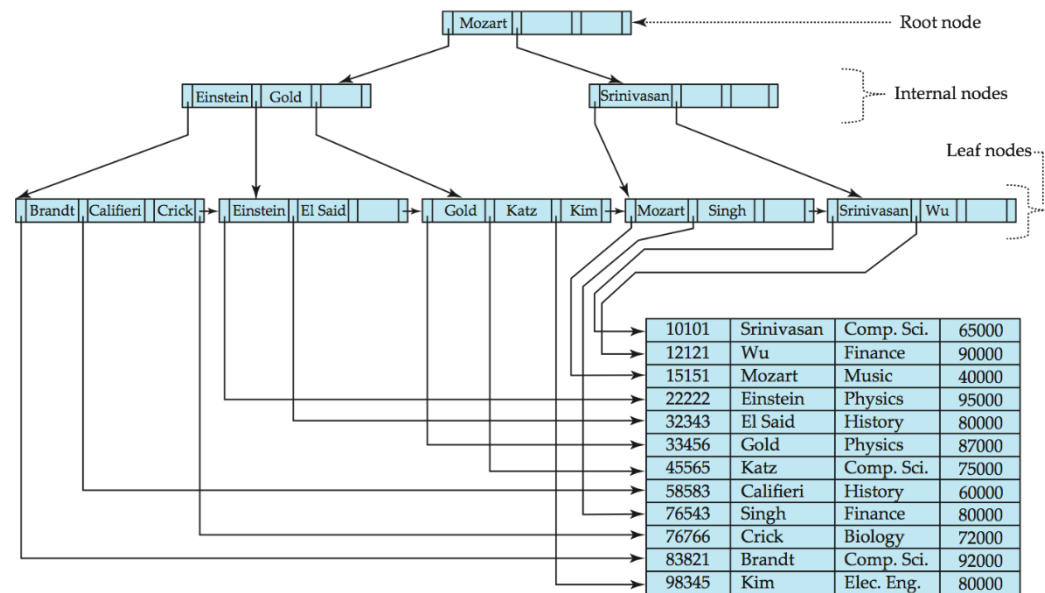


散列索引 Hash Indices



先介绍一些数据库课程中关于B+树的概念和定义

- B+树是一种最广泛使用的索引文件结构之一，采用平衡树结构，是在数据插入和删除的情况下仍能保持执行效率的结构，只需很小的、局部的变化自动重组文件。
- B+树的缺点：会有插入数据和删除数据的额外开销，增加空间开销。



B+树的性质

- 从树根到树叶的每一条路径长度相等
- Each node that is not a root or a leaf has between $\lceil \frac{n}{2} \rceil$ and n children.
- A leaf node has between $\lceil \frac{n-1}{2} \rceil$ and $n - 1$ values
- Special cases: If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.
- leaf nodes
 - P_i 指向具有 search-key值 K_i 的文件记录
 - 叶子节点的指针 P_n 有特殊作用, 每个叶子节点的 P_n 指针指向下一个叶子节点, 将所有的叶子节点按照 search-key 值的顺序链起来, 提高对文件的顺序处理效率
- non-leaf nodes
 - B+树非叶子结点 (也称内部结点) 形成叶子结点上的多级稀疏索引
 - fan-out: 结点的指针数

先介绍一些数据库课程中关于B+树的概念和定义

- 通过定义我们可以知道：
- 树的非叶节点由指向儿子的指针和search-key相间组合而成
- 两个search-key之间的指针指向的数据的值在这两个search-key之间

P_1	K_1	P_2	...	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	-----	-----------	-----------	-------

B+树的插入与删除

- **插入算法：**

- 先找到该插入的位置直接插入，如果当前的节点数量超过了阶数 M 则拆成两个部分，并向上更新索引

- **删除算法：**

- 直接把要删除的节点删除，然后把没有索引key了的非叶节点删除，从旁边找一个叶节点来合并出新的非叶节点

那么，我们来看一些B+树相关的计算

- 高度估计

- 高度最小的情况：

所有的叶节点都满，此时的 $h = \lceil \log_N(M) \rceil$

- 高度最大的情况：

所有的叶节点都半满，此时的 $h = \lfloor \log_{[N/2]}(\frac{M}{2}) \rfloor + 1$

那么，我们来看一些B+树相关的计算

• 高度最大的补充说明

关键字个数的范围

根节点: $[1, m]$

非根节点: $[\lceil m/2 \rceil - 1, m - 1]$

最小关键字:

根节点: 1

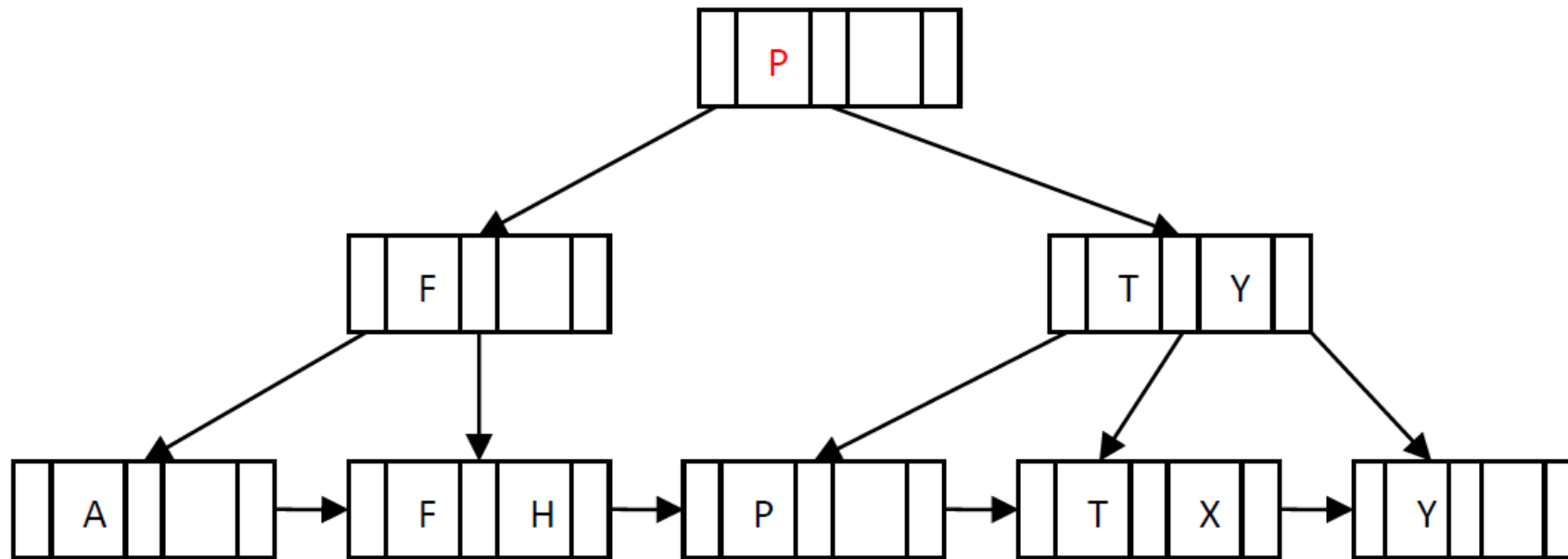
非根节点: $\lceil m/2 \rceil - 1$

叶子节点 →

节点个数	层数	
1	0	 高度为h
2	1	
$2 * \lceil m/2 \rceil$	2	
$2 * \lceil m/2 \rceil^2$	3	
$2 * \lceil m/2 \rceil^3$	4	
.....		
$2 * \lceil m/2 \rceil^{(h-2)}$	h-1	
$2 * \lceil m/2 \rceil^{(h-1)}$	h	

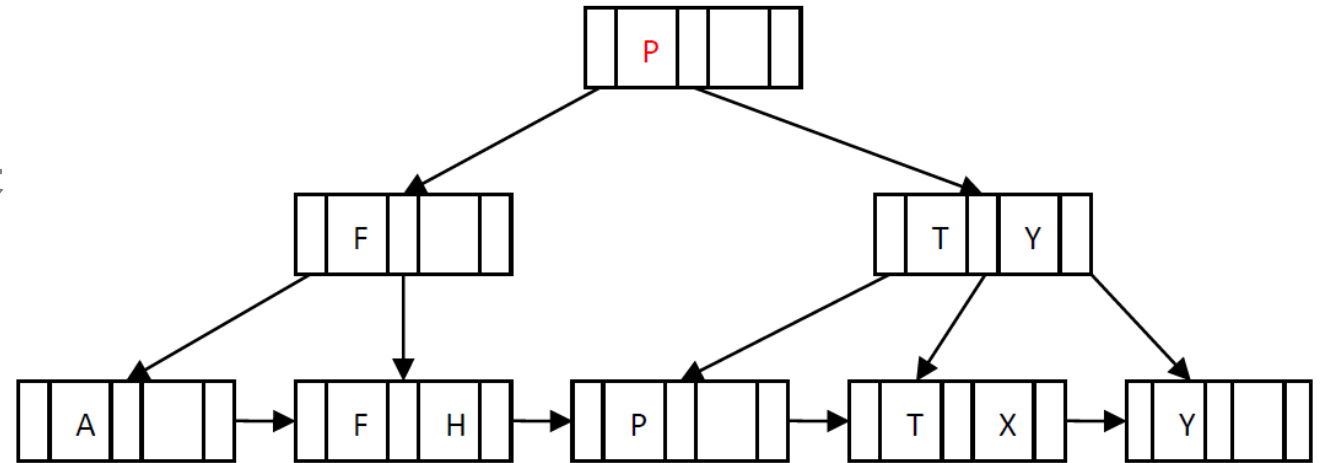
一道B+树的例题带你理解

- 这是一棵 ($n=3$) 的B+树:



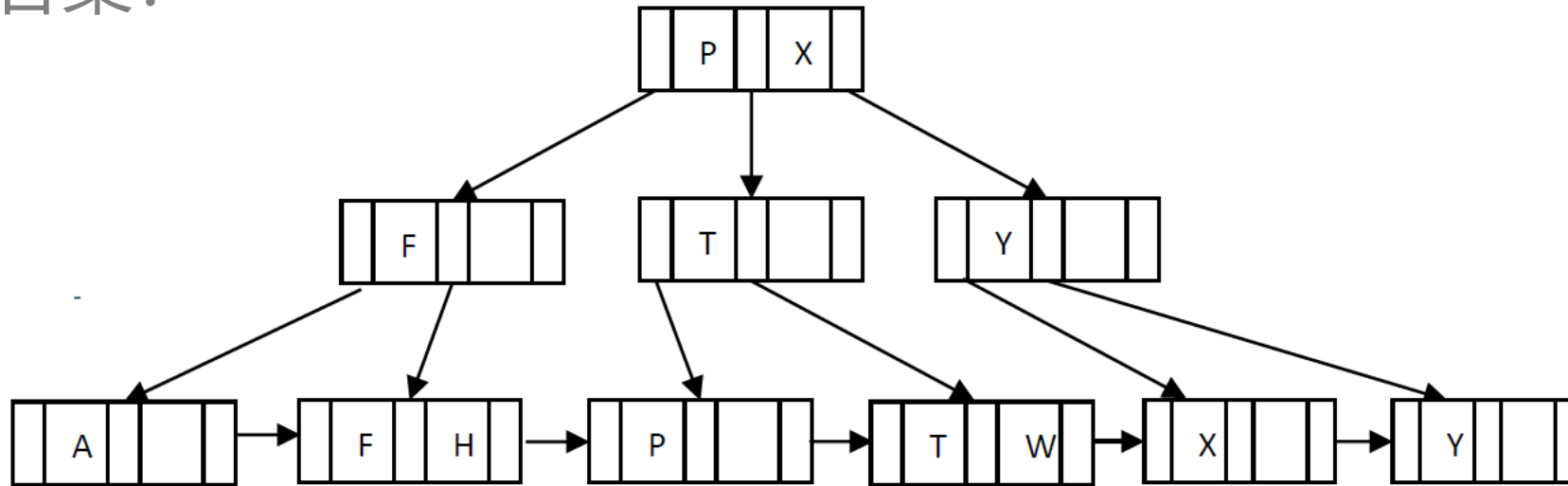
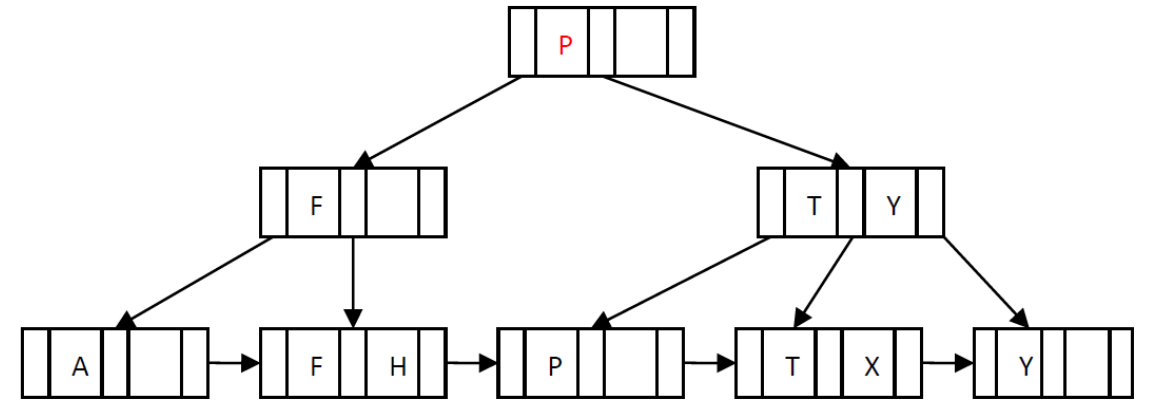
一道B+树的例题带你理解

- 请画出插入 'W' 后的结果



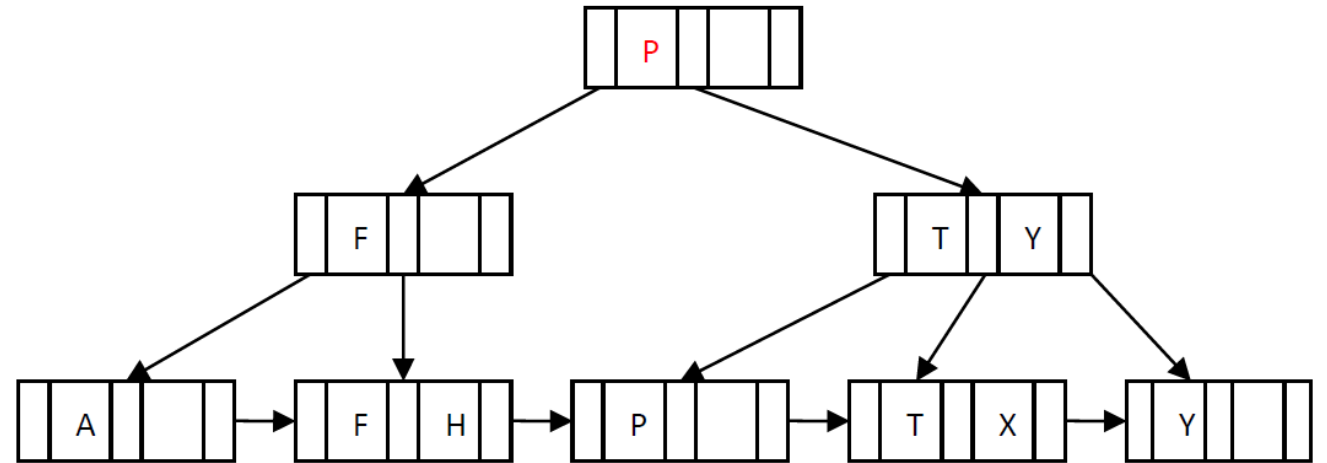
一道B+树的例题带你理解

- 请画出插入 'W' 后的结果
- 答案：



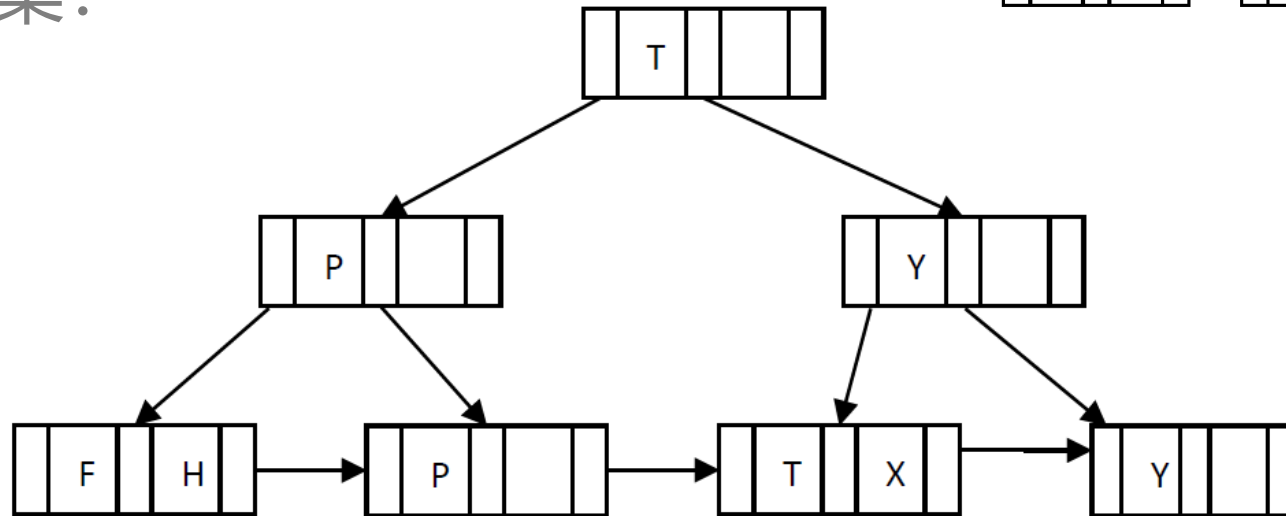
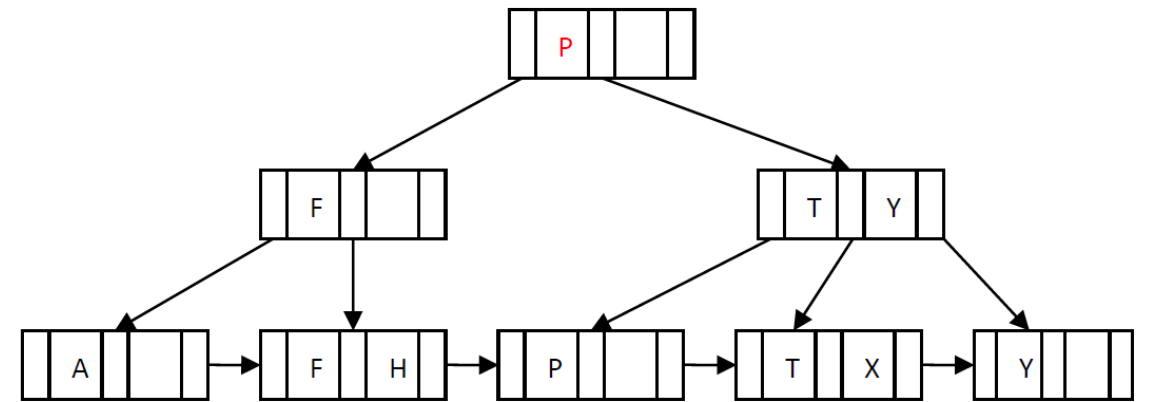
一道B+树的例题带你理解

- 请画出删除 'A' 后的结果



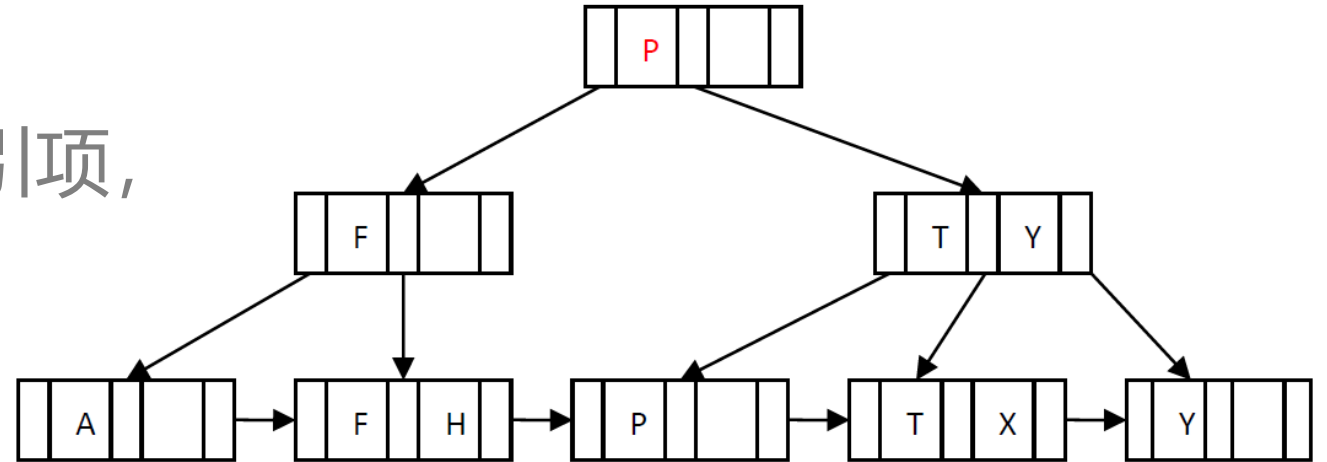
一道B+树的例题带你理解

- 请画出删除 'A' 后的结果
- 答案：



一道B+树的例题带你理解

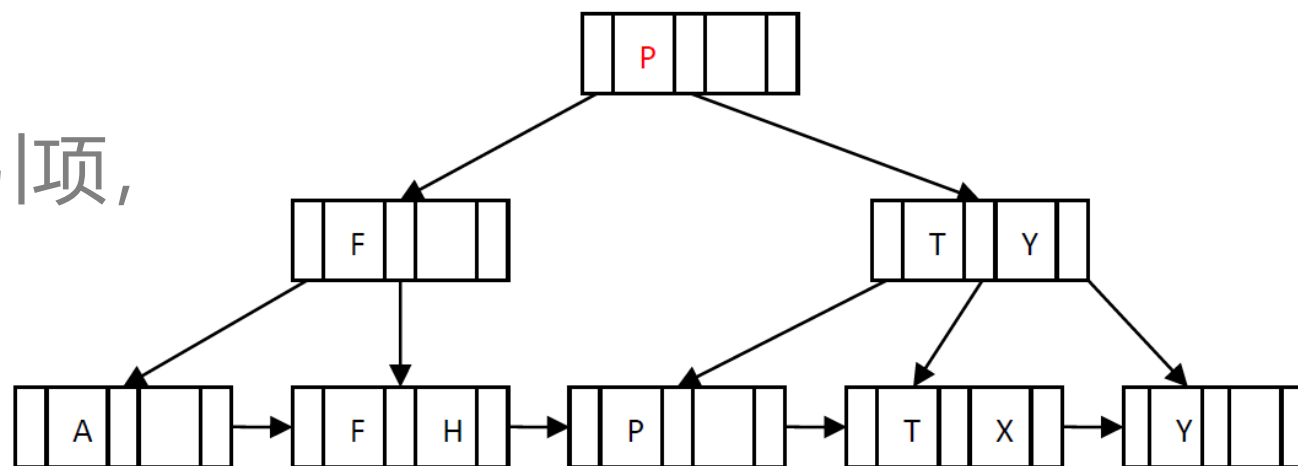
- 假设这棵B+树有1000个索引项，
请估计B+树的**高度**



一道B+树的例题带你理解

- 假设这棵B+树有1000个索引项，请估计B+树的高度

答案：

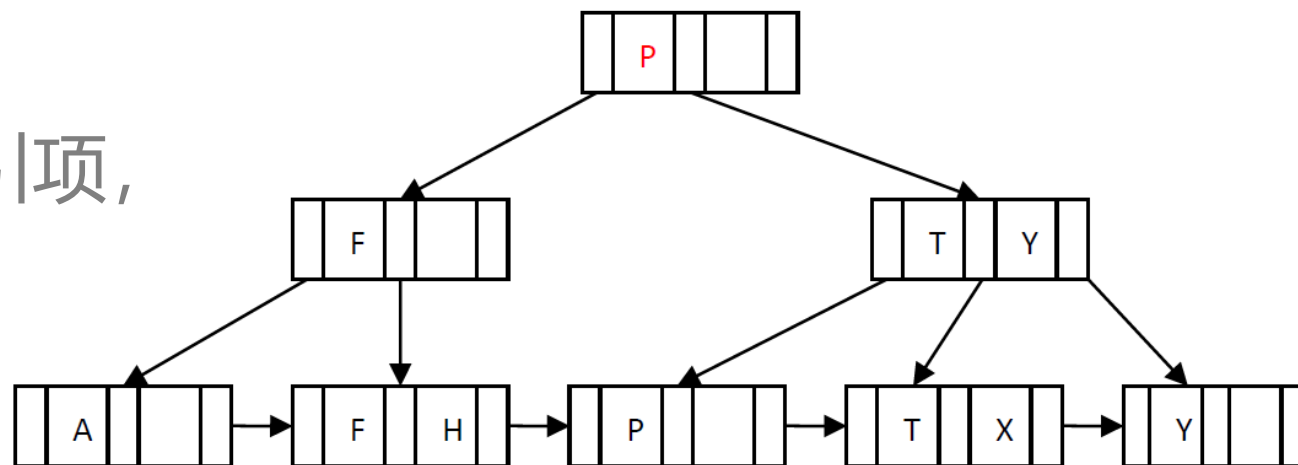


$$\left\lceil \log_3 \frac{1000}{2} \right\rceil + 1 \leq \text{height} \leq \left\lceil \log_{\lceil \frac{3}{2} \rceil} 1000 \right\rceil + 1$$

$$8 \leq \text{height} \leq 11$$

一道B+树的例题带你理解

- 假设这棵B+树有1000个索引项，请估计B+树的**节点数量**

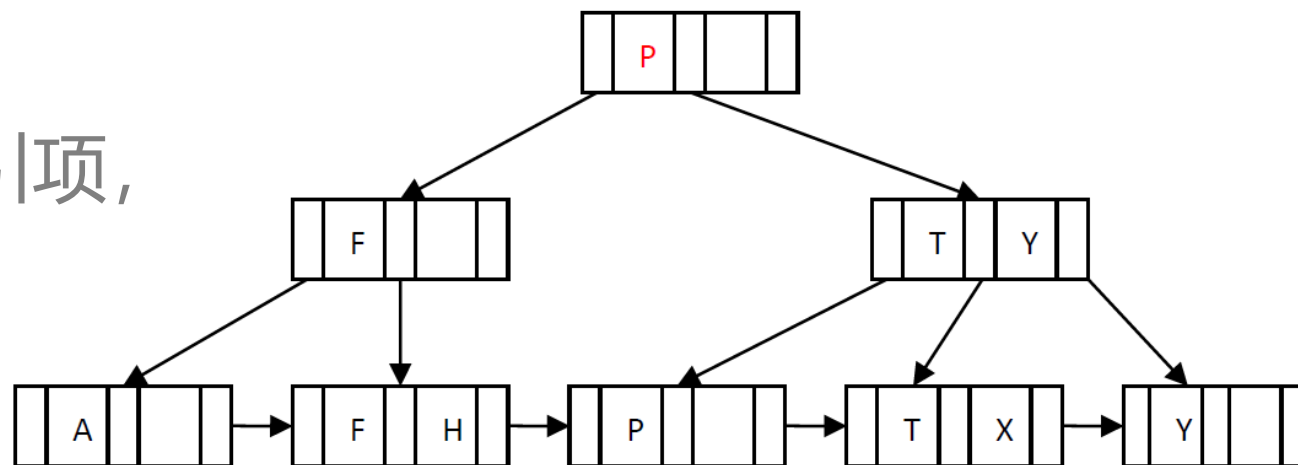


一道B+树的例题带你理解

- 假设这棵B+树有1000个索引项，请估计B+树的节点数量

• 答案：

- （提示：利用最后一层节点的数量倒推回去）



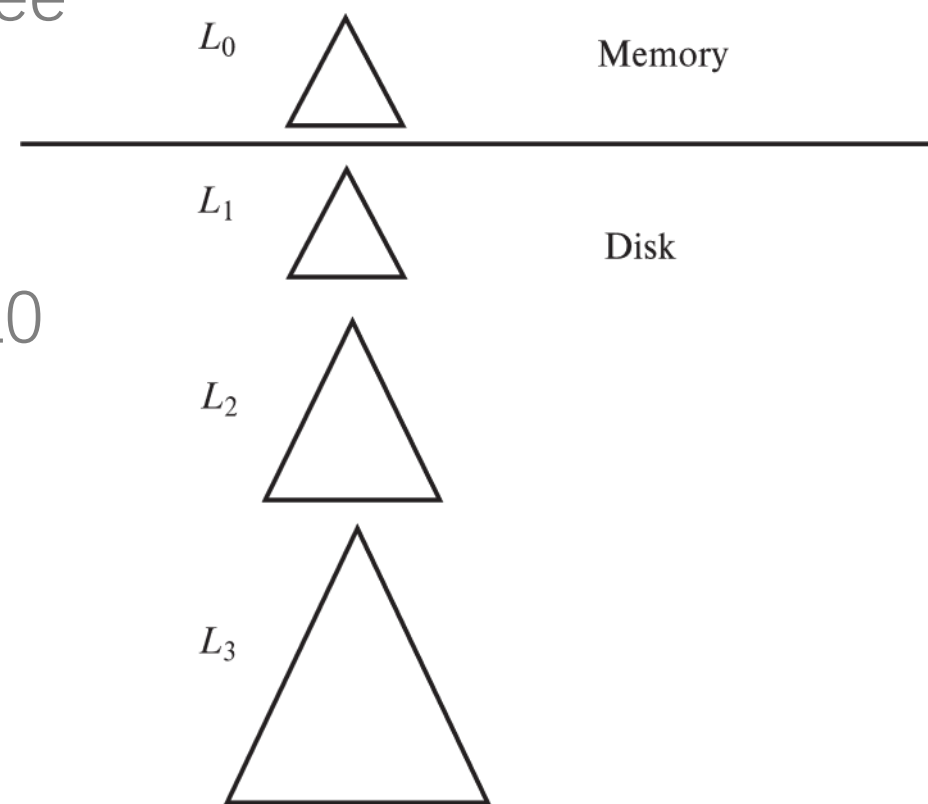
$$\text{size} \geq 500 + 167 + 56 + 19 + 7 + 3 + 1 = 753$$

$$\text{size} \leq 1000 + 500 + 250 + 125 + 63 + 31 + 15 + 7 + 3 + 1 = 1995$$

$$753 \leq \text{size} \leq 1995$$

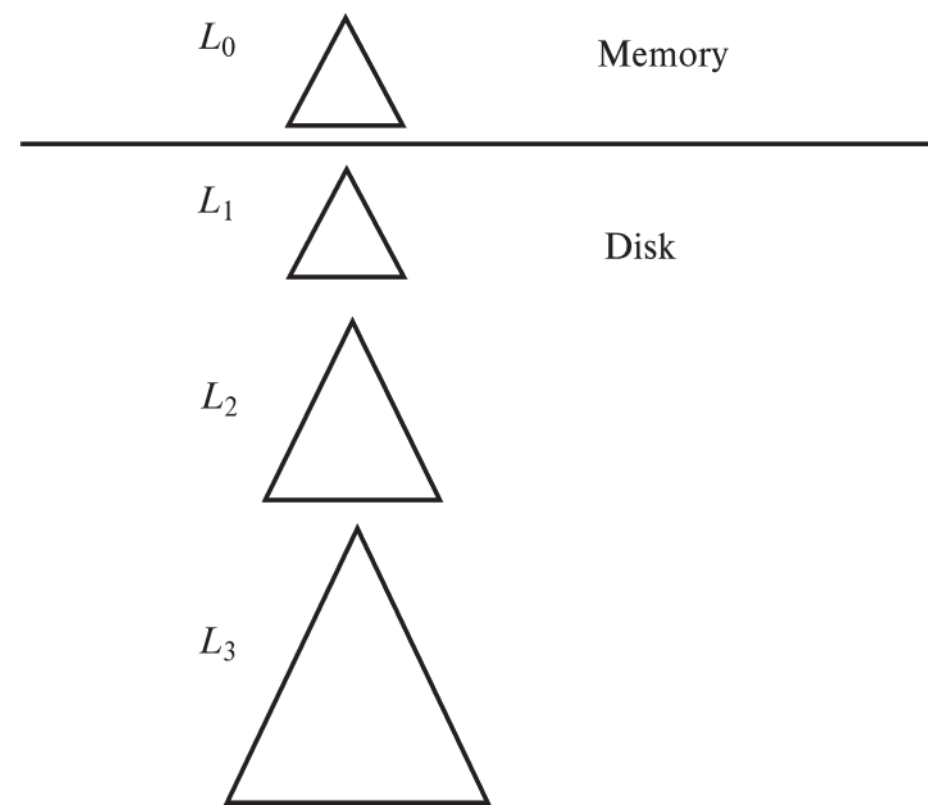
日志结构合并树

- Log Structured Merge (LSM) Tree
- 核心思想：
- 插入记录时首先插入到内存的L0树中，L0满了就将L0合并到L1，L1满了就合并到L2，……



Log Structured Merge (LSM) Tree

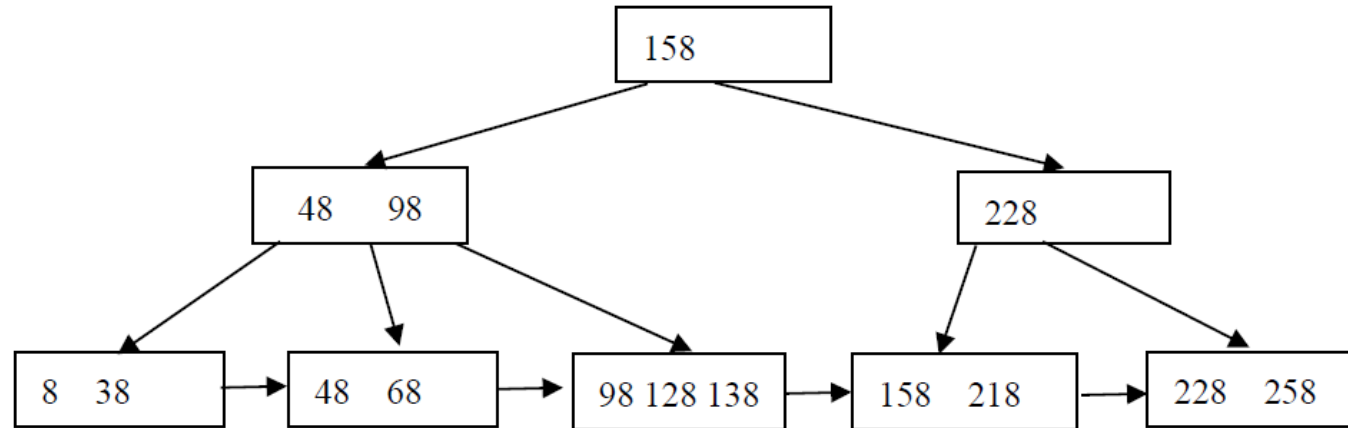
- Log Structured Merge (LSM) Tree将对数据的**修改增量**保持在**内存**中，达到指定的大小限制后将这些修改操作批量写入磁盘（由此**提升了写性能**），是一种基于硬盘的数据结构，与B-tree相比，能显著地减少硬盘磁盘臂的开销。当然凡事有利有弊，LSM树和B+树相比，LSM树**牺牲了部分读性能**，用来大幅提高写性能。



存储与索引练习1

Problem 5: B+ -Tree (12 points, 4 points per part)

For the following B+-tree ($n=4$):



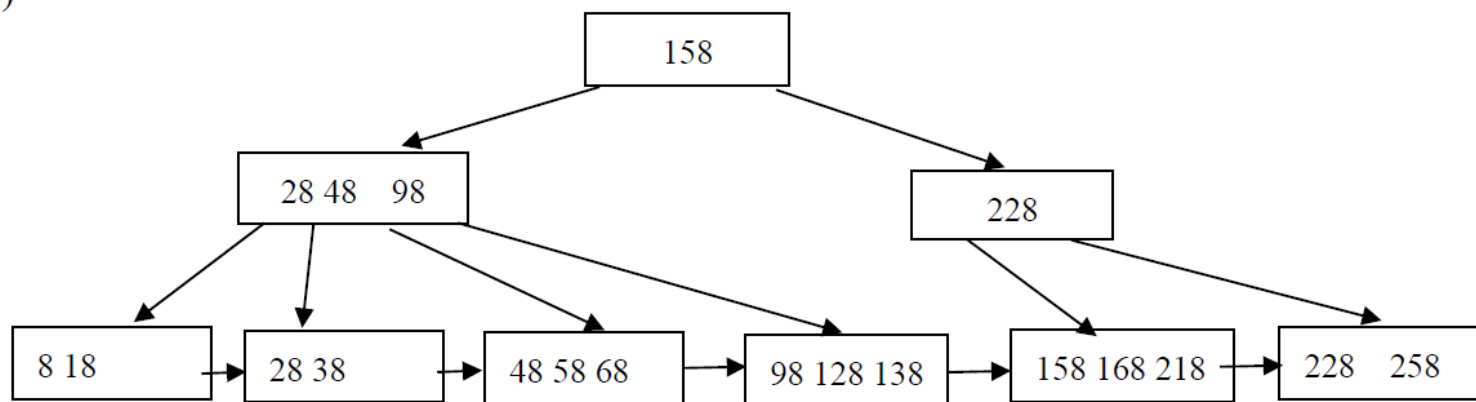
- (1) Draw the B+ -tree after inserting four entries 28,168, 58,18 sequentially.
- (2) Draw the B+ -tree after deleting four entries 8, 38, 98,128 from the original B+-tree sequentially.
- (3) Assuming (a) there are 3 blocks in main memory buffer for B+-tree operation, at first these blocks are empty. (b) Least Recently Used (LRU strategy) is used for buffer replacement. (c) each node of B+-tree occupies a block.
Please count the number of B+-tree blocks transferred to buffer in order to complete the operation in 1).

存储与索引练习1参考答案

Answers of Problem 5:

(12 points, 4 points per part)

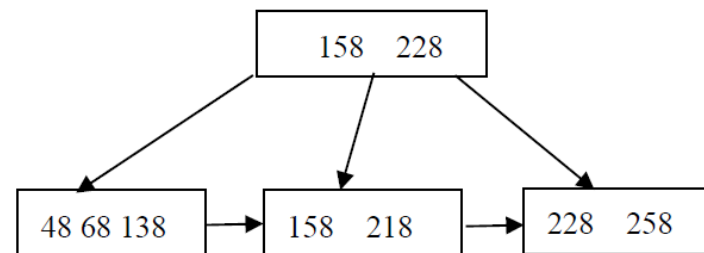
1)



评分细则:

每个entry插入错扣1分
图中缺箭头或线扣1分

2)



评分细则:

每个entry删除错扣1分
图中缺箭头或线扣1分

3) $3+2+2+1=8$

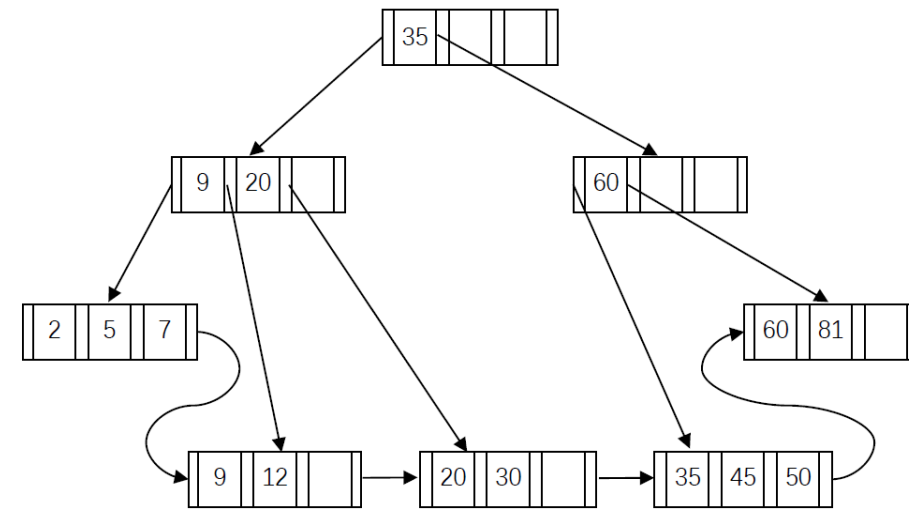
评分细则:

答案为7、9之一，扣1分
答案为5、6、10之一，扣2分
答案为3、4、11之一，扣3分
其他扣4分

存储与索引练习2

For the following B⁺-tree (n = 4), answer following questions:

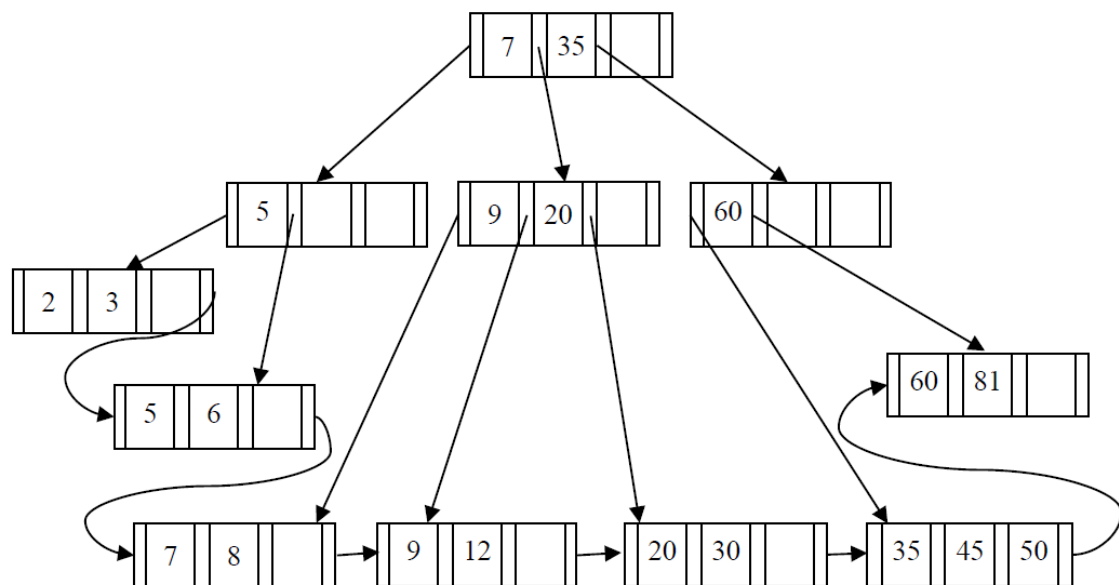
- 1) Show the structure of the tree after inserting sequentially 8, 6, and 3 into the original tree.
- 2) Show the structure of the tree after deleting sequentially 81 and 45 from the original tree.
- 3) If the height of tree is 5, please tell the minimal and maximal numbers of key values in the tree.
- 4) Assume that (a) there are 3 blocks in main memory buffer for B⁺-tree operation, and at first those blocks are empty; (b) the Least Recently Used (LRU) strategy is used for buffer replacement; and (c) each node of B⁺-tree occupies a block. Please count the number of B⁺-tree blocks transferred to buffer in order to complete the operations in 2) of this problem.



存储与索引练习2参考答案

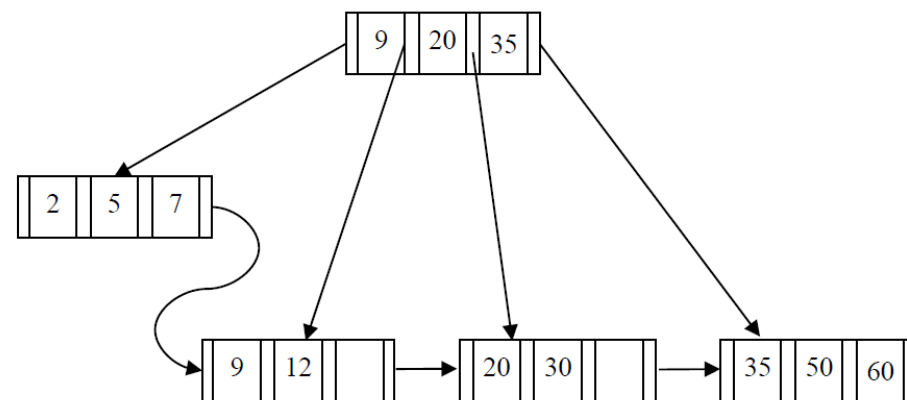
1)

After inserting 8, 6 and 3:



2)

After deleting 81 and 45:



3) Maximal number of key values: $4*4*4*4*3=768$

Minimal number of key values: $2*2*2*2*2=32$

评分细则:

公式列正确即给分

4) $(3 + 1) + 1 = 5$ 或 $(3 + 1) + 2 = 6$

评分细则:

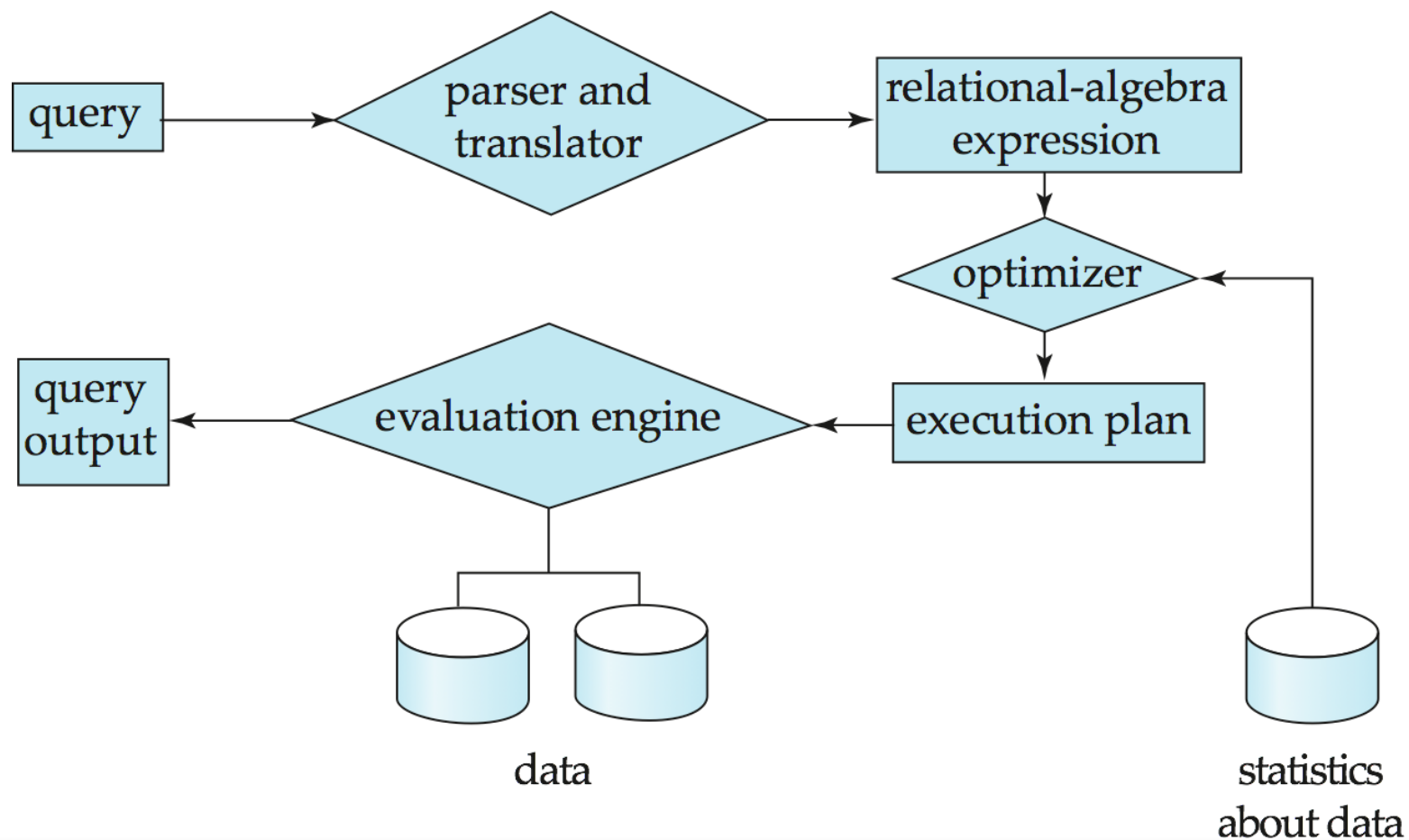
错, 扣 3 分



07

查询处理

查询处理流程



查询代价的衡量

- 查询代码主要包括：
 - 磁盘存取
 - 执行一个查询所需要的CPU时间
 - 在并行/分布式数据库系统中的通信代价
- 在大型数据库中，在**磁盘上存取数据**的代价通常是最主要的代价。

查询处理成本计算

Cost for b block transfers plus S seeks

$$b * t_T + S * t_S$$

- 在计算查询成本时，主要考虑数据传输时间和寻道时间，其他时间可以忽略

选择查询成本

- A1: 线性搜索
 - $T_s + Br * T_t$ 一次初始搜索加上br个块传输
- A2: B+树主索引, 码属性等值比较
 - $H*(T_t + T_s) + (T_s + T_t)$ 索引查找加上一次搜索和传输
- A3: B+树主索引, 非码属性等值比较
 - $H*(T_t + T_s) + Br*T_t$

选择查询成本

	算 法	开 销	原 因
A1	线性搜索	$t_i + b_i * t_r$	一次初始搜索加上 b_i 个块传输, b_i 表示在文件中的块数量
A1	线性搜索, 码属性 等值比较	平均情形 $t_i + (b_i/2) * t_r$	因为最多一条记录满足条件, 所以只要找到所需的记录, 扫描就可以终止。在最坏的情形下, 仍需要 b_i 个块传输
A2	B* 树主索引, 码属 性等值比较	$(h_i + 1) * (t_r + t_i)$	(其中 h_i 表示索引的高度)。索引查找穿越树的高度, 再加上一次 I/O 来取记录; 每个这样的 I/O 操作需要一次搜索和一次块传输
A3	B* 树主索引, 非码 属性等值比较	$h_i * (t_r + t_i) + b * t_r$	树的每层一次搜索, 第一个块一次搜索。 b 是包含具有指定搜索码记录的块数。假定这些块是顺序存储(因为是主索引)的叶子块并且不需要额外搜索
A4	B* 树辅助索引, 码 属性等值比较	$(h_i + 1) * (t_r + t_i)$	这种情形和主索引相似
A4	B* 树辅助索引, 非 码属性等值比较	$(h_i + n) * (t_r + t_i)$	(其中 n 是所取记录数。)索引查找的代价和 A3 相似, 但是每条记录可能在不同的块上, 这需要每条记录一次搜索。如果 n 值比较大, 代价可能会非常高
A5	B* 树主索引, 比较	$h_i * (t_r + t_i) + b * t_r$	和 A3, 非码属性等值比较情形一样
A6	B* 树辅助索引, 比较	$(h_i + n) * (t_r + t_i)$	和 A4, 非码属性等值比较情形一样

嵌套循环连接

```
for each 元组  $t_r$  in  $r$  do begin
  for each 元组  $t_s$  in  $s$  do begin
    测试元组对  $(t_r, t_s)$  是否满足连接条件  $\theta$ 
    如果满足, 把  $t_r \cdot t_s$  加到结果中
  end
end
```

$t_s \cdot t_r$ 表示将这两个元组拼接成一个元组

嵌套循环的计算代价:

设 r 和 s 中各包含 n_r 和 n_s 个元组, b_r 和 b_s 分别代表它们的磁盘块数。

最坏情况下, 缓冲区只能容纳每个关系的一个数据块:

- 数据块传输次数: 对 r 中的每条记录, 都必须对 s 做一次完整的扫描, 共需 $n_r * b_s + b_r$ 次
- 磁盘搜索次数: 内层关系 s 的每次扫描都需要一次磁盘搜索, 因为它是顺序读取的; 读取 r 一共需要 b_r 次磁盘搜索, 因此共需要: $n_r + b_r$

最好情况下, 缓冲区能够同时容纳两个关系:

此时每个数据库只需要读一次, 外加两次磁盘搜索。

- 数据块传输次数: $b_r + b_s$
- 磁盘搜索次数: 2次

如果其中一个关系能够完全放在内存中, 那么把它作为内层关系可以带来好处, 因为内层循环只需要读取一次。(与都能装入内存时一样)

- 数据块传输次数: $b_r + b_s$
- 磁盘搜索次数: 2次

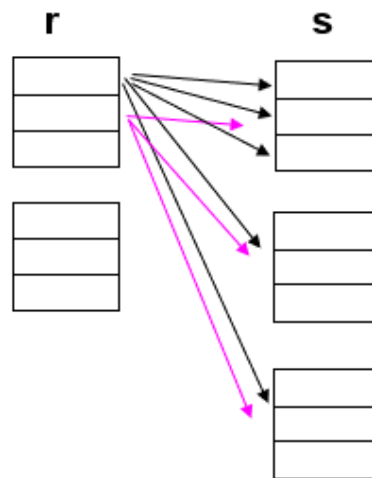
块嵌套循环连接

```

for each 块  $B_r$  of  $r$  do begin
  for each 块  $B_s$  of  $s$  do begin
    for each 元组  $t_r$  in  $B_r$  do begin
      for each 元组  $t_s$  in  $B_s$  do begin
        测试元组对  $(t_r, t_s)$  是否满足连接条件
        如果满足, 把  $t_r \cdot t_s$  加到结果中
      end
    end
  end
end

```

$t_s \cdot t_r$ 表示将这两个元组拼接成一个元组

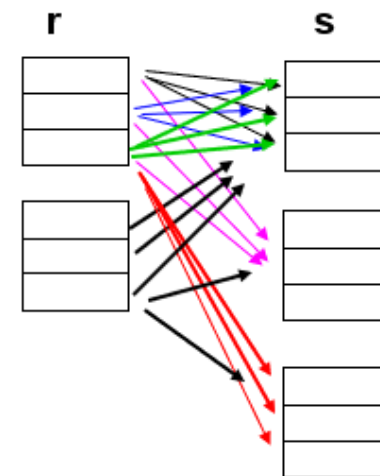


Nested-Loop Join

BT Cost = $6 * 3 + 2 = 20$

Seeks = $6 + 2 = 8$

outer relation Inner relation



$M=3$

Block Nested-Loop Join

Block transfer cost = $2 * 3 + 2 = 8$

Seeks = 4

块嵌套循环连接

在内存中处理

```
for each block  $B_r$  of  $r$  do begin
  for each block  $B_s$  of  $s$  do begin
    for each tuple  $t_r$  in  $B_r$  do begin
      for each tuple  $t_s$  in  $B_s$  do begin
        Check if  $(t_r, t_s)$  satisfy the join condition
        if they do, add  $t_r \cdot t_s$  to the result.
      end
    end
  end
end
```

块嵌套循环的计算代价:

设 r 和 s 中各包含 n_r 和 n_s 个元组, b_r 和 b_s 分别代表它们的磁盘块数。

最坏情况下, 缓冲区只能容纳每个关系的一个数据块:

- 数据块传输次数: 对 r 中的每个块, 都必须对 s 做一次完整的扫描, 共需 $b_r * b_s + b_r$ 次
- 磁盘搜索次数: 内层关系 s 的每次扫描都需要一次磁盘搜索, 因为它是顺序读取的; 读取 r 一共需要 b_r 次磁盘搜索, 因此共需要: $2b_r$

最好情况下, 缓冲区能够同时容纳两个关系:

此时每个数据库只需要读一次, 外加两次磁盘搜索。

- 数据块传输次数: $b_r + b_s$
- 磁盘搜索次数: 2次

如果其中一个关系能够完全放在内存中, 那么把它作为内层关系可以带来好处, 因为内层循环只需要读取一次。(与都能装入内存时一样)

- 数据块传输次数: $b_r + b_s$
- 磁盘搜索次数: 2次

如果两个关系都不能放进内存中, 则**小关系**作为**外层关系**连接效率更高

索引嵌套循环连接

索引嵌套循环的计算代价:

设 r 和 s 中各包含 n_r 和 n_s 个元组, b_r 和 b_s 分别代表它们的磁盘块数。
 t_T 是磁盘传输速度, t_S 是磁盘搜索速度

最坏情况下, 缓冲区只能容纳关系 r 的一块和索引的一块:

- 读取数据 r 需要 b_r 次I/O操作(包括磁盘传输+磁盘搜索),
- 对于关系 r 上的每个元组, 都必须在 s 上进行索引查找

则总的时间代价为: $b_r * (t_S + t_T) + n_r * c$, 其中 c 代表使用连接条件对关系 s 进行单词选择操作的代价

如果两个关系 s 和 r 上均有索引, 使用较少的作为外层循环效率较好。

排序归并连接

```
pr := r 的第一个元组的地址;  
ps := s 的第一个元组的地址;  
while ( ps ≠ null and pr ≠ null ) do  
  begin  
    ts := ps 所指向的元组;  
    Ss := | ts |;  
    让 ps 指向关系 s 的下一个元组;  
    done := false;  
    while ( not done and ps ≠ null ) do  
      begin  
        t's := ps 所指向的元组;  
        if ( t's[ JoinAttrs ] = ts[ JoinAttrs ] )  
          then begin  
            Ss := Ss ∪ | t's |;  
            让 ps 指向关系 s 的下一个元组;  
          end  
        else done := true;  
      end  
    end  
    tr := pr 所指向的元组;  
    while ( pr ≠ null and tr[ JoinAttrs ] < ts[ JoinAttrs ] ) do  
      begin  
        让 pr 指向关系 r 的下一个元组;  
        tr := pr 所指向的元组;  
      end  
    while ( pr ≠ null and tr[ JoinAttrs ] = ts[ JoinAttrs ] ) do  
      begin  
        for each ts in Ss do  
          begin  
            将 tr ⋈ ts 加入结果中;  
          end  
        让 pr 指向关系 r 的下一个元组;  
        tr := pr 所指向的元组;  
      end  
    end  
  end  
end
```

遍历S_s，顺序取出R∩S属性上相等的元组，存放在S_s中

再次循环

遍历R，顺序取出R∩S属性上与S_s中元组的值相等的元组，将其与S_s中的元组连接加入结果集中

归并连接嵌的计算代价分析:

设r和s中各包含 n_r 和 n_s 个元组， b_r 和 b_s 分别代表它们的磁盘块数。
设为每个关系分配 b_b 个缓冲块

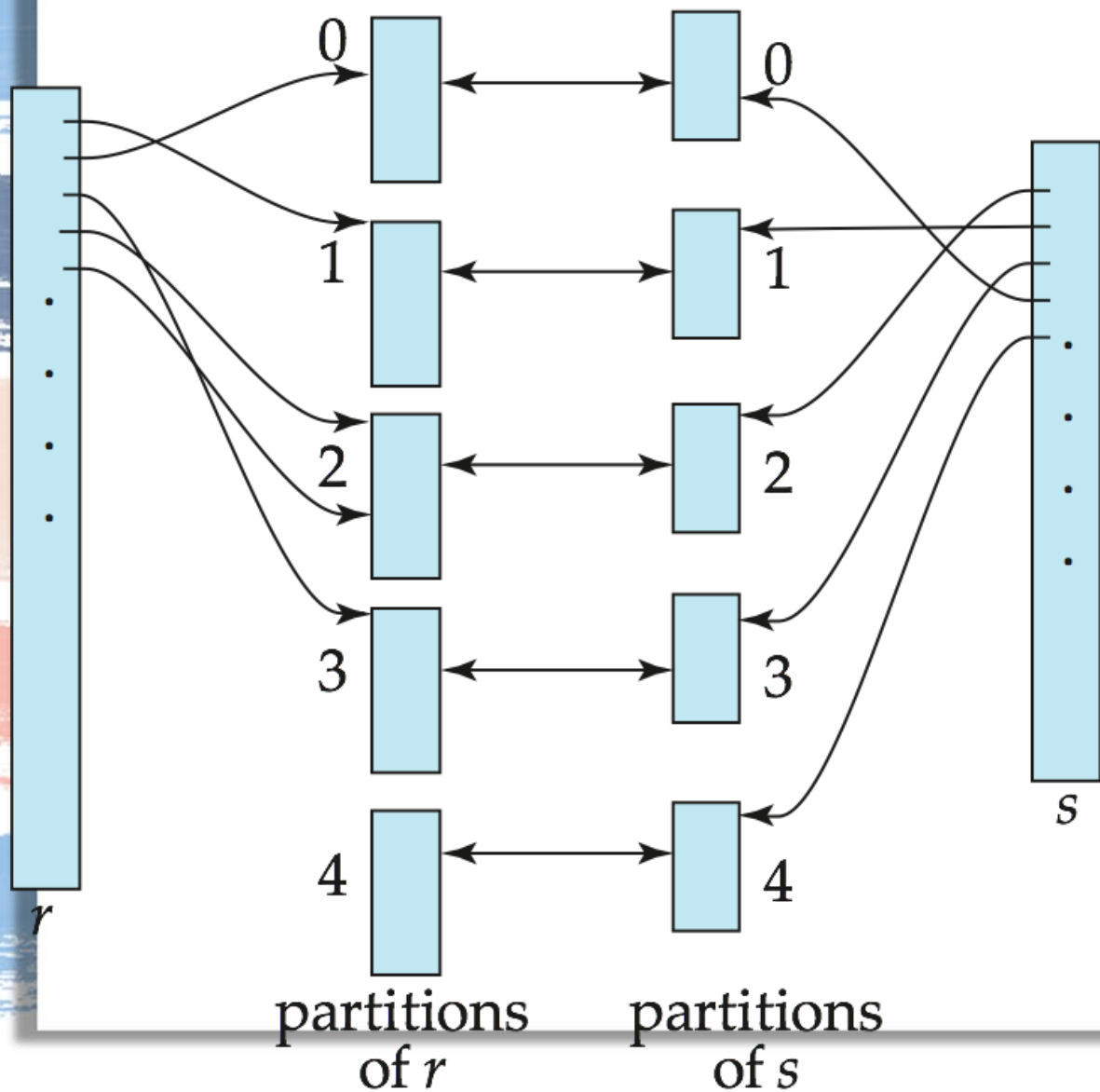
假设排序好的，

磁盘传输次数：因为已经排序，所以每个元组、每个块只要读一次就可以。
即：每个关系文件读一次。共需： $b_s + b_r$

磁盘搜索次数： $[b_s/b_b] + [b_r/b_b]$

如果关系尚未排序，那么必须先对它们进行排序，排序的代价也要考虑上。

散列连接



Hash join: 使用hash函数进行join

- h maps JoinAttrs values to $\{0, 1, \dots, n_h\}$, 将两个关系进行比较和同类型的匹配
- cost of hash-join
 - block transfer: $3(b_r + b_s) + 4n_h$
 - partition: 读 $b_r + b_s$ blocks 写 $(b_r + b_s) + 2n_h$ blocks
 - join: 读 $(b_r + b_s) + 2n_h$
 - seeks: $2(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil)$
 - 如果所有东西都能放进主存里, 则 $n_h = 0$ 并且不需要partition
 - 需要partition的时候 $cost = 2(b_r + b_s[\log_{M-1}(b_s) - 1]) + b_r + b_s$

散列连接

在不考虑散列表溢出发生，也不需要递归划分的情况：

- 磁盘块传输次数：

- 两个关系 r, s 的划分需要对这两个关系分别进行一次完整的读入和写回，共： $2(b_s + b_r)$ 次块传输
- 在构造和探查阶段，每个划分读入一次，这又需要 $(b_s + b_r)$ 次块传输。划分所占用的块可能比 $(b_s + b_r)$ 多，因为有的块是部分满的。由于 n_h 个划分中的每个划分都可能有一个部分满的块，而这个块需要写回、读入各一次，因此对于每个关系而言存取这些部分满的块至多增加 $2n_h$ 次的开销。
- ==> 总的传输次数： $2(b_s + b_r) + 4n_h$

- 磁盘搜索次数：

- 假设为输入和输出缓冲区分配了 b_b 个块，划分总共需要 $2([b_s/b_b] + [b_r/b_b])$ 次磁盘搜索。
- 在构造和探查阶段，每个关系中 n_h 个划分中的每一个划分仅需要一次磁盘搜索，因为每个划分都可以顺序的读取
- ==> 总的搜索次数： $2([b_s/b_b] + [b_r/b_b]) + 2n_h$

需要递归划分的情况：

递归划分中，每一趟预计可将划分的大小减为原来的 $1/(M-1)$ ；划分所需的趟数预计为： $\lceil \log_{M-1}(b_s) \rceil - 1$

- 磁盘块传输次数：

- 每一趟划分中，都需要对关系 s 进行完整的读入和写出，因此需要 $2b_s [\log_{M-1}(b_s) - 1]$ 次块传输
- 对关系 r 进行划分所需要的趟数与划分关系 s 一样，每一趟划分中也需要对关系 r 进行完整的读入和写出，需要 $2b_r [\log_{M-1}(b_s) - 1]$ 次块传输
- 在构造和探查阶段，每个划分读入一次，这又需要 $(b_s + b_r)$ 次块传输
- ==> 总的传输次数： $2(b_s + b_r) [\log_{M-1}(b_s) - 1] + (b_s + b_r)$

- 磁盘搜索次数：

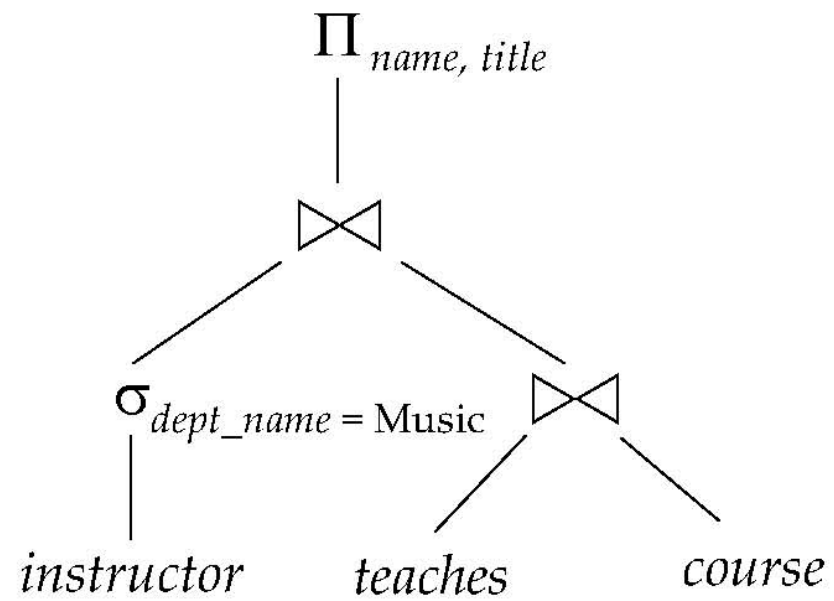
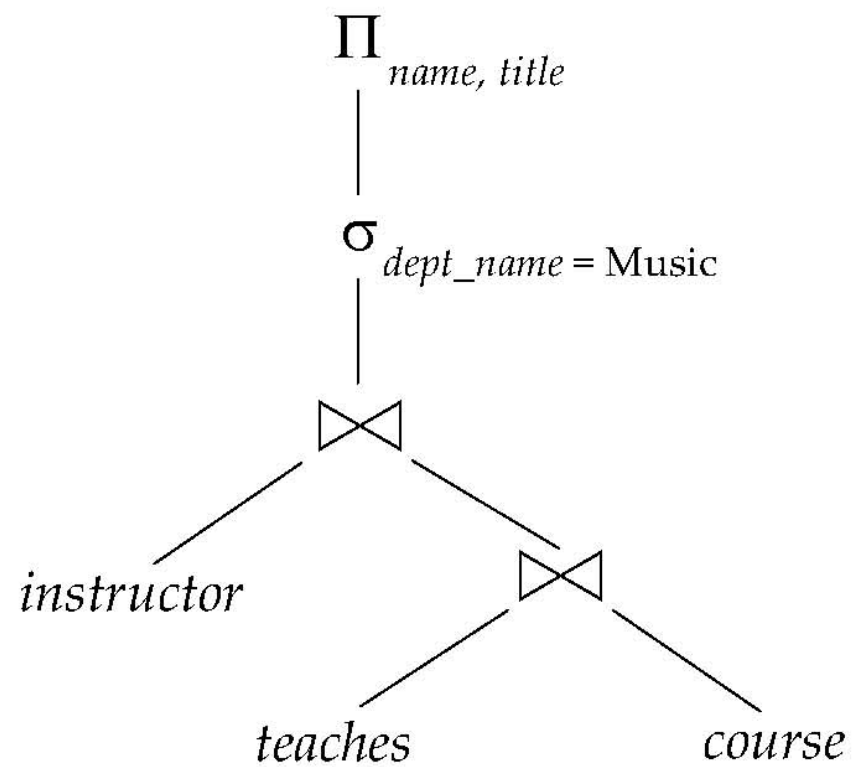
- 假设为输入和输出缓冲区分配了 b_b 个块，忽略构造和探查过程中相对较少的磁盘搜索，
- ==> 总的搜索次数： $2([b_s/b_b] + [b_r/b_b]) [\log_{M-1}(b_s) - 1]$



08

查询优化

查询优化



查询优化的等价规则

1. **Conjunctive selection** operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. **Selection** operations are **commutative**.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

4. Selections can be combined with Cartesian products and theta joins.

- a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$

- b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

查询优化的等价规则

5. Theta-join operations (and natural joins) are **commutative**.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

6. (a) **Natural join** operations are **associative**:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

- (b) **Theta joins** are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

查询优化的成本估计

- 选择成本

- $\sigma_{A=a}(r)$: 假设取值是均匀分布的, 并假设关系 r 的一些记录的属性 $A=a$, 则可估计选择结果有 $n_r/V(A, r)$ 个元组

如果在属性 A 上存在一个直方图, 则可以定位 a 所在的区间, 然后使用区间内元组的个数, 以及值的个数来代替 n 和 V

- $\sigma_{A \leq v}(r)$: 假设取值是均匀分布的, 属性 A 的最小值 $\min(A, r)$ 和最大值 $\max(A, r)$

可以对满足条件 $A \leq v$ 的记录数进行下列估计: 若 $v < \min(A, r)$, 则为 0; 若 $v \geq \max(A, r)$, 则为 n_r ; 否则, 为:

$$n_r \cdot \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$$

查询优化的成本估计

- 复杂选择成本

Conjunction: $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$. Assuming independence, estimate of tuples in the result is: $n_r * \frac{s_1 * s_2 * \dots * s_n}{n_r^n}$

Disjunction: $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$. Estimated number of tuples:

$$n_r * \left(1 - \left(1 - \frac{s_1}{n_r} \right) * \left(1 - \frac{s_2}{n_r} \right) * \dots * \left(1 - \frac{s_n}{n_r} \right) \right)$$

Negation: $\sigma_{\neg \theta}(r)$. Estimated number of tuples: $n_r - \text{size}(\sigma_{\theta}(r))$

查询优化的成本估计

- 连接成本

- 笛卡尔积的情况下，关系R,S的join最终元组的个数为 $n_r \times n_s$
- 如果 $R \cap S$ 为空，则自然连接的结果和笛卡尔积的结果相同
- 如果非空，且 $R \cap S$ 是R的key，则R,S的自然连接最终结果中的元组个数不会超过r
- 如果 $R \cap S$ 的结果是S到R的外键，则最后的元组数和s中的元组数相同
- 一般情况 自然连接的最终结果的size估计值为 $\frac{n_r \times n_s}{\max(V(A,r), V(A,s))}$

查询优化练习1

card(cno:char(5), name:char(8), depart:char(10), balance:integer)
pos(pno:char(4), campus:char(8), location:char(10))
detail(cno:char(5), pno:char(4), cdate:date, ctime:time,
amount:integer,
remark:char(10))

Following tables show an instance of the database:

card

cno	name	depart	balance
c0001	张帅	CS	100
c0002	李丽	EN	200
c0003	王浩	CS	300
c0004	刘萌	CS	400
c0005	赵亮	MA	500

Problem 6: Query Processing (16 points, 4 points per part)

For the relational schemas of the campus card database given in **problem 1**, there are following assumptions:

- $n_{\text{card}}=10,000$, $n_{\text{pos}}=100$, $n_{\text{detail}}=10,000,000$
- $l_{\text{card}}=25$, $l_{\text{pos}}=22$, $l_{\text{detail}}=29$
- $V(\text{campus}, \text{pos}) = 6$, $V(\text{location}, \text{pos}) = 20$
- $V(\text{depart}, \text{card}) = 100$, $V(\text{name}, \text{card}) = 5000$
- The value of attribute **cdate** in **detail** table is uniformly distributed between '2017-01-01' and '2017-12-31'.
- block size is 4K bytes.
- size of B+-tree pointer is 4 bytes.
- **card** and **detail** tables are stored as sequential files based on search key **cno**.
- there is a B+-tree index on **detail(cno)**.

1) Estimate the size (i.e. number of records) returned by following SQL statement :

```
select d1.cno, d2.cno
from detail d1, detail d2
where d1.pno=d2.pno and d1.cdate=d2.cdate and
      d1.cdate between '2017-05-01' and '2017-07-31'
```

- (2) Estimate the number of blocks of **card** and **detail** tables respectively.
- (3) Estimate the height of the B+-tree index on **detail(cno)**.
- (4) Estimate the cost for evaluating expression " $\sigma_{\text{name}='张帅'}(\text{card}) \bowtie \text{detail}$ " using file scan for σ operation followed by indexed-loop join method for $\bowtie$ operation. (Hint: the cost is measured by number of blocks transferred to main memory and times to seek disk.)

查询优化练习1参考答案

Answers of Problem 6:

(16 points, 4 points per part)

评分细则: 每个运算结果错扣 1 分

1) $(10000000 * 10000000) / (100 * 365) * 3/12 = 684.93M$

评分细则:

三个条件: 1/100, 1/365, 3/12 错各扣1分

少一个10000000未乘扣1分

按实际天数天数92/365代替3/12, 也对。

2) Record number per block of card = $4096/25 = 163$

Blocks of card = $10000/163 = 61.3 \rightarrow 62$

Record number per block of detail = $4096/29 = 141.24 \rightarrow 141$

Blocks of detail = $10000000/141 = 70922$

评分细则:

按不同的估算公式(要合理), 答案略有差异, 不扣分。

3) Fan-out rate n of the B+-tree = $(4096-4)/(5+4) + 1 = 455$

4) Min height of B+tree = $\log_{455}(10000) \rightarrow 2$ (向上取整)

Max height of B+tree = $\log_{228}(10000/2) + 1 = \rightarrow 2$ (向下取整)

So height of B+tree = 2

评分细则:

B+-tree高度为3, 不扣分。

5) Cost for evaluating σ operation (2分, 各1分)

block transfer = $62 t_T$

seek time = $1 t_S$

cost for the natural join operation (2分, t_S 和 t_T 各1分)

return number of σ name='张帅' (card) = $(10000/5000) = 2$

block number for each card cno in detail = $(10000000/10000)/141 = 7.09 = 8$

cost for the natural join operation = $2 * (2 t_S + 2 t_T + 1 t_S + 8 t_T)$

= $2 * (3 t_S + 10 t_T) = 6 t_S + 20 t_T$

pipeline evaluation:

Total cost = $(1 t_S + 62 t_T) + (6 t_S + 20 t_T) = 7 t_S + 82 t_T$

评分细则:

选择和连接操作的 t_S 和 t_T 各1分

若采用materialized evaluation方法, t_S 多1, t_T 多2

若只给出一般公式, 而未有具体数据带入, 给1分。

查询优化练习2

Movie(title, type, director)
Comment(title, user_name, grade)

Problem 6: Query Processing (12 points, 4 points each)

For the relational schemas in problem 1, there are following assumptions:

Number of tuples, movie: 5,000, comment: 1,000,000;

Blocking factor, movie: 50, comment: 100;

Number of distinct values, $V(\text{director, movie}) = 500$, $V(\text{grade, comment}) = 5$;

Block size is 4K bytes;

Movie has a B^+ -tree index on title, and each index block contains 60 entries (i.e. $n=60$)

Answer following questions (*We do not consider cost to write output to disk*):

- 1) Estimate the size of the result of 1) in **Problem 1**.
- 2) Suppose that the *block nested-loop join* is used to implement $\text{movie} \bowtie \text{comment}$, buffer in main memory has 12 blocks, and movie is chosen as inner relation. In order to obtain the best performance, how to assign buffer blocks to movie and comment respectively? Please estimate the number of block accesses and the number of seeks required by the solution.
- 3) Suppose that the *index nested-loop join* is used to implement $\sigma_{\text{director}=\text{"Yimou Zhang"}}(\text{movie}) \bowtie \text{comment}$. Please estimate the number of block accesses and the number of seeks required by the solution (assume the worst case of memory, and that one extra buffer block is used for the root index block and it needs to be read from disk only once).

查询优化练习2参考答案

Problem 6: Query Processing (12 points, 4 points each)

1) $5,000/500/5 = 2$

评分细则:

2,4 均可, 没有计算扣 2 分

400 扣 1 分, 10 扣 1 分

2) Number of blocks of movie is $5000/50=100$

Number of blocks of comment is $1,000,000/100=10,000$

Since the equi-join attribute title forms a key on inner relation, we can stop inner loop on the first match.

Assign 10 blocks to comments, 1 block to movies, and 1 block for output.

Number of block accesses: $(10000/10)*100+10000 = 110000$ 或

$$10000 * 100/10 + 100 = 100100$$

Number of seeks: $2*10000/10=2000$

3) Minimal height = $\log_{60}(5000) \rightarrow 3$ (向上取整)

Max height = $\log_{30}(5000) \rightarrow 3$ (向上取整)

So, the height of the B^+ -tree index on movie(title) is 3.

Number of block accesses: $10000+1000000/500*3+1$

Number of seeks: $10000+1000000/500*3+1$

评分细则:

不是白卷且答案合理均给分



09

并发控制

事务

事物的四个性质ACDI:

- 原子性 Atomicity

事务中的所有步骤只能完全执行(commit)或者回滚(rollback)

- 持久性 Durability

更新之后哪怕软硬件出了问题，更新的数据也必须存在

- 一致性 Consistency

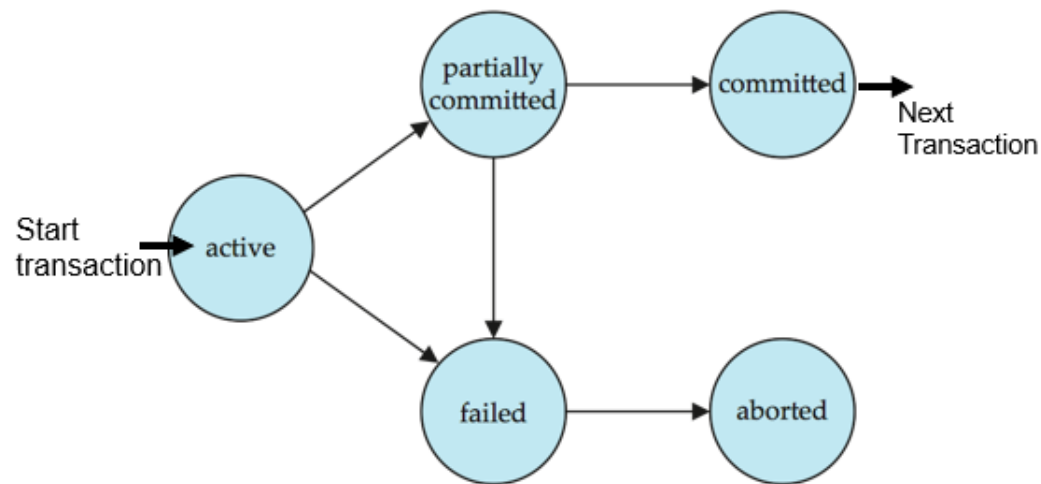
单独执行事务可以保持数据库的一致性

- 独立性 Isolation

事务在并行执行的时候不能感知到其他事务正在执行，执行中间结果对于其他并发执行的事务是隐藏的

事务

事务的状态



- active 初始状态，执行中的事务都处于这个状态
- partially committed 在最后一句指令被执行之后
- failed 在发现执行失败之后
- aborted 回滚结束，会选择是重新执行事务还是结束
- committed 事务被完整的执行

事务调度

Schedules 调度

- 一系列用于指定并发事务的执行顺序的指令
 - 需要包含事务中的所有指令
 - 需要保证单个事务中的指令的相对顺序
- 事务的最后一步
 - 成功执行，最后一步是commit instruction
 - 执行失败最后一步是abort instruction
- serial schedule 串行调度：一个事务调度完成之后再下一个
- equivalent schedule 等价调度：改变处理的顺序但是和原来等价

事务调度

可串行化调度

多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同，称这样的调度策略为可串行化调度。

事务调度

可串行化调度

多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同，称这样的调度策略为可串行化调度。

- 基本假设：事务不会破坏数据库的一致性，只考虑读写两种操作
- 冲突可串行化 conflict serializability
 - 同时读不引发冲突，而读写并行或者同时写会引发冲突
 - **conflict equivalent**: 两个调度之间可以通过改变一些不冲突的指令来转换，就叫做冲突等价
 - **conflict serializable**: 冲突可串行化：当且仅当一个调度S可以和一个串行调度等价
- **Precedence graph** 前驱图
 - 图中的顶点是各个事务，当事务 T_i, T_j 冲突并且 T_i 先访问出现冲突的数据的时候，就画一条边 $T_i \rightarrow T_j$
 - 一个调度是冲突可串行化的当且仅当前驱图是无环图
 - 对于无环图，可以使用**拓扑逻辑排序**获得一个合适的执行顺序

事务调度

前驱图绘制

前驱图用于冲突可串行的判断，前驱图无环则是冲突可串行化。

Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j , and at least one of these instructions wrote Q .

1. $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
2. $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They conflict.
3. $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They conflict
4. $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They conflict

事务调度

基于锁的协议

- lock是一种控制并发访问同一数据项的机制
- 两种lock mode
 - exclusive(X)：表示数据项可以读和写，用lock-X表示
 - shared(S)：表示数据项只能读，用lock-S 表示
- 两个事务的冲突矩阵：

	S	X
S	true	flase
X	false	false

- 如果请求的锁和其他事务对这个数据项已经有的锁不冲突，那么就可以给一个事务批准一个锁
- 对于一个数据项，可以有任意多的事务持有S锁，但是如果有一个事务持有X锁，其他的事务都不可以持有这个数据项的锁
- 如果一个锁没有被批准，就会产生一个请求事务，等到所有冲突的锁被release之后再申请

事务调度

基于锁的协议

- dead lock 死锁：两个事务中的锁互相等待造成事务无法执行，比如事务2的锁需要事务1先release，但是事务1的release步骤再事务2的申请锁后面，就会造成事务12的死锁
- Starvation 饥荒：一个事务在等一个数据项的Xlock，一群别的事务在等他release，造成饥荒

二阶段锁协议

Two-Phase Locking Protocol 二阶段锁协议：确保冲突可串行化的调度

- 两个阶段 growing和shrinking, growing只接受锁而不释放, shrinking反之
- 无法解决死锁的问题
- **strict two-phase locking**
 - 每个事务都要保持所有的exclusive锁直到结束
 - 为了解决级联回滚的问题
- **Rigorous two-phase locking**
 - 所有的锁必须保持到事务commit或者abort

并发控制练习1

Consider following three transactions:

T1: read(A)

Read(B)

Write(B)

T2: read(B)

Read(A)

Write(A)

T3: read(A)

Write(A)

1) For the following schedule, please draw the precedence graph, and explain whether it is conflict serializable.

T1	T2	T3
Read(A)		
Read(B)		
		Read(A)
		Write(A)
	Read(B)	
	Read(A)	
Write(B)		
	Write(A)	

并发控制练习1

Consider following three transactions:

T1: read(A)

Read(B)

Write(B)

T2: read(B)

Read(A)

Write(A)

T3: read(A)

Write(A)

2) For the following schedule, please explain whether it is cascadeless.

T1	T2	T3
Read(A)		
Read(B)		
Write(B)		
	Read(B)	
	Read(A)	
	Write(A)	
		Read(A)
Abort		

并发控制练习1

Consider following three transactions:

T1: read(A)

Read(B)

Write(B)

T2: read(B)

Read(A)

Write(A)

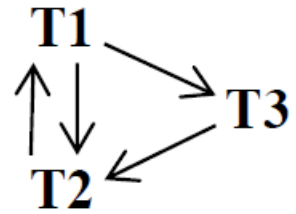
T3: read(A)

Write(A)

3) Please explain whether the two-phase locking protocol can be used to implement the schedule in 1).

并发控制练习1参考答案

1)



The schedule is not serializable, because there are cycles in the graph.

2) **The schedule is not cascadeless.**

3) **No. This is because the schedule in 1) exists cycles.**



10

错 误 恢 复

错误恢复

目标：保持数据库的一致性，事务的原子性和持久性

错误恢复算法：

- 基于日志的恢复算法
- Aries恢复算法

基于日志的恢复算法——日志

日志(log)被存储在稳定的存储中，包含一系列的日志记录

- 事务开始`<T start>`
- 写操作之前之前的日志记录`<Ti,X,V1,V2>` (X)是写的位置, V1, V2分别是写之前和之后的X处的值
- 事务结束的时候写入`<Ti commit>`

基于日志的恢复算法——恢复

单个事务回滚时的基本操作

- 从后往前扫描，当发现记录 $\langle T_i, X_i, V_1, V_2 \rangle$ 的时候
- 将 X 的值修改为原本的值
- 在日志的末尾写入记录 $\langle T_i, X_i, V_1 \rangle$
- 发现start记录的时候，停止扫描并在日志中写入abort记录

基于日志的恢复算法——恢复

- 恢复的两个阶段：redo和undo
- redo需要先找到最后一个check point并且设置undo-list
 - > 1.从checkpoint开始往下读
 - > 2.当发现修改值的记录的时候，redo一次将X设置为新的值
 - > 3.当发现start的时候将这个事务加入undo-list
 - > 4.当发现commit或者abort的时候将对应的事务从undo-list中移除
- undo
 - > 1.从日志的末尾开始往回读
 - > 2.当发现记录 $\langle T_i, X_j, V_1, V_2 \rangle$ 并且 T_i 在undo-list中的时候，进行一次回滚
 - > 3.当发现 T_i start并且 T_i 在undo-list中的时候，写入abort日志并且从undo-list中移除 T_i
 - > 4.当undo-list空了的时候停止undo

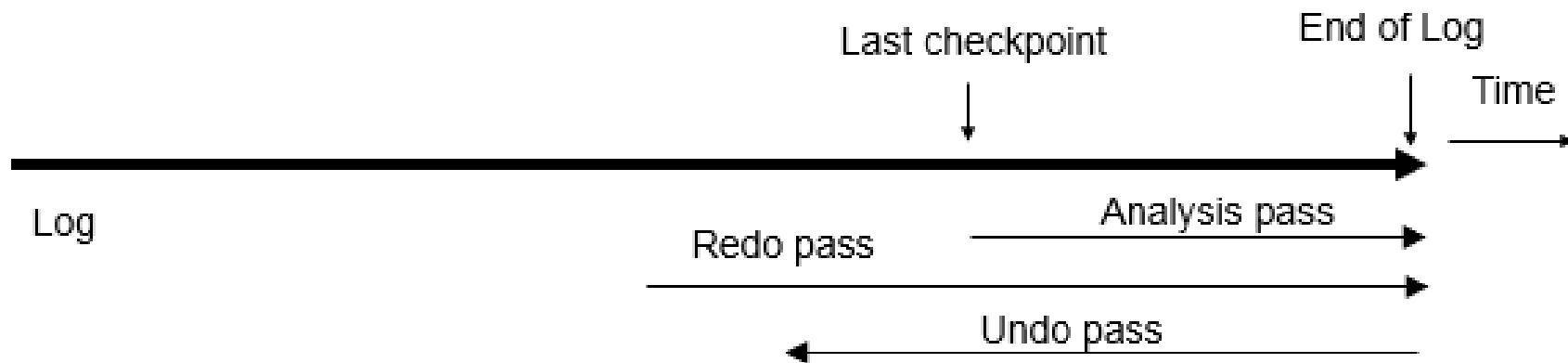
Aries恢复算法

- 使用LSN(log sequence number)来标注日志
 - 以页的形式来存储LSN来标注数据库页表中进行了哪些更新
- 使用脏页表(dirty page table) 来避免不必要的redo

LSN	TransID	PrevLSN	RedoInfo	UndoInfo
-----	---------	---------	----------	----------

Aries恢复算法——三轮遍历

- ❑ Analysis, redo and undo passes
- ❑ Analysis determines where redo should start
- ❑ Undo has to go back till start of earliest incomplete transaction



Aries恢复算法——三轮遍历

- 分析阶段：需要决定哪些事务undo，哪些页是脏页
 - - 从**最后一条完整的checkpoint日志记录**开始
 - - 读取脏页表的信息
 - - 设置RedoLSN = min RecLSN(脏页表中的)，如果脏页表是空的就设置为checkpoint的LSN
 - - 设置undo-list：checkpoint中记录的事务
 - - 读取undo-list中每一个事务的最后一条记录的LSN
 - - 从checkpoint开始正向扫描
 - - 如果发现了不在undo-list中的记录就写入undo-list
 - - 当发现一条更新记录的时候，如果这一页不在脏页表中，用该记录的LSN作为RecLSN写入脏页表中
 - - 如果发现了标志事务结束的日志记录(commit, abort) 就从undo-list中移除这个事务
 - - 搜索直到undo-list中的每一个事务都到了最后一条
 - - 分析结束之后
 - - RedoLSN决定了从哪里开始redo
 - - 所有undo-list中的事务都需要回滚

Aries恢复算法——三轮遍历

- Redo阶段
 - - 从RedoLSN开始**正向扫描**，当发现更新记录的时候
 - - 如果这一页不在脏页表中。或者这一条记录的LSN小于页面的RecLSN就忽略这一条
 - - 否则从磁盘中读取这一页，如果磁盘中得到的这一页的PageLSN比这一条要小，就redo，否则就忽略这一条记录

Aries恢复算法——三轮遍历

- Undo阶段
 - - 从日志末尾先前向前搜索，undo所有undo-list中有的事务
 - - 符合如下条件的记录可以跳过
 - - 用分析阶段的最后一个LSN来找到每个日志最后的记录
 - - 每次选择一个最大的LSN对应的事务undo
 - - 在undo一条记录之后
 - - 对于普通的记录，将NextLSN设置为PrevLSN
 - - 对于CLR记录，将NextLSN设置为UndoNextLSN
 - - 如何undo：当一条记录undo的时候
 - - 生成一个包含执行操作的CLR
 - - 设置CLR的UndoNextLSN 为更新记录的LSN

错误恢复练习1

A DBMS uses Aries algorithm for system recovery. Following figure is a log file just after system crashes. The log file consists of 14 log records with LSN from 1001 to 1014. The figure does not show PrevLSN and UndoNextLSN in log records. Assume that last completed checkpoint is the log record with LSN 1008.

Please answer following questions:

- 1) Which log record is the start point of Redo Pass?
- 2) Which log record is the end point of Undo Pass?
- 3) After Analysis Pass, what is the undo list?
- 4) After recovery, what is the value of data items identified by “102.1” and “102.2”, respectively?
- 5) What additional log records are appended to log file during recovery?

1001: <T1, begin>
1002: <T1, 101.1, 11, 21>
1003: <T2, begin>
1004: <T2, 102.1, 52, 62>
1005: <T2, commit>
1006: <T3 begin>
1007: <T3, 102.2, 73, 83>
1008: checkpoint

Tx	LastLSN
T1	1002
T3	1007

PageID	PageLSN	RecLSN
101	1002	1002
102	1007	1004

1009: <T1, 101.2, 31, 41>
1010: <T4 begin>
1011: <T3, 102.2, 73>
1012: <T3, abort>
1013: <T4, 102.1, 62, 64>
1014: <T1, commit>

错误恢复练习1参考解答

1) 1002

评分细则:

多答扣 1-3 分

2) 1010

评分细则:

多答扣 1-3 分

3) T4

评分细则:

(T4,1013) 也给分, 其余不给分

4) “102.1” = 62, “102.2” = 73

评分细则:

错一个扣 1 分, 错 2 个扣完
多一个扣 1 分, 多 2 个不给分

5)

1015: <T4, 102.1, 62>

1016: <T4, abort>

评分细则:

见 (4)

错误恢复练习2

A DBMS uses **Aries** algorithm for system recovery. Following figure is a log file just after system crashes. The log file consists of 16 log records with LSN from 1001 to 1016. The figure does not show PrevLSN and UndoNextLSN in log records. Assuming that last completed checkpoint is the log record with LSN 1012.

1001: <T1 begin>

1002: <T1 , 8001.1, 11, 22>

1003: <T2 begin>

1004: <T2 , 8001.2, 33, 44>

1005: <T3 begin>

1006: <T3, 8002.1, 55, 66 >

1008: <T1 commit>

1009: <T4 begin>

1010: <T4, 8001.1, 22, 77 >

1011: <T2, 8002.2, 88, 99>

1012: checkpoint

Txn	LastLSN
T2	1011
T3	1006
T4	1010

PageID	PageLSN	RecLSN
8001	1010	1010
8002	1011	1006

1013: <T2 commit>

1014: <T3, 8002.1, 66, 77>

1015: <T4, 8003.1, 11, 22>

1016: <T4 commit>

错误恢复练习2

Please answer following questions:

- (1) Which log record is the start point of Redo Pass?
- (2) Which log record is the end point of Undo Pass?
- (3) After Analysis Pass, what content is the dirty page table?
- (4) After recovery, what is the value of data items identified by “8002.1” and “8002.2” respectively?
- (5) After recovery, what additional log records appended to log file?

1001: <T1 begin>

1002: <T1 , 8001.1, 11, 22>

1003: <T2 begin>

1004: <T2 , 8001.2, 33, 44>

1005: <T3 begin>

1006: <T3, 8002.1, 55, 66 >

1008: <T1 commit>

1009: <T4 begin>

1010: <T4, 8001.1, 22, 77 >

1011: <T2, 8002.2, 88, 99>

1012: checkpoint

Txn	LastLSN
T2	1011
T3	1006
T4	1010

PageID	PageLSN	RecLSN
8001	1010	1010
8002	1011	1006

1013: <T2 commit>

1014: <T3, 8002.1, 66, 77>

1015: <T4, 8003.1, 11, 22>

1016: <T4 commit>

错误恢复练习2参考解答

1) 1006 (2分)

2) 1005 (2分)

3)

PageID	PageLSN	RecLSN
8001	1010	1010
8002	1014	1006
8003	1015	1015

评分细则：
少一个扣1分

4) “8002.1” = 55 (1分)

“8002.2” = 99 (1分)

5)

1017: <T3, 8002.1, 66>

1018: <T3, 8002.1, 55>

1019: <T3 abort>

评分细则：
少一个扣1分



提问与解答

总结经验

计划未来